HAILO

Empowering Intelligence

Hailo OS User Guide

Release 1.11.0

March 2026

Disclaimer and Proprietary Information Notice

Copyright

This version of the document supersedes all previous versions.

General Notice

To the fullest extent permitted by law, Hailo provides this document “as is” and disclaims all warranties, either express or implied, statutory or otherwise, including but not limited to the warranties of merchantability, non-infringement of third-party rights, and fitness for particular purposes.

This document may inadvertently contain technical inaccuracies or other errors. Hailo assumes no liability for any such errors and for damages, whether direct, indirect, incidental, consequential, or otherwise, that may result from such errors, including but not limited to loss of data or profits.

The content in this document is subject to change without notice. Hailo reserves the right to make changes to document content without notification to users.

Trademarks

All trademarks, trade names and logos of Hailo® are and shall remain the sole property of Hailo Technologies Ltd. We make use of such software, tools, systems and other components of third parties as detailed in this document. Any reference made in this document to such third party is only a description of our use or compatibility with such a third party's product, software, tool or system. No reference or attribution made by us suggests partnership, cooperation or endorsement of Hailo® or any of our products or parts thereof by such a third party. All rights, title and interest in and to the trademarks, trade names and logos of third parties included in this document are and shall remain of its legal owner.

Arm®, CoreSight™, Cortex®TrustZone® are registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.

CUDA® is a registered trademark of the Nvidia Corporation

GStreamer, gstreamer.org, the gstreamer.org logo, and all other trademarks, service marks, graphics and logos used in connection with gstreamer.org, or the Website are trademarks or registered trademarks of GStreamer or GStreamer’s licensors.

LINUX FOUNDATION and YOCTO PROJECT are registered trademarks of the Linux Foundation. Linux® is the registered trademark of Linus Torvalds in the U.S. and other countries

Version	Date	Description
0.1.0	March 2023	Preliminary release
0.1.1	June 2023	Added board customization Updated drivers Updated U-Boot to SPL
0.2.0	July 2023	Added DDR ECC feature Added SCU recovery mode Added PWM usage example
0.3.0	October 2023	Added GPIO Linux information Added SDIO Linux information Added I2S Linux information Added Watchdog timers Added SPI LSB-first
0.5.0	January 2024	Added update on yml file details Added GP DMA Added 8-bit eMMC example Updated Hailo SPI flash layout
0.6.0	February 2024	Added SPI communication with external processor Updated DDR configuration generation
0.7.0	March 2024	Updated SPI device tree example Updated SDIO storage images
1.5.0	April 2024	Document series renamed to Linux Distribution Updated SCU watchdog timeout Updated SPI flash layout Added device tree SDIO phy configuration description
0.9.0	May 2024	Added GPIO interrupt description Updated SCU Boot Configuration Added DDR bandwidth profiler
1.0.0	July 2024	Manual renamed to Hailo OS Distribution Unified DT and DDR Configuration Added Section Enabling ECC
1.1.0	September 2024	Added Linux USB description Renamed DDR bandwidth Profiler to NoC Profiler

		Added MIPI DSI to Linux Renamed bitbake recipes to 'core-image-hailo' Added reference to Supported Functions and Pin Groups Added OTA Update Guide
1.3.0	January 2025	Added how to check running image version Added Programming SPI Flash and eMMc/SD via UART
1.6.0	May 2025	Added Section Supported Functions and Pin Groups Applicable for 1.6.1 release
1.7.0	July 2025	Added new Section 3.3.2 SCU Thermal Throttling Linux Integration Introduction of differentiation between 15H and 15L products Added Section SCU Recovery Mode for 15L
1.9.0	October 2025	Introduced Hailo-15L eMMC boot partition layout, I2S and NoC counters Added Device Tree examples for Hailo-15L Updated Hailo-15 Profiler for 15H and 15L Updated SCU recovery modes for 15H and 15L Added Setting Up FTDI Updated U-Boot selection in Partition Initialization Functionality
1.9.1	December 2025	Updated Section Programming the SPI Flash Fixed the issue "Wrong DDR Configuration setting may cause boot failure"
1.10.0	February 2026	Document was revised for clarity. Profiler tool was renamed to **Hailo-15 Profiler** and its configuration was updated. SCU Recovery Modes for 15H and 15L removed, refer to respective SBC User Guide
1.10.1	February 2026	Align with SW release 1.10.1
1.11.0	March 2026	Added Section 2.3.2 U-Boot Boot flow Updated device tree example for DDR configuration as part of SCU boot Page 65

Table of Contents

List of Figures

List of Tables

1. Overview

1.1. Introduction

The **Hailo OS** is a **Linux®-based system software platform** and a core component of the **Hailo-15™ Vision Processor Software Package**. It provides the complete operating system, boot infrastructure, and build environment required to support the full device lifecycle, including bring-up, development, integration, customization, and deployment of products based on Hailo-15™ AI vision processors.

Hailo OS enables customers to:

- Bring up new hardware platforms and validate system-level functionality.
- Customize the BSP for specific products.
- Integrate Hailo-15 AI, imaging, media, and DSP pipelines on top of a supported Linux platform
- Implement secure boot, recovery, and OTA updates.
- Port and deploy existing Linux applications on Hailo-15-based systems.

This user guide focuses on system-level software components and workflows to support rapid prototyping, and product development. The guide applies to all Hailo-15 devices. However, certain aspects e.g., boot flows, storage layouts, and recovery mechanisms differ between Hailo-15H and Hailo-15L. These differences are highlighted where relevant.

1.2. Key Components

Hailo OS is composed of tightly integrated system components that together provide a complete and extensible embedded Linux platform:

- **Linux® Kernel**

A production-ready, Hailo-supported Linux kernel adapted for Hailo-15 devices. It provides core operating system services including process scheduling, memory management, interrupt handling, power management, and hardware abstraction through standard and Hailo-specific device drivers.

- **U-Boot Bootloader and SPL**

An industry-standard, open-source bootloader used to initialize system hardware, load firmware and the Linux kernel, and support multiple boot sources, recovery modes, secure boot flows, and OTA update mechanisms.

U-Boot SPL is used as the first AP-side execution stage following SCU boot.

- **System Control Unit (SCU) Firmware and Services**

A dedicated Hailo-15 microcontroller firmware responsible for critical system control functions such as power sequencing, clock management, thermal monitoring and throttling, watchdog supervision, boot orchestration, and recovery handling. The SCU exposes standardized control interfaces to U-Boot and Linux via the ARM® System Control and Management Interface (SCMI).

- **SCU Bootloader**

The SCU Bootloader is the first image running immediately following the BootROM. By using a dedicated configuration file, it selects the storage and memory offset from which to boot the SCU firmware and the next images in the boot process. SCU bootloader is also in charge of boot fallback functionality.

- **Yocto® Project Build System**

A Yocto-based build environment that defines the Hailo BSP, enabling customers to generate customized Linux images, kernels, device trees, bootloader binaries, OTA artifacts, and SDKs. The build system supports board bring-up, product differentiation, and long-term maintenance.

- **Hardware Watchdog**

The **hardware watchdog** timers provide runtime control, monitoring, and coordination.

- **Hailo-15 Libraries**

Pre-integrated libraries enabling AI inference, media pipelines, imaging, DSP task control, and multimedia frameworks, including:

  - HailoRT library
  - Hailo Media Library
  - Hailo Imaging SW package
  - Hailo DSP libraries
  - GStreamer SW package

- **Hailo Tools**

System-level development and analysis tools, including the Hailo-15 Profiler, Hailo-15 NoC Profiler, and CMA heaps monitoring utilities.

Together, these components provide a cohesive software platform for rapid prototyping, product development, and deployment on Hailo-15–based systems.

The versions of key system software components running on the device can be queried using the following system paths. See Table 1 below.
This information is useful for system debugging and software validation.

Table 1. System software version paths

File Path	Description
/sys/devices/soc0/hailo_versions/hailo_scu_fw_version	SCU firmware version
/sys/devices/soc0/hailo_versions/hailo_uboot_version	U-Boot version
/sys/devices/soc0/hailo_versions/hailo_linux_version	Hailo OS Linux version

1.3. Linux Kernel, Drivers and Device Tree

At the core of Hailo OS is an industry-standard **Linux kernel** adapted and validated for the Hailo-15™ VPU. The kernel provides the execution environment for all software running on the system and manages key functions such as task scheduling, memory allocation, interrupting handling, and device management.

To enable flexibility across different boards and designs, Hailo OS follows standard embedded Linux practices. Hardware details are not hard coded into the kernel. Instead, each system is described using a **device tree**, which defines the hardware components present on the board and how they are connected. The device tree specifies information such as memory regions, interrupts, clocks, GPIOs, and bus topology, without containing executable code.

Device drivers are kernel components that implement control logic for specific hardware blocks. Drivers are designed to be board-agnostic and obtain all board-specific configuration from the device tree at runtime.

During system **boot**, the **bootloader** provides the device tree to the Linux kernel. The kernel parses this information, discovers the available hardware, and dynamically matches devices to their corresponding drivers. This mechanism allows the same kernel and driver binaries to be reused across multiple hardware variants, simplifying maintenance and customization. The overall flow is illustrated in Figure 1 .

Note that Hailo-15 systems use separate device trees for U-Boot and Linux. Boot-time configuration such as DDR setup, SCU parameters, and early hardware initialization is defined in the U-Boot device tree, while runtime hardware configuration resides in the Linux device tree.

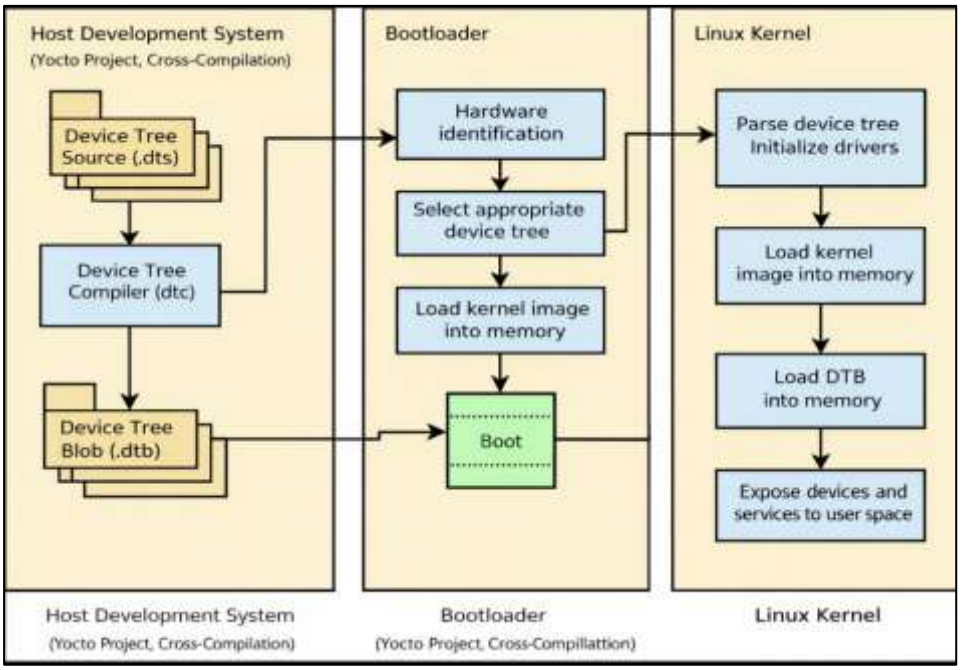


Figure 1. Building and loading Linux kernel process

Hailo provides ready-to-use software images for its Hailo-15 Single Board Computers (SBCs), allowing users to boot a complete Linux system and begin development immediately. Customers may further customize the kernel configuration, device tree, and user-space components to meet their specific product requirements.

Understanding how the Linux kernel, device drivers and device trees work together, as well as the use of the U-Boot bootloader, Hailo-15 SCU, and watchdog, is essential for effective system (hardware & software) design, customization, and debugging.

1.4. Github Repositories

Repository Name	Content	Location
Meta Hailo SoC	Top-level Yocto/OpenEmbedded meta layers (BSP, imaging, media, DSP, Linux, examples). Provides the build infrastructure and recipes required to compile full BSP images for Hailo-15–based systems using Yocto/BitBake	`https://github.com/hailo-ai/meta-hailo-soc`
Linux Yocto Hailo	Linux Kernel and Hailo-15 drivers	`https://github.com/hailo-ai/linux-yocto-hailo`
Hailo SoC Profiler	System-level profiler for Hailo-15	`https://github.com/hailo-ai/Hailo-SoC-Profiler`

1.4.1. Meta Hailo SoC Github Repository

Area	Content	Typical Usage
Yocto Meta Layers	A set of Yocto meta layers such as `meta-hailo-bsp`, `meta-hailo-bsp-examples`, `meta-hailo-dsp`, `meta-hailo-imaging`, `meta-hailo-linux`, `meta-hailo-media-library`, `meta-hailo-tappas`	Each layer encapsulates specific functionality: board support, multimedia, DSP firmware, kernel config, media libraries, AI vision apps. Used by Yocto to compose a target image
KAS Orchestration Files	A `kas/` directory with YAML configs (e.g., `hailo15-evb.yml`) that define the build setup, layer order, and BitBake environment	Used to initialize and orchestrate the Yocto build environment for specific hardware targets. Provides the entry point to start BitBake builds.
Hardware Support (BSP)	`meta-hailo-bsp` and `meta-hailo-bsp-examples` containing machine definitions, BSP recipes, board configs, and example build setups	Provide board-level support (machine definitions, bootloader/firmware integration) that defines how the OS and images are built for specific platforms
Operating System & Kernel Support	`meta-hailo-linux` layer includes recipes for the Linux kernel and OS configuration packaging	Incorporates the kernel (e.g., `linux-yocto-hailo`) and necessary Linux packages into the final image
DSP & Imaging SW packages	`meta-hailo-dsp` for DSP firmware and drivers; `meta-hailo-imaging` for image signal processing and camera sensor support	Integration of Hailo-15 DSP and Imaging SW packages into Yocto images
Media & Vision Libraries	`meta-hailo-media-library` contains media pipeline and multimedia API recipes.	Integration of media processing libraries, GStreamer plugins, and AI app frameworks into Yocto images
Documentation / Usage Files	`docs/` and `kas/` directories with build instructions and example configurations	Guidelines how to initialize the build environment (`kas build hailo15-evb.yml`), configure BitBake, and target specific hardware or image types

1.4.2. Linux Yocto Hailo Github Repository

Area	Content	Typical Usage
Top-level kernel directories	Standard Linux kernel top-level directories. e.g., `arch/`, `drivers/`, `include/`, `kernel/`, `mm/`, `net/`, `fs/`, `scripts/`, `tools/`, `Documentation/`	Navigate core subsystems; implement and customize kernel drivers
Device trees and board enablement	DTS/DTSI files and bindings (typically under `arch/arm64/boot/dts/` for ARM64 platforms; bindings under `Documentation/devicetree/bindings/`)	Add/modify board support (new boards, peripherals, pinmux, clocks, memory regions); align device trees with customer boards; validate boot and peripheral bring-up.
Hailo-specific kernel drivers	Vendor-added or modified drivers typically located under `drivers/` e.g., `drivers/media/`, `drivers/soc/`, `drivers/firmware/`, `drivers/dma/`	Enable Hailo-15 peripherals and accelerators; integrate ISP and media pipelines; support product-specific functionality
Kernel configuration	Defconfig/fragments and Kconfig changes (commonly referenced by Yocto recipes; config fragments often applied during build)	Control feature set, modules, debugging/perf options; generate stable, reproducible kernel configurations
Build helper scripts	Top-level helper files such as `build.sh` and `build.cfg.default`	Local build orchestration for developers (non-Yocto builds), quick compilation/testing,
Yocto positioning (how it fits into a Yocto flow)	Yocto Project build framework; Hailo's Yocto integration typically lives in layers (meta-layers) that reference a kernel source like this	A Yocto layer (e.g., BSP/meta layers) pins this repo+branch/tag and applies patches, Device trees' updates, and config fragments; produces kernel artifacts as part of full images/SDKs.
Documentation and developer notes	`README`, `Documentation/`	Guidance for building, supported-platforms, and Hailo-specific integration

2. Linux Components

2.1. Kernel Drivers

Hailo OS includes a comprehensive set of Linux drivers for the Hailo-15 as summarized in Table 2. These drivers are fully integrated into the Linux kernel, and most are provided as open source, enabling customers to customize and extend them. Some components may include third-party licensed IP; these are noted where applicable.

Table 2. Summary of the Linux integrated drivers

Hardware	Remarks
Connectivity	
UART	Provides standard serial port interface to hailo-15 device - used for logs, Linux console, and simple control/debug communication with external hosts or peripherals.
Ethernet	Configures and drives the on-chip MAC and its PHY to provide standard network connectivity (IPv4/IPv6, TCP/UDP, etc.) RGMII/RMII links. It hooks into the Linux net stack (net_device), uses DT for clocks, resets and pinmux, streaming video, or data offload between the Hailo-15 and the outside world.
I2C	Manages the I2C masters, providing standard Linux /dev/i2c-* interfaces and i2c_adapters used by client drivers. It is used to communicate with low-speed peripherals (e.g. camera sensors, PMICs, audio codecs, EEPROMs) defined in the device tree under each I2C bus.
QSPI (Hailo-15H) XSPI (Hailo-15L)	For Hailo15H, QSPI driver is the SPI-NOR controller used to access the external quad-SPI flash. It provides the SoC's boot and persistent storage interface, exposing the flash as an MTD device. For Hailo15L, XSPI driver plays the same role but for an "extended SPI" controller with additional IO/SDMA/aux/wrapper regions and its own IRQ. In the future, XSPI driver will provide an interface to SPI-NAND storage.
USB	Manages the Hailo USB 2.0/3.x controller and PHY, exposing standard Linux USB host/device functionality. It provides enumeration and data transfer for USB peripherals or gadget modes, using clocks and resets to power-manage the USB subsystem.

Storage	
SDIO	Configures the Hailo-15 SD/SDIO/eMMC host controllers and connects them to the Linux MMC core. It is used to access removable SD cards and the on-board eMMC device as block devices (e.g. root filesystem, data storage), handling clocking, resets and card-detect as described by the sdio* nodes in the device tree.
Video	
MIPI®-CSI-2®	CSI and camera sensors driver controls the Hailo-15 on-chip MIPI-CSI2 receiver blocks (and associated D-PHY) that bring camera pixels into the Hailo-15. It configures lanes, formats and routing, and exposes V4L2/video interfaces used by camera sensor drivers and the ISP/vision pipeline.
ISP	Multiple drivers for subsystems
MIPI-DSI	Manages the MIPI-DSI host controller and PHY when present, driving display panels over a high-speed serial link. It configures timings, lanes and pixel formats, and plugs into the DRM/KMS framework to present a standard Linux display pipeline.
Audio	
I2S and Audio Master Card	I2S driver and audio card master/slave descriptions glue the I2S controller and codecs into ALSA SoC (ASoC) machines. They define master/slave roles, routing and formats, exposing standard ALSA PCM devices for audio playback/capture over the Hailo I2S interfaces.
Platform	
ARM SCMI	ARM SCMI driver implements the System Control and Management Interface transport between Linux on the AP and the firmware/SCU. On Hailo-15 device it is used for clocks, resets, power domains, DVFS, and Hailo-specific services (boot info, fuses, throttling, identification, SKU, etc.), providing standard kernel APIs (clk, reset, etc.).
ARM PSCI	PSCI driver uses ARM's Power State Coordination Interface to perform CPU on/off, reset and system suspend/resume by invoking secure firmware calls. On Hailo-15 it is the standard way Linux hot plugs CPUs and enters low power states without hard coding SoC specific power sequences.
ARM GIC	ARM GIC driver configures the ARM Generic Interrupt

	Controller, which routes and prioritizes all interrupts from devices (Ethernet, SDIO, QSPI, etc.) to the CPUs. On Hailo-15 it's responsible for setting up interrupt lines, affinities and handling, so that all peripheral drivers can receive IRQs in a uniform way.
ARM timer	ARM timer driver sets up the Hailo-15 generic timers to provide the kernel tick, high-resolution timers and timekeeping. On the Hailo-15 it underpins jiffies, ktime_get*(), and scheduling/timeouts for all drivers and userspace.
ARM PMU	ARM PMU driver exposes the Hailo-15 Application Processor performance monitoring unit counters to the Linux perf subsystem. It allows profiling and performance analysis (cycles, cache misses, branch stats, etc.) on Hailo-15 for tuning workloads and debugging performance issues.
pl320_mbox	PL320 mailbox driver controls the ARM PL320 hardware mailbox block used for inter-processor communication. On Hailo-15 it provides mailbox channels (via the generic mailbox framework) between the AP and other firmware/cores (e.g. NNC/DSP), enabling lightweight message/interrupt-based coordination.
EDAC	EDAC (Error Detection and Correction) driver monitors memory and certain internal buses for ECC and parity errors. On Hailo-15 it reads error counters and status registers (from the edac block) and reports corrected/uncorrected errors via standard EDAC interfaces for reliability monitoring.
PVT	PVT (Process-Voltage-Temperature) driver reads on-chip sensors that report die temperature and sometimes supply or process indicators. On Hailo-15 it uses SCMI sensor indices to expose PVT readings via hwmon or similar interfaces, enabling thermal management and health monitoring.
Voltage Regulator	Controls internal or external regulators (often via I2C/SCMI) that power different SoC domains. On Hailo-15 device they integrate with the Linux regulator framework so consumers (NNC, cameras, etc.) can request/adjust voltages and power states abstractly.
PWM	Handles the on-chip PWM controllers that generate variable-duty-cycle outputs. On Hailo-15 it exposes standard Linux PWM channels that can be used for LED dimming, fan control, backlight brightness, or other duty-cycle-controlled signals.
Watchdog	Programs a hardware timer that reboots the system if it is not periodically "kicked" by software. On Hailo-15 it provides an interface that system daemons use to ensure recovery from hangs or software deadlocks.
GP VDMA	Configures the general-purpose video/VDMA engine (gp_vdma node) to perform memory-to-memory or

	peripheral DMA transfers. On Hailo-15 it is used to offload bulk data movement from the CPU, integrating with the DMA engine framework.
Pincntr	Configures the Hailo SoC pin muxing and pad control blocks. It lets other drivers select pin functions (GPIO vs peripheral), drive strength, pulls, and other pad settings through the kernel pinctrl and pinconf frameworks.
Gyro	Supports a connected gyroscope (via I2C/SPI) and plugs into the IIO framework. Used on Hailo-15 based system to read angular rate data for user-space applications.

2.1.1. SDIO (Secure Digital Input/Output)

Synopsys Proprietary. Used with permission

2.1.1.1. Overview

SDIO is supported under Linux and was integrated for use as standard storage device with a file system.

The SDIO controller device properties are exposed through the device files under the directory:

/sys/class/mmc_host/mmc[N]/

Where [N] represents the SDIO controller number.

2.1.1.2. SD Voltage Select Pin

To enable higher frequencies in the SD card, it is necessary to switch the voltage (if supported by the SD card). The voltage switch can be indicated externally on the board by a GPIO pin.

Enable the voltage switch by setting the following properties in the device tree's SDIO node:

- U-Boot:

```
sd-vsel-gpio-pin = <22>; // pin 22 is assigned for voltage switch
enable-sd-vsel-gpio; // enable voltage switch
sd-vsel-polarity-high // gpio set to 1 indicates 1.8v
```

- Linux:

```
sd-vsel-gpios = <&gpio1 6 GPIO_ACTIVE_HIGH>; //pin assigned for voltage switch
sd-vsel-polarity-high // gpio set to 1 indicates 1.8v
```

2.1.1.3. Device Tree Examples

The example below shows a Linux U-Boot SDIO device tree which SDIO0 is connected to an SD Card and SDIO1 is connected to an eMMC device.

```
&sdio0 { // SD card example
  status = "okay";
  phy-config { // phy configuration - board specific
    card-is-emmc = <0x0>;
     bus-width = <4>; // sdio0 operate with 4 lanes
    cmd-pad-values = <0x1 0x3 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                dat-pad-values = <0x1 0x3 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                rst-pad-values = <0x1 0x3 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                clk-pad-values = <0x1 0x3 0x0 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                sdclkdl-cnfg = <0x0 0x32>; //extdly_en, cckdl_dc
                drive-strength = <0xC 0xC>; //pad_sp, pad_sn
     sd-vsel-gpios = <&gpio1 6 GPIO_ACTIVE_HIGH>; //pin assigned for voltage switch
     sd-vsel-polarity-high // gpio set to 1 indicates 1.8v
  };
};

&sdio1 { // eMMC example
  non-removable; // in EVB sdio1 is eMMC so non-removable should be enabled
  status = "okay";
  bus-width = <4>; // sdio1 can operate with 4 or with 8 lanes. Once sdio1 bus-width is 8
sdio0 disabled
  phy-config { // phy configuration - board specific
    card-is-emmc = <0x1>;
                cmd-pad-values = <0x2 0x2 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                dat-pad-values = <0x2 0x2 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                rst-pad-values = <0x2 0x2 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                clk-pad-values = <0x2 0x2 0x0 0x0>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
                sdclkdl-cnfg = <0x0 0x32>; //extdly_en, cckdl_dc
                drive-strength = <0xC 0xC>; //pad_sp, pad_sn
     };
};
```

The example below shows a Linux U-Boot SDIO device tree which SDIO0 is disabled and SDIO1 is connected to an eMMC device in 8-bit mode.

```
&sdio0 { // SD card example
  // sdio0 disabled
};
&sdio1 { // eMMC example
  non-removable; // in EVB sdio1 is eMMC so non-removable should be enabled
  status = "okay";
  bus-width = <8>; // sdio1 is 8 lanes
  phy-config { // phy configuration - board specific
    card-is-emmc = <0x1>;
        cmd-pad-values = <0x2 0x2 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
        dat-pad-values = <0x2 0x2 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
        rst-pad-values = <0x2 0x2 0x1 0x1>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
        clk-pad-values = <0x2 0x2 0x0 0x0>; // txslew_ctrl_n, txslew_ctrl_p,
weakpull_enable, rxsel
        sdclkdl-cnfg = <0x0 0x32>; //extdly_en, cckdl_dc
        drive-strength = <0xC 0xC>; //pad_sp, pad_sn
    };
};
```

Reference: https://www.sdcard.org/downloads/pls/

2.1.1.4. Device Tree SDIO Phy Configuration

The SDIO device tree exposes phy configuration registers. Each register exposes its fields as values. See Table 3.

For SDIO phy registers description please refer to the Hailo-15 datasheet.

Table 3. SDIO Phy device tree configuration

Device Tree Entry	Corresponding Register	Register Value Sequence
card-is-emmc	EMMC_CTRL_R	CARD_IS_EMMC
cmd-pad-values	CMDPAD_CNFG	TXSLEW_CTRL_N TXSLEW_CTRL_P WEAKPULL_EN RXSEL
dat-pad-values	DATPAD_CNFG	TXSLEW_CTRL_N TXSLEW_CTRL_P WEAKPULL_EN

		RXSEL
rst-pad-values	CLKPAD_CNFG	TXSLEW_CTRL_N TXSLEW_CTRL_P WEAKPULL_EN RXSEL
clk-pad-values	RSTNPAD_CNFG	TXSLEW_CTRL_N TXSLEW_CTRL_P WEAKPULL_EN RXSEL

See below an example for Phy configuration:

```
phy-config { // phy configuration - board specific
        // < [card_is_emmc] >
        card-is-emmc = <0x1>;
        // < [txslew_ctrl_n] [txslew_ctrl_p] [weakpull_enable] [rxsel] >
        cmd-pad-values = <0x2 0x2 0x1 0x1>;
        // < [txslew_ctrl_n] [txslew_ctrl_p] [weakpull_enable] [rxsel] >
        dat-pad-values = <0x2 0x2 0x1 0x1>;
        // < [txslew_ctrl_n] [txslew_ctrl_p] [weakpull_enable] [rxsel] >
        rst-pad-values = <0x2 0x2 0x1 0x1>;
        // < [txslew_ctrl_n] [txslew_ctrl_p] [weakpull_enable] [rxsel] >
        clk-pad-values = <0x2 0x2 0x0 0x0>;
```

2.1.2. SPI (Serial Peripheral Interface)

SPI bus is supported in master mode and can be used by the standard Linux kernel API. To load the driver by its kernel, it is recommended to define the specific SPI slave in the device tree.

SPI slaves which are not flash devices must be marked in the device tree as

"*hailo,non-flash-device;*"

By default, transfers are MSB first. This can be changed to LSB-first by adding

to the SPI device in the device tree:

"spi-lsb-first;"

By default, spi works with 4 Byte address mode, with single lane of communication. Enabling quad mode can be done by changing bus width in SPI device in the device tree:

"spi-tx-bus-width = <4>;

spi-rx-bus-width = <4>;"

The SPI slaves are represented by:

/sys/bus/spi/devices/spi0.[N]*/*

Where [N] represents the slave CS number.

2.1.2.1. Device Tree Example

Example for a non-flash device called 'test_device' connected to a QSPI flash controller with CS1 and the kernel driver 'hailo-spi-slave-test'.

```
&qspi {
        test_device@1 {
        compatible = "hailo,hailo-spi-slave-test";
        spi-max-frequency = <6250000>; // 6.25 MHz
        reg = <1>;
        hailo,non-flash-device;
};
```

2.1.2.2. SPI Slave Processor Communication

The Hailo-15 SPI flash controller can be used to communicate with external processors that act as SPI slave devices. In this configuration, the external processor responds using a flash-like protocol.

When implementing SPI slave communication with Hailo-15, follow these guidelines.

Hailo-15 behavior

1. **Transmitting data from Hailo-15:** Each message must begin with a synchronization byte that is not 0x03, 0x05 or 0x9f (for example 0xaa may be used)

2. **Receiving data on Hailo-15:** Each message must begin with a synchronization byte equal to 0x9f

Examples of transmitting and receiving data from Hailo-15 are shown below.

See below an example for **Transmitting data from Hailo-15:**

```
int hailo_spi_send(int fd, uint8_t *buf, uint32_t buf_size)
{
    struct spi_ioc_transfer tr[2];
    uint8_t tx_sync_byte = 0xAA; // must not be 0x03, 0x05, 0x9F
    int ret;

    memset(tr, 0, sizeof(tr));

    tr[0].tx_buf = (uintptr_t)& tx_sync_byte;
    tr[0].len = 1;

    tr[1].tx_buf = (uintptr_t)buf;
    tr[1].len = buf_size;

    ret = ioctl(fd, SPI_IOC_MESSAGE(2), tr);
    return ret;
}
```

See below an example for **Receiving data on Hailo-15**

```
int hailo_spi_recv(int fd, uint8_t *buf, uint32_t buf_size)
{
    struct spi_ioc_transfer tr[2];
    uint8_t rx_sync_byte = 0x9F; // must be 0x9F
    int ret;

    memset(tr, 0, sizeof(tr));

    tr[0].tx_buf = (uintptr_t)&rx_sync_byte;
    tr[0].len = 1;

    tr[1].rx_buf = (uintptr_t)buf;
    tr[1].len = buf_size;

    ret = ioctl(fd, SPI_IOC_MESSAGE(2), tr);
    return ret;
}
```

External SPI slave processor behavior

1. **Receiving data:**

Read and validate the synchronization byte (for example, 0xAA), then read the remaining message bytes.

2. **Sending data:**

Read the synchronization byte from Hailo-15, then transmit the response data in the subsequent byte transfers.

pseudo-code examples for the external SPI slave processor transmit and receive flows are provided below.

Slave processor send example (pseudo):

```
void spislave_send(uint8_t *buf, uint32_t buf_size)
{
    uint8_t sync_byte;

      sync_byte = spislave_read();
      spislave_reply(buf, buf_size);
}
```

Slave processor receives example (pseudo)

```
void spislave_recv(uint8_t *buf, uint32_t buf_size)
{
    uint8_t sync_byte;

      sync_byte = spislave_read();
      spislave_read(buf, buf_size);
}
```

2.1.3. GPIO (General Purpose Input/Output)

2.1.3.1. Overview

GPIO is supported in Linux using the standard GPIO subsystem 'gpiolib'. Refer to block diagram in Figure 2. It can be controlled by the standard Linux GPIO API and includes the standard Linux GPIO device files.

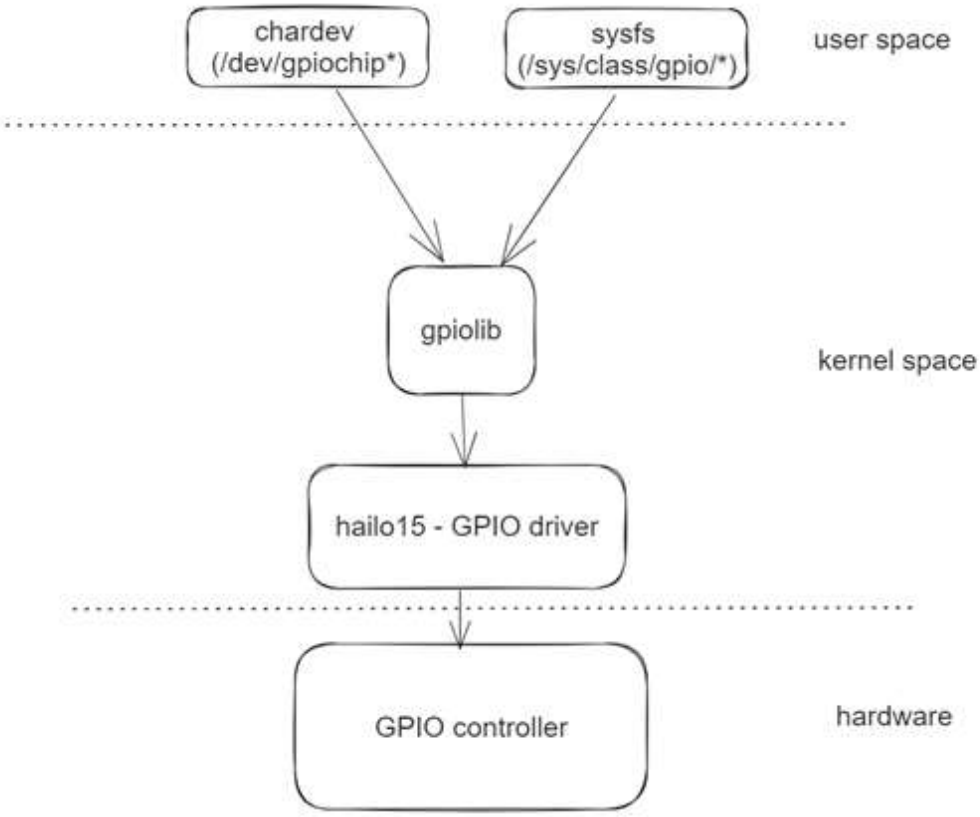


Figure 2. Linux GPIO subsystem architecture

Each GPIO controller is represented by the path:

/dev/gpiochip[N]

Where [N] represents the GPIO controller number.

The standard tools *gpioinfo*, *gpioset*, and *gpioget* are also supported.

Figure 3. shows an example of a command screen with the inclusion of the gpio tool sets.

```
root@hailo15:~# gpioinfo 0
gpiochip0 - 16 lines:
        line   0: "gpio_in_out_0" unused input active-high
        line   1: "gpio_in_out_1" unused input active-high
        line   2:      "pwm_2"       kernel   input  active-high [used]
        line   3:      "pwm_3"       kernel   input  active-high [used]
        line   4: "cam0_reset_n" unused input active-high
        line   5: "gpio_in_out_5" unused input active-high
        line   6: "i2c2_sda_in_out" kernel input active-high [used]
        line   7: "i2c2_scl_out" kernel input active-high [used]
        line   8: "uart2_rxd_pad_in" kernel input active-high [used]
        line   9: "uart2_txd_pad_out" kernel input active-high [used]
        line  10: "uart0_cts_pad_in" unused input active-high
        line  11: "uart0_rts_pad_out" unused input active-high
        line  12: "uart3_rxd_pad_in" kernel input active-high [used]
        line  13: "uart3_txd_pad_out" kernel input active-high [used]
        line  14: "gpio_in_out_14" unused input active-high
        line  15: "pcie_mperst_out" unused input active-high
root@hailo15:~# gpioset 0 14=1
root@hailo15:~# gpioget 0 14
0
```

Figure 3. Linux GPIO commands example

2.1.3.2. Device Tree Example

Named GPIO pins:

```
&gpio0 {
    gpio-ranges = <&pinctrl 0 0 16>;

    gpio-line-names =
        "gpio_in_out_0",
        "gpio_in_out_1",
        "pwm_2",
        "pwm_3",
        "cam0_reset_n",
        "gpio_in_out_5",
        "i2c2_sda_in_out",
        "i2c2_scl_out",
        "uart2_rxd_pad_in",
        "uart2_txd_pad_out",
        "uart0_cts_pad_in",
        "uart0_rts_pad_out",
        "uart3_rxd_pad_in",
        "uart3_txd_pad_out",
        "gpio_in_out_14",
        "pcie_mperst_out";
};
```

Reference

https://docs.kernel.org/driver-api/gpio/driver.html

2.1.3.3. GPIO Interrupts

GPIO interrupts are supported under Linux using the standard interrupt subsystem. It can be controlled using the standard Linux Interrupt Handling API.

To register to an interrupt on a GPIO pin, the following properties must be set in the device tree:

- interrupt-parent – A reference to the node of the GPIO bank associated with the GPIO pin. Either "gpio0" (for pins 0-15) or "gpio1" (for pins 16-31).
- interrupts – a tuple of the relative pin offset and flags. The offset is expected to be in 0-15 range. The offset of pins 0-15 is the same as their pin number. The offset of pin 16 is 0, of pin 17 is 1 and so on.
- Pinctrl – names
- pinctrl-0 – a reference to the device tree node associated with the requested pin.

For example, the following device tree configuration allows triggering an IRQ handler function each time GPIO4 is asserted:

```
&my_node {
        my_gpio_triggered_device_1: my_gpio_triggered_device@0
            ...
            interrupt-parent = <&gpio0>;
            interrupts = <4 IRQ_TYPE_EDGE_RISING>;
            pinctrl-names = "default";
            pinctrl-0 = <&pinctrl_gpio4>;
        };
};

&pinctrl {
        pinctrl_gpio4: gpio4 {
                function = "gpio";
                groups = "gpio4_grp";
        };
};
```

2.1.4. PWM (Pulse Width Modulation)

PWM is supported under Linux using the standard PWM subsystem. It can be controlled through the standard Linux PWM API and includes the standard Linux PWM device files.

Each PWM channel is represented by the path:

/sys/class/pwm/pwmchip0/pwm[N]

Where [N] represents the PWM channel number.

Table 4. PWM Linux device files

Function	Device file
Export PWM channels to sysfs	/sys/class/pwm/pwmchip0/export
Enable / disable channel	/sys/class/pwm/pwmchip0/pwm0/enable
Period	/sys/class/pwm/pwmchip0/pwm0/period
Duty cycle	/sys/class/pwm/pwmchip0/pwm0/duty_cycle

For example, to enable a PWM on channel 0 with 10 millisecond period and 4 millisecond duty cycle with terminal commands:

```
echo 0 > /sys/class/pwm/pwmchip0/export
echo 10000000 > /sys/class/pwm/pwmchip0/pwm0/period
echo 4000000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

Please note that if more than one channel is enabled, a time period cannot be set for any PWM.

2.1.5. USB (Universal Serial Bus)

Device tree nodes configuration is required to activate the USB interface:

‘cdns_usb3’ - USB3 driver (relevant also for USB2).

‘cdns_torrent_phy’ - PCIe/USB torrent phy driver.

‘hailo_torrent_phy’ for the Hailo torrent phy wrapper, needs to be enabled when using USB3 (with status = “okay”), there is no need to enable it when using only USB2. This node also expects the following properties: ‘usb-lane’, 'usb-lange-pma-pll-full-rate-divider, lanes-config.

```
&hailo_torrent_phy {
  status = "okay";
  usb-lane = <3>;
  usb-lane-pma-pll-full-rate-divider = <PCI_PMA_PLL_FULL_RATE_CLK_DIVIDER_4>;
  lanes-config = <PCI_PHY_LINK_LANES_CFG__2x1_PLUS_1x2__MASTER_LANES_LN0_LN2_LN3>;
};

&cdns_torrent_phy {
  /* same as &torrent_phy node previously, node renamed to &cdns_torrent_phy */
  ...
};

&hailo_usb3 {
  status = "okay";
};

&cdns_usb3 {
  /* same as &usb3 node previously, node renamed to &cdns_usb3 */
};
```

To enable USB2 only:

1. Do not enable ‘&hailo_torrent_phy’, and ‘&cdns_torrent_phy’
2. Do not link to the torrent phy in the ‘&cdns_usb3’ node. **Don’t** set the following:

```
phys = <&torrent_phy_usb3>;
phy-names = "cdns3,usb3-phy";
```

3. Add the following to &cdns_usb3 node:

```
maximum-speed = "high-speed";
```

If the USB overcurrent line is not used, it is required to use this property in Hailo's USB driver wrapper for the USB interface to work:

```
&hailo_usb3{
disconnected-overcurrent;
};
```

2.1.6. I2S (Inter-IC Sound)

2.1.6.1. Overview

The Hailo-15 I2S driver is integrated into the standard Linux sound subsystem and is accessed through ALSA. It supports both audio playback and audio capture.

Audio device files are located under the standard Linux directory:

/dev/snd/

Table 5. I2S Linux device files

Function	Device file
Card 0 control	/dev/snd/controlC0
Card 0 Device 0 for capture audio channel	/dev/snd/pcmC0D0c
Card 0 Device 0 for playback audio	/dev/snd/pcmC0D0p

2.1.6.2. Usage

The standard Linux 'alsa-utils' package provides utilities for controlling and testing Audio devices as shown below.

- List playback devices: aplay -l

  Playback example:

Signed 16 bit Little Endian, Rate 8000 Hz, Stereo:
- **aplay** -v /tmp/recording-8000.wav
Signed 16 bit Little Endian, Rate 44100 Hz, Stereo:
- **aplay** -v /tmp/recording-44100.wav
Signed 16 bit Little Endian, Rate 48000 Hz, Stereo:
- **aplay** -v /tmp/recording-48000.wav

- List capture devices: arecord -l

Recording examples (10 seconds):

Signed 16 bit Little Endian, Rate 8000 Hz, Stereo:
- arecord -vvv -f S16_LE -r 8000 -c2 -t wav /tmp/recording-8000.wav -d10
Signed 16 bit Little Endian, Rate 44100 Hz, Stereo:
- arecord -vvv -f S16_LE -r 44100 -c2 -t wav /tmp/recording-44100.wav -d10
Signed 16 bit Little Endian, Rate 48000 Hz, Stereo:
- arecord -vvv -f S16_LE -r 48000 -c2 -t wav /tmp/recording-48000.wav -d10

2.1.6.3. Hailo-15H Notes:

Hailo-15H supports three exclusive modes of operation:

- Master mode (capable to capture and playback)
- Slave Rx (only capable to capture)
- Slave Tx (only capable to playback)

Only one of I2S operating mode can operate at any given time.

The underlying hardware always operates with the following fixed parameters:

- 16bit sample width
- Little-Endian format
- 48 KHz sample rate

2.1.6.4. Device Tree Properties

I2s_cpu_master node properties:

Recording:

- `rx-sample-pace:`
  - cyclic samples offset pattern
    - E.g.: `rx-sample-pace = <59 59 58>`, includes samples
      - sample[any-offset+59]

        - sample[any-offset+59+59]
        - sample[any-offset+59+59+58]
- `rx-sample-pace-pattern-sync-times`:
    - required number of consecutive times of cyclic samples offset pattern for syncing on starting offset that Hailo driver start to act.
        - E.g.: `rx-sample-pace-pattern-sync-times = <2>` includes 7 samples to acquire synchronization
            - sample[any-offset]
            - sample[any-offset+59]
            - sample[any-offset+59+59]
            - sample[any-offset+59+59+58]
            - sample[any-offset+59+59+58+59]
            - sample[any-offset+59+59+58+59+59]
            - sample[any-offset+59+59+58+59+59+58]
- `rx-sample-cmp-to` : comparison options for cyclic samples offset pattern synchronization:
    - "**zero**" : Compare current sample to zero.
    - "**prev-val**" : Compare current sample to the previous sample.

Playback:

- `tx-sample-pace`:
    - cyclic samples offset pattern, should fill as `rx-sample-pace = <59 59 58>` minus 1.
        - E.g.: if `rx-sample-pace = <59 59 58>`, then `is rx-sample-pace = <58 58 57>`.
        - Included samples
            - sample[any-offset+58]
            - sample[any-offset+58+58]
            - sample[any-offset+58+58+57]
- `tx-sample-offset` :
    - Starting sample offset for sampling duplication.
    - E.g.: `tx-sample-offset = <0>;`
        - duplication samples:
            - sample[0+58]
            - sample[0+58+58]
            - sample[0+58+58+57]

2.1.6.5. Device Tree Examples for Hailo-15H

Master mode with TI TLV320AIC3104 CODEC:

```
& _1 {
   status = "okay";
   codec_tlv320aic3104: tlv320aic3104@18 {
      #sound-dai-cells = <0>;
      compatible = "ti,tlv320aic3104";
      reg = <0x18>;
      ai3x-source-clk = "bclk";
      ai3x-micbias-vg = <2>;
       /* Regulators */
       AVDD-supply = <&regulator_3p3v>;
       IOVDD-supply = <&regulator_1p8v>;
       DRVDD-supply = <&regulator_3p3v>;
       DVDD-supply = <&regulator_1p8v>;
   };
};

&i2s_cpu_master {
   status = "okay";
   rx-sample-pace-pattern-sync-times = <2>;
   rx-sample-pace = <64 64 64>;
   rx-sample-cmp-to = "prev-val";
   tx-sample-offset = <81>;
   tx-sample-pace = <63>;
};

&audio_card_master {
   status = "okay";
   simple-audio-card,bitclock-master = <&cpu_dai_master>;
   simple-audio-card,frame-master = <&cpu_dai_master>;
   cpu_dai_master: simple-audio-card,cpu {
      sound-dai = <&i2s_cpu_master>;
      system-clock-frequency = <1562500>;
      system-clock-direction-out;
   };
   codec_dai_master: simple-audio-card,codec {
      sound-dai = <&codec_tlv320aic3104>;
      system-clock-frequency = <1562500>;
   };
};
```

Master mode using Realtek ALC5660 CODEC:

```
/dts-v1/;

#include "../hailo/hailo15-base.dtsi"

&i2c_2 {
   status = "ok";
   codec_rt5660: rt5660@1c {
      #sound-dai-cells = <0>;
      compatible = "realtek,rt5660";
      reg = <0x1c>;
      rt-sysclk-src = "pll1";
      rt-pll-src = "bclk";
   };
};

&i2s_cpu_master {
   status = "ok";
   rx-sample-pace-pattern-sync-times = <1>;
   rx-sample-offset = <0>;
   rx-sample-pace = <59 59 58>;
   rx-sample-cmp-to = "zero";
   tx-sample-offset = <58>;
   tx-sample-pace = <58 58 57>;
};

&audio_card_master {
   status = "ok";
   simple-audio-card,bitclock-master = <&cpu_dai_master>;
   simple-audio-card,frame-master = <&cpu_dai_master>;

   cpu_dai_master: simple-audio-card,cpu {
      sound-dai = <&i2s_cpu_master>;
      system-clock-frequency = <1562500>;
      system-clock-direction-out;
   };
   codec_dai_master: simple-audio-card,codec {
      sound-dai = <&codec_rt5660>;
      system-clock-frequency = <1562500>;
   };
};
```

Slave mode RX with ALC5660 CODEC:

```
/dts-v1/;

#include "../hailo/hailo15-base.dtsi"

&i2c_2 {
   status = "ok";
   codec_rt5660: rt5660@1c {
           #sound-dai-cells = <0>;
           compatible = "realtek,rt5660";
           reg = <0x1c>;
   };
};

&i2s_cpu_slave_rx {
   status = "okay";
};

&audio_card_slave_rx {
   status = "okay";
   simple-audio-card,bitclock-master = <&codec_dai_rx>;
   simple-audio-card,frame-master = <&codec_dai_rx>;

   cpu_dai_rx: simple-audio-card,cpu {
      sound-dai = <&i2s_cpu_slave_rx>;
      system-clock-frequency = <12288000>;
   };
   codec_dai_rx: simple-audio-card,codec {
      sound-dai = <&codec_rt5660>;
      system-clock-frequency = <12288000>;
   };
};
```

Slave mode TX with ALC5660 CODEC:

```
/dts-v1/;

#include "../hailo/hailo15-base.dtsi"

&i2c_2 {
   status = "ok";
   codec_rt5660: rt5660@1c {
      #sound-dai-cells = <0>;
      compatible = "realtek,rt5660";
      reg = <0x1c>;
   };
```

```
};

&i2s_cpu_slave_tx {
  status = "okay";
};

&audio_card_slave_tx {
  status = "okay";
  simple-audio-card,bitclock-master = <&codec_dai_tx>;
  simple-audio-card,frame-master = <&codec_dai_tx>;

  cpu_dai_tx: simple-audio-card,cpu {
    sound-dai = <&i2s_cpu_slave_tx>;
    system-clock-frequency = <12288000>;
  };
  codec_dai_tx: simple-audio-card,codec {
    sound-dai = <&codec_rt5660>;
    system-clock-frequency = <12288000>;
  };
};
```

2.1.6.6. Device Tree Examples for Hailo-15L

The default I2S operation mode is:

- Slave mode
- Defined in hailo15l-audio.dtsi
- Using 12288000 Hz external oscillators to CODEC

Master common DTS node properties:

```
&i2c_1 {
      status = "okay";
      i2c-sda-falling-time-ns = <1000>;
      pinctrl-names = "default";
      pinctrl-0 = <&pinctrl_i2c1_scl>,
                  <&pinctrl_i2c1_sda>;
      codec_tlv320aic3104: tlv320aic3104@18 {
            #sound-dai-cells = <0>;
            ai3x-source-clk = "bclk";
            ai3x-micbias-vg = <2>;
            AVDD-supply = <&regulator_3p3v>;
```

```
            IOVDD-supply = <&regulator_1p8v>;
            DRVDD-supply = <&regulator_3p3v>;
            DVDD-supply = <&regulator_1p8v>;
      };
      };
```

Master using SoC PCLK:

```
&i2s_hailo {
    status = "okay";
    i2s_0_cpu: audio-controller-0@10d000 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s0_sck>,
                    <&pinctrl_i2s0_sdi>,
                    <&pinctrl_i2s0_sdo>,
                    <&pinctrl_i2s0_ws>;
         HAILO_I2S_CTRL_CFG(0, PCLK);
    };
};

&audio_card_0 {
      status = "okay";
      SIMPLE_AUDIO_CARD_CFG(0, MASTER);
      cpu_dai_0: simple-audio-card,cpu {
            CPU_DAI_CFG(0, MASTER);
      };
      codec_dai_0: simple-audio-card,codec {
            sound-dai = <&codec_tlv320aic3104>;
            CODEC_DAI_CFG(0, MASTER);
      };
};
```

Master using SoC XTAL:

```
&i2s_hailo {
    status = "okay";
    i2s_0_cpu: audio-controller-0@10d000 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s0_sck>,
                    <&pinctrl_i2s0_sdi>,
                    <&pinctrl_i2s0_sdo>,
                    <&pinctrl_i2s0_ws>;
         HAILO_I2S_CTRL_CFG(0, XTAL);
    };
};

&audio_card_0 {
      status = "okay";
      SIMPLE_AUDIO_CARD_CFG(0, MASTER);
      cpu_dai_0: simple-audio-card,cpu {
            CPU_DAI_CFG(0, MASTER);
      };
      codec_dai_0: simple-audio-card,codec {
            sound-dai = <&codec_tlv320aic3104>;
            CODEC_DAI_CFG(0, MASTER);
      };
};
```

Master using SoC FAST_ACLK:

```
&i2s_hailo {
    status = "okay";
    i2s_0_cpu: audio-controller-0@10d000 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s0_sck>,
                    <&pinctrl_i2s0_sdi>,
                    <&pinctrl_i2s0_sdo>,
```

```
                    <&pinctrl_i2s0_ws>;
          HAILO_I2S_CTRL_CFG(0, FAST_ACLK);
    };
};

&audio_card_0 {
      status = "okay";
      SIMPLE_AUDIO_CARD_CFG(0, MASTER);
      cpu_dai_0: simple-audio-card,cpu {
            CPU_DAI_CFG(0, MASTER);
      };
      codec_dai_0: simple-audio-card,codec {
            sound-dai = <&codec_tlv320aic3104>;
            CODEC_DAI_CFG(0, MASTER);
      };
};
```

Master using SoC EXTERNAL MCLK (12288000 Hz):

```
&i2s_hailo {
    status = "okay";
    i2s_0_cpu: audio-controller-0@10d000 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s0_sck>,
                    <&pinctrl_i2s0_sdi>,
                    <&pinctrl_i2s0_sdo>,
                    <&pinctrl_i2s0_ws>;
         HAILO_I2S_CTRL_CFG(0, EXTERNAL_MCLK, 12288000);
    };
};

&audio_card_0 {
      status = "okay";
      SIMPLE_AUDIO_CARD_CFG(0, MASTER);
      cpu_dai_0: simple-audio-card,cpu {
```

```
            CPU_DAI_CFG(0, MASTER);
      };
      codec_dai_0: simple-audio-card,codec {
            sound-dai = <&codec_tlv320aic3104>;
            CODEC_DAI_CFG(0, MASTER);
      };
};
```

Slave using Codec external MCLK (12288000 Hz):

```
&i2c_1 {
      status = "okay";
      i2c-sda-falling-time-ns = <1000>;
      pinctrl-names = "default";
      pinctrl-0 = <&pinctrl_i2c1_scl>,
                  <&pinctrl_i2c1_sda>;
      codec_tlv320aic3104: tlv320aic3104@18 {
            #sound-dai-cells = <0>;
            ai3x-source-clk = "mclk";
            ai3x-micbias-vg = <2>;
            AVDD-supply = <&regulator_3p3v>;
            IOVDD-supply = <&regulator_1p8v>;
            DRVDD-supply = <&regulator_3p3v>;
            DVDD-supply = <&regulator_1p8v>;
      };
};
&i2s_hailo {
    status = "okay";
    i2s_0_cpu: audio-controller-0@10d000 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s0_sck>,
                    <&pinctrl_i2s0_sdi>,
                    <&pinctrl_i2s0_sdo>,
                    <&pinctrl_i2s0_ws>;
    };
```

```
};

&audio_card_0 {
      status = "okay";
};
```

Slave using Codec external MCLK (24 MHz):

```
&i2c_1 {
      status = "okay";
      i2c-sda-falling-time-ns = <1000>;
      pinctrl-names = "default";
      pinctrl-0 = <&pinctrl_i2c1_scl>,
                  <&pinctrl_i2c1_sda>;
      codec_tlv320aic3104: tlv320aic3104@18 {
            #sound-dai-cells = <0>;
            ai3x-source-clk = "mclk";
            ai3x-micbias-vg = <2>;
            AVDD-supply = <&regulator_3p3v>;
            IOVDD-supply = <&regulator_1p8v>;
            DRVDD-supply = <&regulator_3p3v>;
            DVDD-supply = <&regulator_1p8v>;
      };
};
&i2s_hailo {
    status = "okay";
    i2s_0_cpu: audio-controller-0@10d000 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s0_sck>,
                    <&pinctrl_i2s0_sdi>,
                    <&pinctrl_i2s0_sdo>,
                    <&pinctrl_i2s0_ws>;
        HAILO_I2S_CTRL_CFG(0, SLAVE_CLK);
    };
};
```

```
&audio_card_0 {
	status = "okay";
	SIMPLE_AUDIO_CARD_CFG(0, SLAVE);
	cpu_dai_0: simple-audio-card,cpu {
		CPU_DAI_CFG(0, SLAVE, 24000000);
	};
	codec_dai_0: simple-audio-card,codec {
		sound-dai = <&codec_tlv320aic3104>;
		CODEC_DAI_CFG(0, SLAVE, 24000000);
	};
};
```

2.1.7. GP DMA (General Purpose Direct Memory Access)

2.1.7.1. Overview

The Hailo-15 includes a hardware DMA engine that enables high-performance data transfers to and from external DDR device without CPU intervention. The Hailo GP DMA driver is integrated with the standard Linux DMA Engine subsystem, allowing it to be used through well-defined kernel APIs.

The driver builds scatter-gather lists and dispatches the DMA transactions to perform memory copy operations efficiently.

Both the Hailo GP DMA driver and the Linux DMA Engine subsystem operate on physical addresses. When working with virtual addresses, address translation must be performed before initiating DMA operations.

DMA Engine APIs are designed to be invoked from **kernel space**. As a result, DMA operations must be initiated by a kernel module or a device driver that acts as an intermediary between user space and the DMA Engine subsystem.

2.1.7.2. Device Tree Example

The device tree defines the DMA controller and channels.

Example for enabling the GP DMA controller and the first 4 of the 16 channels.

```
gp_vdma: dma@7c05E000 {
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "hailo,hailo15-gp_vdma";
        reg = <0 0x7c05E000 0 0x200>;
        ranges;
        interrupts = <GIC_SPI HW_INTERRUPTS__SW_DMA_AP_INT_0_IRQ
IRQ_TYPE_LEVEL_HIGH>;
        dma-channel@0 {
                reg = <0x7c05C000 0x20>;
        };
        dma-channel@1 {
                reg = <0x7c05C020 0x20>;
        };
        dma-channel@2 {
                reg = <0x7c05C040 0x20>;
        };
        dma-channel@3 {
                reg = <0x7c05C060 0x20>;
        };
};
```

To disable the GP DMA, add to the device tree:

status = "disabled";

2.1.7.3. GP DMA Integration

The Hailo GP DMA driver Can be tested with the standard Linux DMA test module *'dmatest'*. The test is a kernel module that interacts through device files.

Note that loading kernel modules requires permission.

Table 6. Linux 'dmatest' usage

Function	Device File
List available channels	ls -1 /sys/class/dma/
Init 'dmatest' kernel module	modprobe dmatest
Run for 1 time	echo 1 > /sys/module/dmatest/parameters/iterations
Use channel 0	echo dma0chan0 > /sys/module/dmatest/parameters/channel
Run test	echo 1 > /sys/module/dmatest/parameters/run

Reference:

https://www.kernel.org/doc/html/v5.15/driver-api/dmaengine/dmatest.html

2.1.8. Pinmux

2.1.8.1. Overview

The pinmux is implemented through the standard Linux pinctrl subsystem. It manages and maps pins, pin groups and functions. When a driver requests a specific function (e.g. access to I2C2, the pinctrl subsystem checks whether it's possible to assign it and configures the pinmux accordingly.

The pins are configured from the device tree.

2.1.8.2. Device Tree Example

A list of supported functions and pin groups and their associated pins for the Hailo-15H are described in Section 9.

Below is a device tree example configuring usage of I2C_2.

```
&pinctrl {
    pinctrl_i2c2: i2c2 {
        function = "i2c2";
        groups = "i2c2_0_grp";
    };
};

&i2c_2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_i2c2>;
};
```

2.1.9. MIPI DSI (Display Serial Interface)

Hailo-15 supports DSI (Display Serial Interface), a high-speed interface standard defined by the MIPI® (Mobile Industry Processor Interface) Alliance, primarily used for connecting display panels.

The driver is integrated into the standard Linux display subsystem and rendering is achieved through the standard DRM subsystem.

The MIPI DSI interface supports RGB888 color format only.

2.1.9.1. Device Tree Example

To use MIPI DSI, it is necessary to enable several components in the board device tree, in the following example we use the Raspberry Pi screen, 7-inch touch screen-panel with 800x480 resolution:

```
&i2c_1 {
      lcd@45 {
            status = "okay";
            compatible = "raspberrypi,7inch-touchscreen-panel";
            reg = <0x45>;
            port {
               panel_in: endpoint {
                  remote-endpoint = <&dsi0_out>;
               };
            };
         };
};
&dpi0 {
   status = " okay";
};
```

```
&dsi0 {
   status = " okay";
   ports {
      port@0 {
         reg = <0>;
         dsi0_out: endpoint {
            remote-endpoint = <&panel_in>;
         };
      };
   };
};

&dphy0 {
   status = " okay";
};

&vision_subsys {
   status = " okay";
};
```

2.1.9.2. MIPI DSI Test Pattern

To display a test pattern on the screen, add to the device tree:

```
&dsi0 {
      tvg-mode-enable;
};
```

2.1.9.3. DRM Tool

To obtain and view the status of the display capabilities, run the terminal command:

```
modetest -M hailo-drm
```

2.1.9.4. MIPI DSI Using GStreamer

The element “kmssink” element is used to render video frames to the DSI screen.

GStreamer pipeline example to display a test video:

```
gst-launch-1.0 videotestsrc ! video/x-raw,width=800,height=480,format=BGR,framerate=60/1 ! kmssink driver-name="hailo-drm" can-scale=false force-modesetting=true
```

GStreamer pipeline example to display a camera sensor video feed:

```
gst-launch-1.0 v4l2src device=/dev/video0 ! queue ! videoconvert ! video/x-raw,width=800,height=480,format=BGR,framerate=30/1 ! queue ! kmssink driver-name="hailo-drm" can-scale=false force-modesetting=true
```

2.2. Hailo-15 Libraries

Hailo’s Linux distribution includes pre-integrated libraries that support Hailo-15 capabilities and should be used in Hailo examples.

Table 7. Hailo-15 Linux libraries

Library Name	Description	Usage
HailoRT	AI inferencing runtime	Required for neural network management and inference
Hailo Media Library	Media control and handling	Controls video capturing, video encoding and image correction/enhancement features
Hailo Imaging Software suite	Image signal processing and camera sensor support	

DSP Library	Digital Signal Processing	
GStreamer	Multimedia framework	Required for Media Library

2.2.1. Digital Signal Processor

2.2.1.1. Overview

The DSP operates as a co-processor to the AP and is controlled by it. Hailo provides the libraries for control from the AP under Linux for user applications. It also provides pre-compiled firmware focused on image processing.

2.2.1.2. DSP System Integration

Hailo provides a library that manages DSP task execution and queueing from Linux.

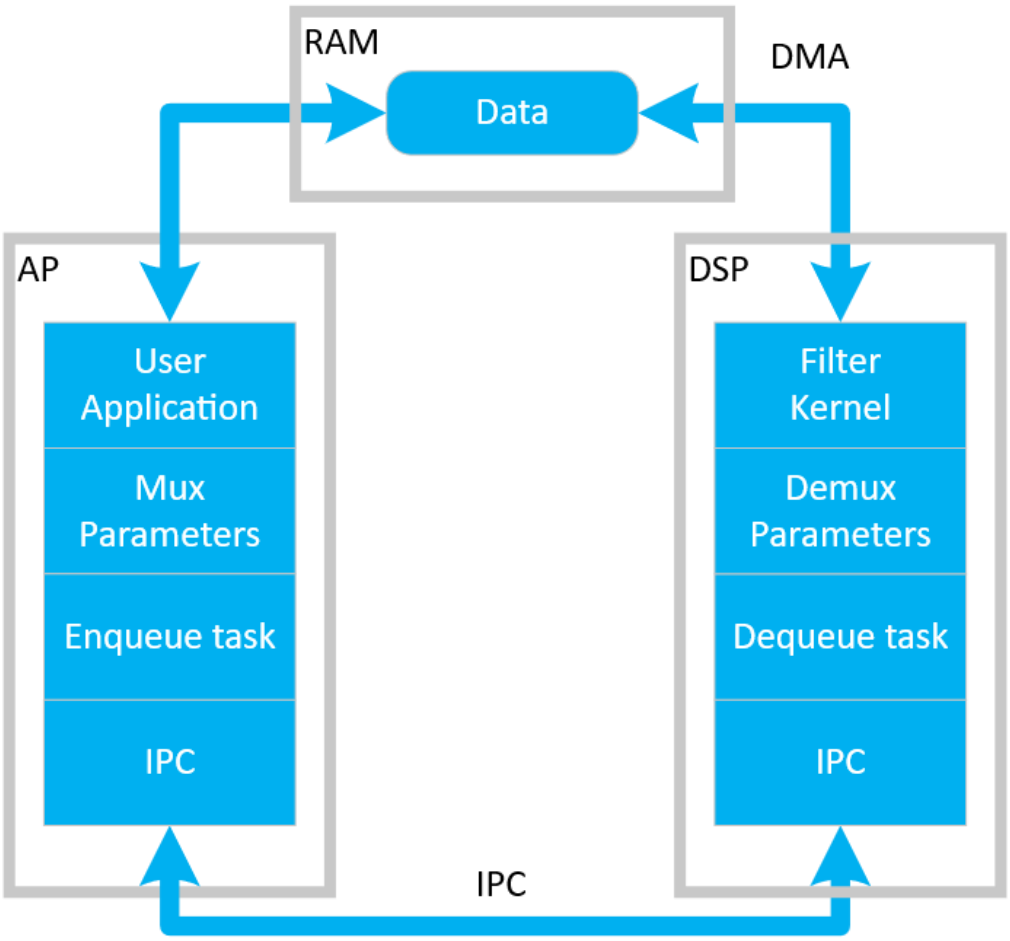


Figure 4. DSP system integration architecture

The DSP has its own dedicated DMA which allows to operate on data in-place or write to memory results in parallel and independent of the AP.

2.2.1.3. Hailo DSP Firmware

The Hailo provided DSP firmware is focused on image processing.

Refer to the Hailo DSP user guide for more information.

2.3. U-Boot Bootloader

2.3.1. Introduction

Hailo provides the industry standard U-Boot bootloader for Hailo-15's Application Processor (AP). It is distributed as open source with the Yocto Project build system which maximizes flexibility of adoption for customer boards and development processes.

U-Boot supports multiple boot sources, on or offboard, that can be switched and chosen during runtime. Since it's open source, additional boot sources, user indications and peripheral support can be added and adopted by the user. It also supports scripting for complex boot sequences and specialized behavior. For the most part, U-Boot is used to load and run the Linux kernel.

2.3.2. U-Boot Boot Flow

The system boot process is divided into three main stages: **U-Boot SPL**, **TF-A (Trusted Firmware-A)**, and **U-Boot**. Each stage is responsible for progressively initializing the system and handing off execution to the next stage.
The system boot process begins with the on-chip ROM code, which loads the SCU firmware (SCU-FW). After SCU-FW initialization, execution continues with **U-Boot SPL**, followed by **TF-A (BL31)** and finally **U-Boot**, which boots the Linux kernel.

It can be summarized in a simple flow:

ROM → SCU-FW → U-Boot SPL → TF-A (BL31) → U-Boot → Linux

1. **U-Boot SPL (Secondary Program Loader)**

   U-Boot SPL is the first non-secure software stage executed after SCU firmware, which itself is loaded by the ROM code. Its responsibility is to perform minimal hardware initialization and load the next stage.

   Responsibilities:

   1. Initialize basic hardware components:
      - MMU
      - CPU caches
      - Serial console (for early debug)
   2. Perform SCMI handshake with SCU firmware

3. Determine boot source from environment (eMMC / SD / NOR)
4. Load the firmware image (`u-boot-tfa.itb`) from the selected boot device
5. Transfer execution to TF-A (BL31)

2. TF-A (Trusted Firmware-A, BL31)

TF-A runs in the secure world and prepares the system for non-secure execution.

Responsibilities:

6. Initialize secure world components
7. Perform low-level platform setup
8. Hand off execution to U-Boot (non-secure world)

2. U-Boot

U-Boot is responsible for full system initialization and loading the operating system.

Responsibilities:

9. Initialize system components:
   - SCMI interface
   - DDR memory
   - Peripherals
10. Load U-Boot environment variables
11. Execute autoboot sequence:

Optional boot menu interaction

12. Load Linux image (`fitImage` containing kernel + DTB) from:
    - eMMC
    - SD card
    - NFS
13. Boot the Linux kernel

2.3.3. Boot Sources

U-Boot can load the Linux kernel image from several boot sources.

On-board sources are typically used for application development, while off-board sources are more convenient for kernel customization and development.

Table 8. U-Boot sources

Interface	Source
Ethernet	Network device
SDIO	SD card or eMMC
SPI	SPI NOR flash

2.3.4. U-Boot SPL

U-Boot SPL (Secondary Program Loader) is supported and provided by Hailo. It resides in SPI flash, is loaded by the SCU, and runs on the Hailo-15 Application Processor. U-Boot SPL enables loading of the main U-Boot image from SDIO storage.

2.3.4.1. U-Boot SPL Boot Sources

The U-Boot SPL boot source is determined by the "*spl_boot_source*" environment variable in the U-Boot environment file.

Supported values for the next image location are:

'mmc1' – SDIO0

'mmc2' –SDIO1

'mmc12' –SDIO0 and if it fails try SDIO1

'mmc21' –SDIO1 and if it fails try SDIO0

'uart' – loaded via UART

'ram' – placed in RAM

'nor' - SPI flash

For the '*nor*' option, the "*u-boot-tfa.bin*" file should be programmed into SPI flash using the SPI flash programming tool by using the '*--uboot-tfa*' flag.

2.3.5. U-Boot Drivers

Hailo provides integrated U-Boot drivers and platform integration for Hailo-15. These drivers are open source and can be modified by the user as needed.

Table 9. Summary of U-Boot integrated drivers

Hardware	Remarks
Connectivity	
UART	Provides standard serial port interface to hailo-15 UART. Usually used for logs capture, Linux console, and simple control/debug communication with external hosts or peripherals

Ethernet	Configures and drives the on-chip MAC and its PHY to provide standard network connectivity (IPv4/IPv6, TCP/UDP, etc.) RGMII/RMII links. It hooks into the Linux net stack (net_device), uses DT for clocks, resets and pinmux, streaming video, or data offload between the SoC and the outside world.
I2C	Manages the I2C masters, providing standard Linux /dev/i2c-* interfaces and i2c_adapters used by client drivers.
QSPI (Hailo-15H)	Usually used for communication with low-speed peripherals (e.g. camera sensors, PMICs, audio codecs, EEPROMs) defined in the device tree under each I2C bus.
XSPI (Hailo-15L)	The SPI-NOR controller used to access the external quad-SPI flash devices. It provides the Hailo-15 boot and persistent storage interface, exposing the flash as an MTD device.
Storage	
SDIO	Configures the Hailo-15 SD/SDIO/eMMC host controllers and connects them to the Linux MMC core. Usually used to access removable SD cards and on-board eMMC as block devices (e.g. root filesystem, data storage), handling clocking, resets and card-detect as described by the sdio* nodes in the device tree.
QSPI (Hailo-15H)	The SPI-NOR controller used to access the external quad-SPI flash devices. It provides the Hailo-15 boot and persistent storage interface, exposing the flash as an MTD device. Supports only: Master mode, Mode 0 Half duplex, 8-bit transfers, 6.25 MHz speed, 0 and 1 CS
XSPI (Hailo-15L)	The SPI-NOR controller used to access the external eXtended-SPI flash devices with additional IO/SDMA/aux/wrapper regions and its own IRQ. It provides the Hailo-15 boot and persistent storage interface, exposing the flash as an MTD device.

	In the future, this driver will provide an interface to SPI-NAND storage.
Platform	
ARM SCMI	Implements the System Control and Management Interface transport between Linux on the AP and the firmware/SCU. Used for Hailo-15 clocks, resets, power domains, DVFS, and specific services (boot info, fuses, throttling, identification, SKU, etc.), providing standard kernel APIs (clk, reset, etc.).
pl320_mbox	Controls the ARM PL320 hardware mailbox block used for inter-processor communication. On Hailo-15 it provides mailbox channels (via the generic mailbox framework) between the AP and other firmware/cores (e.g. NNC/DSP), enabling lightweight message/interrupt-based coordination.
Watchdog	Programs a hardware timer that reboots the system if it is not periodically “kicked” by software. Provides an interface that system daemons use to ensure recovery from hangs or software deadlocks.
Pinctrl	Configure the Hailo-15 pin muxing and pad control blocks. It lets other drivers select pin functions (GPIO vs peripheral), drive strength, pulls, and other pad settings through the kernel pinctrl and pinconf frameworks.
GPIO	Exposes the Hailo-15 GPIO banks as standard Linux GPIO controllers. It provides gpiochips used by other drivers and by userspace (via gpiolib/libgpiod) for simple digital I/O, interrupts, and board-level control signals.

2.3.5.1. Ethernet MAC Address

The Ethernet MAC Address can be assigned during boot by using the standard U-Boot environmental variable.

By default, U-Boot assigns a randomized MAC address during boot , which is

used until Linux takes over. If the 'ethaddr' variable is set to a valid MAC address in the format xx:xx:xx:xx:xx:xx where ('x' indicate a hexa-decimal digit), the MAC address will be used by both U-Boot and Linux.

For example, to set mac address 'e2-44-00-a3-52-c2' in the U-Boot terminal or as part for a script:

```
setenv ethaddr e2:44:00:a3:52:c2
```

Hailo also supports loading the MAC address from a reserved location in SPI flash. The behavior is controlled by the CONFIG_MAC_ADDR_IN_SPIFLASH option and the MAC_ADDR_OFFSET setting (default: 0x7000) in the U-boot defconfig.

By default, loading the MAC address from SPI flash is disabled.

For convenience, the Hailo-15 SPI flash programming tool supports the following flag to program a MAC address into SPI flash: '--flash-mac-addr <xx:xx:xx:xx:xx:xx>'

3. System Control Unit

3.1. Overview

The System Control Unit (SCU) is a dedicated microcontroller responsible for managing critical Hailo-15 control functions to ensure system stability and reliability.

The SCU is responsible for the following functions:

- Temperature monitoring
- Clock management
- Reset management
- Power management
- Boot management
- Recovery mode handling

Hailo provides dedicated SCU firmware that implements these functions. The firmware is signed by Hailo and is verified and authenticated during the boot process.

3.2. SCU System Integration

The SCU firmware implements the ARM® System Control and Management Interface (SCMI) protocol and communicates with the Hailo-15 system through a dedicated mailbox mechanism. It is integrated, via an SCMI agent, into Linux and U-boot, enabling the Hailo-15 Application Processor to issue commands, query status and alerts. This message-passing mechanism isolates critical control functionality from the AP software stack and ensures reliable execution of SCU tasks.

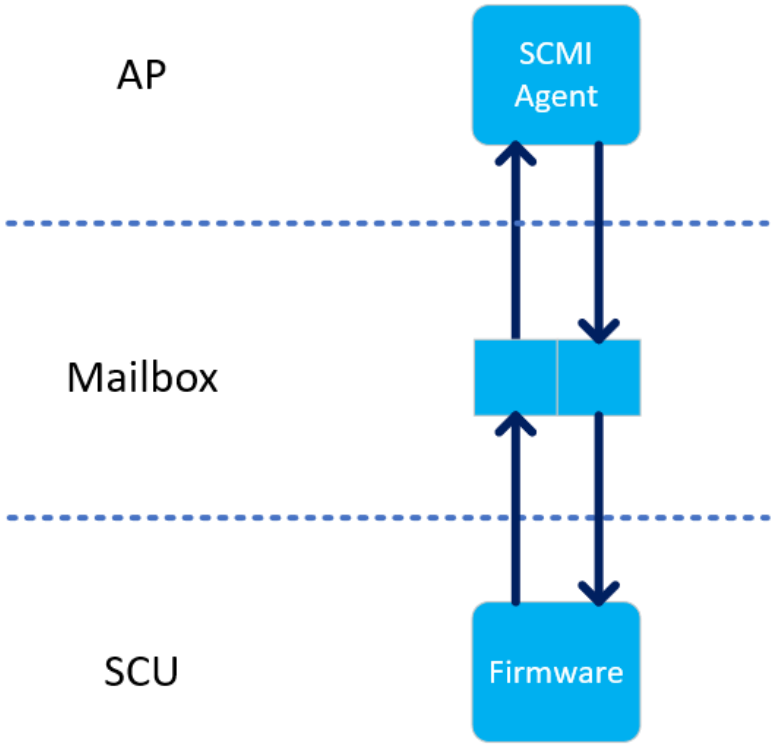


Figure 5. SCU integration

3.3. Temperature Monitoring

The SCU continuously monitors on-chip temperature sensors. The AP can query sensor readings via Linux device files as shown in Table 10.

Table 10. Sensor device files

Sensor	Device file
Temperature sensor 0	/sys/devices/virtual/thermal/thermal_zone1/temp
Temperature sensor 1	/sys/devices/virtual/thermal/thermal_zone2/temp

If excessive temperature is detected, the SCU initiates clock throttling (Thermal throttling). Upon reaching 120°C, the SCU performs an immediate shutdown without AP intervention.

3.3.1. Thermal Throttling Principles

Thermal Throttling automatically adjusts Hailo-15 device load based on current temperature conditions to maintain safe operation:

- As temperature rises, device load is reduced to generate a cooling effect.
- As temperature decreases, device load may be increased to improve performance or re-enable applications' features.

Thermal control is achieved through:

- Increasing or decreasing the Hailo-15 operating frequency.
- Controlling application pipeline features, such as:
    - Enabling or disabling low-light mode.
    - Enabling or disabling camera streams.

3.3.2. Thermal Throttling Linux Integration

3.3.2.1. Hailo-15 Thermal Management System

Thermal throttling is implemented using the Hailo thermal management system, which includes the following components:

1. **hailo-thermal-engine**: Monitors thermal trip events and logs heating and cooling transitions to the system log.
2. **hailo-thermal-service.sh**: Ensures that hailo-thermal-engine is started during Linux boot.
3. **libhailo-throttling.so**: Manages system throttling states based on thermal trip events.

User applications can link the 'libthrottling.so' library to register callbacks for throttling state transitions.

3.3.2.2. Hailo Thermal Engine

The 'hailo-thermal-engine' executable monitors thermal trip events via Linux Netlink interface and reacts accordingly. The following events are detected:

- TRIP_x_HEATING (where x = 0,1,2,3,4): Indicates rising temperature levels.
- TRIP_x_COOLING (where x = 0,1,2,3,4): Indicates falling temperature levels.

3.3.2.3. Hailo Thermal Service

The 'hailo-thermal-service.sh' shell script is part of Linux system init scrips. It starts 'hailo-thermal-engine' upon Linux boot and stops 'hailo-thermal-engine' upon Linux Halt/Shutdown/reboot.

Please note that the provided `/etc/init.d/rcS` (boot-time system initialization script) must remain unchanged, as it invokes `S04udev -> ../init.d/udev`, which creates and mounts `/dev/shm` as `tmpfs`. This step is required for the service to function correctly.

3.3.2.4. Hailo Throttling Library

The 'libhailo_throttling.so' library is provided as part of the libhailo-throttling Yocto Project package. It manages throttling states derived from thermal trip events, and allows applications to register enter and exit callback function per each state.

Throttling States IDs:

- ThrottlingStateId::UNINIT – throttling state not initialized yet
- ThrottlingStateId::FULL_PERFORMANCE – no throttling actions
- ThrottlingStateId::S0 – Lowest throttling level
- ThrottlingStateId::S1
- ThrottlingStateId::S2
- ThrottlingStateId::S3
- ThrottlingStateId::S4 – Highest throttling level

Throttling State Transitions:

- On TRIP_x_HEATING, the system moves to STATE_x.
- On TRIP_x_COOLING, the system moves to STATE_x-1.
- On TRIP_0_COOLING, the system returns to STATE_FULL_PERFORMANCE.

Simple Throttling Management Application Example

The Yocto Project recipe 'hailo-simple-throttling-app.bb' provides a simple application example:

- AI denoise is enabled after 5 seconds upon entering FULL_PERFORMANCE state from S0.
- do_enterCb_fullPerf(..) estimates remaining cooldown duration to enable AI denoise. This remaining cooldown calculation is needed if the application has been restarted.

3.4. SCU Boot Configuration

SCU boot-time controlled features are configured using a dedicated device tree node within the U-Boot device tree. Refer to Table 11.

Table 11. SCU boot configuration device tree nodes

Property name	Parameter	Description
shutdown-gpio-pin	GPIO pin number (0-31)	GPIO number assigned to indicate Hailo-15 shutdown
reset-gpio-pin	GPIO pin number (0-31)	GPIO number assigned to indicate Hailo reset
watchdog-timeout	Watchdog timeout value in seconds	1-20 –SCU watchdog in seconds. 0 – indicate SCU watchdog is disabled
Connectivity node		sub node for hailo_boot_info
uart-baudrate	UART baud rate	Part of Connectivity node, set the UART baud rate. Default is 115200. Valid values are: 4800, 9600, 19200, 38400, 57600, 230400, 460800, 921600
uart-#-reverse-rts	Is reverse polarity needed for RTS	Reversed polarity needed for the UART # for RTS in UART flow control. UART number can be 0-3.
uart-#-reverse-cts	Is reverse polarity needed for CTS	Reversed polarity is needed for the UART # for CTS in UART flow control. UART number can be 0-3

See below an example for SCU boot configuration device tree:

```
hailo_boot_info: hailo_boot_info@0 {
    compatible = "hailo,boot-info";
    dtb-ddr-load-address = <0x0 CONFIG_HAILO15_DTB_ADDRESS 0x0 CONFIG_HAILO15_DTB_MAX_SIZE>;
    shutdown-gpio-pin = <20>;
    watchdog-timeout = <10>;
    connectivity {
        uart-baudrate = <115200>;
        uart-0-reverse-rts;
        uart-0-reverse-cts;
    };
};
```

Property name	Parameter	Description
ddr_config node		sub node for hailo_boot_info See hailo15_ddr_configuration.dtsi Constants located in: u-boot/arch/arm/dts/include/dt-bindings/soc/hailo15_ddr_config.h
zero_out_memory_enable	Initialize memory	Setting this property will indicate SCU firmware to initialize memory to '0' Default is False.
bist_enable	Run BIST	Run DDR startup BIST test. Default is don't perform
ecc_mode	ECC mode for the DDR controller	**Mode**: DDR_CTRL_ECC_MODE_DISABLED=0 **Description**: DDR ECC disabled. **This is the default**

	mode.
DDR_CTRL_ECC_MODE_ENABLED=1	DDR ECC calculation enabled. Error detection and correction are disabled.
DDR_CTRL_ECC_MODE_DETECTION=2	DDR ECC calculation and detection are enabled. Error correction is disabled.
DDR_CTRL_ECC_MODE_CORRECTION=3	DDR ECC calculation, detection and correction is enabled. This is the recommended mode if you want to use ECC.

DDR configuration as part of SCU boot configuration device tree example:

```
#include "dt-bindings/soc/hailo15_ddr_config.h"
#include "hailo15_ddr_base_regconfig.dtsi"
#if (HAILO_SOC_TYPE == HAILO_SOC_HAILO15)
   #include "hailo15_ddr_patch_include.dtsi"
#elif (HAILO_SOC_TYPE == HAILO_SOC_HAILO15L)
   #include "hailo15l_ddr_patch_include.dtsi"
#endif
&hailo_boot_info {
   ddr_config {
      working_mode = <DDR_WORKING_MODE_NORMAL>;
#ifdef CONFIG_HAILO15_DDR_ENABLE_ECC
      ecc_mode = <DDR_CTRL_ECC_MODE_CORRECTION>;
#else
      ecc_mode = <DDR_CTRL_ECC_MODE_DISABLED>;
#endif
      bist_enable = <1>;
      operational_freq = <2>;
      assembled_dram_pn = "";
      layout_swizzling_config = <0x8 0x06173452 0x8 0x21043576 0x8 0x52016437 0x8 0x56247310>;
      layout_byte_swap = <0x3 0x2 0x1 0x0>;
      stop_before_controller_start = <0>;
      f1_frequency = <1600000000>;
      f2_frequency = <2133000000>;
      periodic_io_calibration_disable = <0>;
      dram_auto_low_power_mode =
<DRAM_AUTO_LOW_POWER_MODE_AUTOMATIC_INTERFACE>;
      ddr_phy_low_power_mode = <DDR_PHY_LOW_POWER_MODE_LIGHT_SLEEP>;
   };
};
```

4. Watchdog Timers

4.1. Overview

The Hailo-15 contains several independent watchdog timers to mitigate software crashes and scenarios where the system is in an undefined state. It is up to the software to periodically re-arm the watchdog timer as an indication that the system is working properly.

There are independent watchdog timers for the AP and the SCU.

4.2. SCU Watchdog

The SCU has its own dedicated hardware watchdog timer. It is enabled during boot by the SCU firmware provided by Hailo.

An SCU watchdog timeout event will reset the Hailo-15.

The SCU watchdog timeout is 1 second.

4.3. AP Watchdog

The AP has its own dedicated hardware watchdog timer that covers the application domain. Hailo provides drivers for both U-Boot and the Linux kernel. A different timeout can be set for U-Boot and Linux and is configurable from the device tree. The AP watchdog timer can also be disabled in that manner.

By default, the AP watchdog is enabled by U-Boot with a timeout of 60 seconds. Once the Linux kernel is loaded the Linux driver takes over with a timeout of 10 seconds.

When the AP watchdog timer reaches a timeout event, it is caught by the SCU, which in turn will reset the Hailo-15.

4.3.1. Watchdog Timer Integration

The AP watchdog timer is exposed and can be managed under Linux using special device files.

Table 12. Watchdog timer Linux device files

Function	Device file
AP watchdog configured timeout (in seconds)	/sys/class/watchdog/watchdog0/timeout
Time left for AP watchdog event trigger (in seconds)	/sys/class/watchdog/watchdog0/timeleft
AP watchdog standard device file	/dev/watchdog

Device tree example:

```
watchdog@7c2c1000 {
        compatible = "arm,sp805", "arm,primecell";
        reg = <0 0x7c2c1000 0 0x1000>;
        clocks = <&scmi_clk HAILO15_SCMI_CLOCK_IDX_HCLK>,
                 <&scmi_clk HAILO15_SCMI_CLOCK_IDX_HCLK>;
        clock-names = "wdog_clk", "apb_pclk";
        timeout-sec = <10>; // user defined
};
```

To disable the AP watchdog, add to the device tree:

status = "disabled";

5. Customization Guidelines

5.1. Kernel Customization

Kernel customization enables users to adapt, change, port and optimize the Linux kernel according to their requirements and system constraints.

Hailo provides a default configuration for its Hailo-15 based boards, which can serve as a starting point for evaluation, prototyping and product development.

5.1.1. Prerequisites

To install the required tools, perform the following steps on a Linux machine (Ubuntu 20.04 LTS is required). Administrative privileges may be required.

1. Verify the operating system version:

   ubuntu20 version

2. Update the package repositories

```
sudo apt update
```

3. Install the required packages

```
sudo apt install python3-pip chrpath diffstat gawk zstd lz4 kas
```

4. Install `python3-newt` package

```
sudo apt install python3-newt
```

5.1.2. Kernel Menuconfig

To locally edit the Linux kernel menuconfig , run:

```
bitbake linux-yocto-hailo -c menuconfig
```

The Linux Kernel menuconfig screen will be displayed as shown in Figure 6

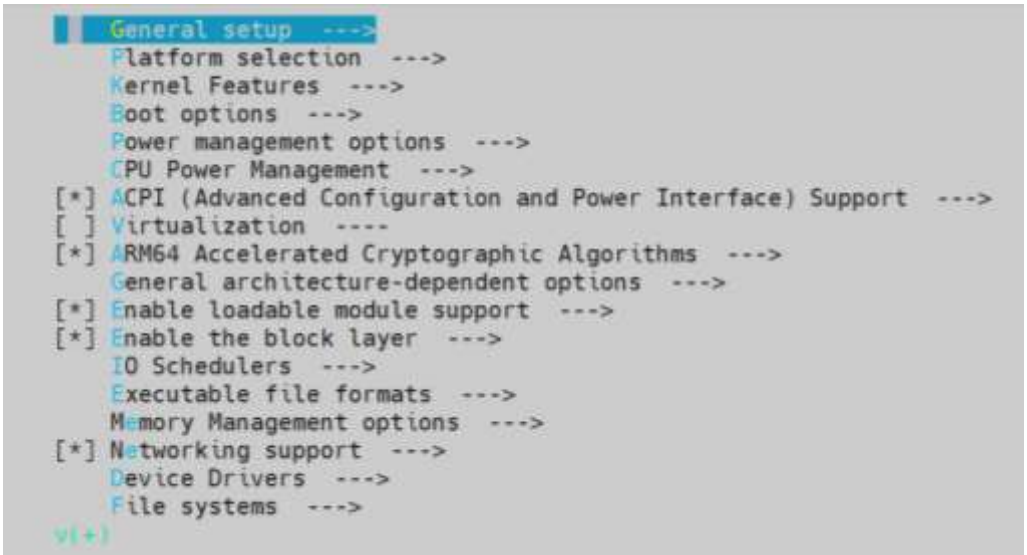


Figure 6. Linux kernel Yocto menuconfig

After saving and exiting the menuconfig, Yocto Project will use the newly saved ".config" file when compiling the kernel in the current local Yocto Project environment until local manual changes are cleaned.

To clean the manual local changes in menuconfig, run:

```
bitbake linux-yocto-hailo -c cleansstate
```

Yocto Project will indicate it is using an environment with local changes with the following warning:

"WARNING: …:do_compile is tainted from a forced run".

To compile only the linux kernel, run:

```
bitbake linux-yocto-hailo
```

To permanently save the menuconfig changes:

First generate a fragment file containing the changed configuration:

```
bitbake linux-yocto-hailo -c diffconfig
```

Yocto Project will generate a 'fragment.cfg' file and print its path.

To permanently apply the configuration from the fragment:

Add the fragment file in a bbappend to the Linux kernel recipe in your custom Yocto layer (meta-<vendor>):

1. Create a directory 'meta-<vendor>/recipes-kernel/linux-yocto-hailo'
2. Place the fragment.cfg file in that directory
3. Create a file "linux-yocto-hailo.bbappend" in the directory, with the following content:


```
FILESEXTRAPATHS:prepend := "${THISDIR}:"

SRC_URI:append = " file://fragment.cfg"
```

For more information on creating a custom Yocto layer, see Section 5.2.

5.2. Yocto Build System

The Yocto Project build system allows the user to quickly and easily customize and build the configuration and compile an image for the Application Processor.

The build system is most conveniently run on a development host for cross compilation. The artifacts produced can then be loaded and run on Hailo-15.

To compile an image using the Yocto Project build system:

1. If not already done, install required packages

   ```
   sudo apt install python3-pip chrpath diffstat gawk zstd lz4 kas
   ```

2. Clone the git repository and navigate to the release folder:

   ```
   git clone https://github.com/hailo-ai/meta-hailo-soc.git
   cd meta-hailo-soc
   ```

3. Initiate first time build to create and configure the default environment (this needs to be ran only once):

   ```
   kas build kas/hailo15-evb.yml
   ```

   Where the .yml file represents the target board.

4. Source the build environment:

   ```
   source poky/oe-init-build-env
   ```

5. To compile the full development image, run bitbake using the command:

   ```
   bitbake core-image-hailo-dev
   ```

   or, to compile a smaller image, run bitbake using the command:

   ```
   bitbake core-image-hailo
   ```

   or, to compile the kernel only, run bitbake using the command:

   ```
   bitbake linux-yocto-hailo
   ```

6. Optionally, for kernel customization see Section 5.1 and for U-Boot customization refer to Section 5.3.

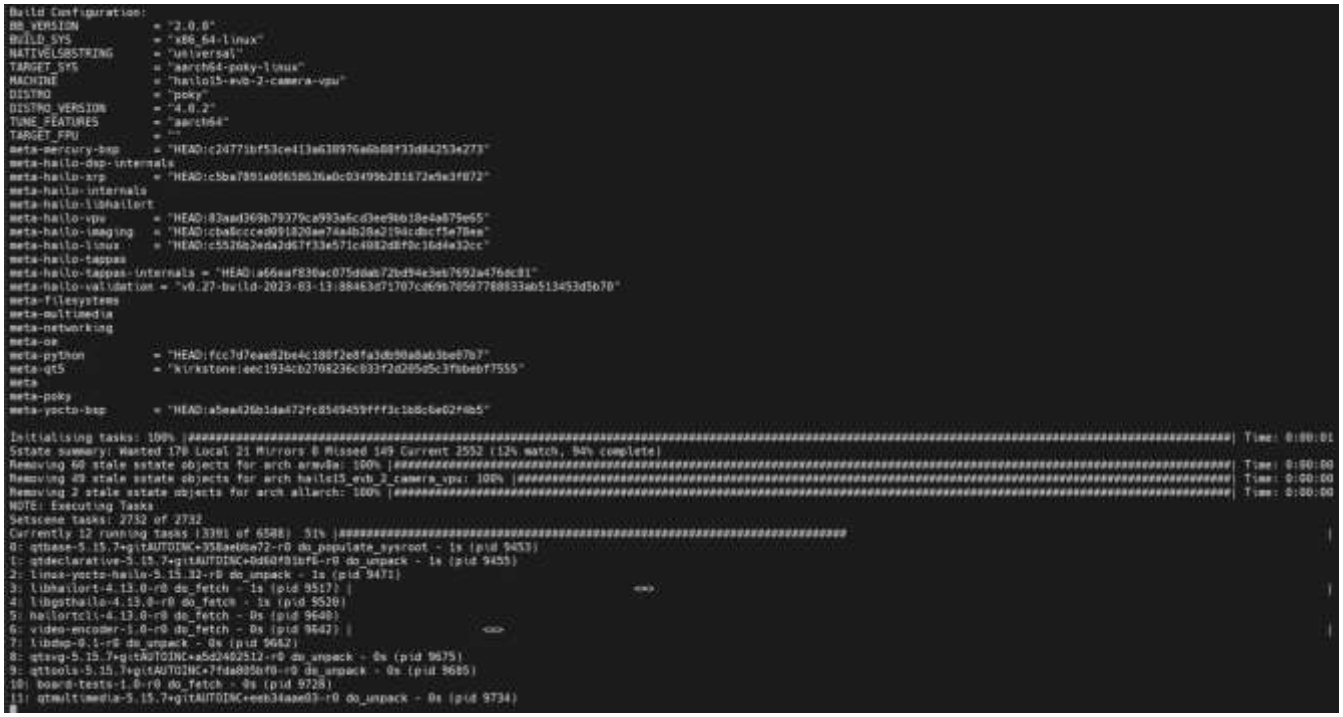


Figure 7. Yocto project build process running

Once the compilation is completed, binaries alongside a unified Flattened Image Tree (FIT) file will be created. The FIT image will contain all the required binaries for convenience (kernel, ramdisk, device tree…) to enable Linux to run.

5.2.1. Yocto SDK

The Yocto Project SDK allows for cross-compilation and development of applications on a development host to be run on a Hailo-15 target.

Hailo provides toolchain and sysroot for its pre-built image.

To generate the SDK, on the development host navigate to the EA release folder and run the commands:

```
source poky/oe-init-build-env
```

```
bitbake core-image-hailo-dev -c populate_sdk
```

The SDK will be generated in the directory:

Build/tmp/deploy/sdk

5.2.2. Development with Local Repositories

Development with a local repository (linux kernel, u-boot, etc…) is a common

task when developing using the Yocto Project build system.

Yocto Project includes a *devtool* command that allows overriding the source of a repository for a recipe, to allow development using a local repository clone, instead of a remote Git repository.

For a full usage guide to the *devtool* command, refer to the Yocto Project reference article:

https://docs.yoctoproject.org/4.0.11/ref-manual/devtool-reference.html

First enter the Yocto Project environment with the command:

```
source poky/oe-init-build-env
```

To use an existing local repository, run:

```
devtool modify -n <RECIPE> <LOCAL_REPO_PATH>
```

To tell *devtool* to clone the repo into the directory and use it, run:

```
devtool modify <RECIPE> <LOCAL_REPO_PATH>
```

where `<RECIPE>` can be linux-yocto-hailo, u-boot, etc...

For example:

```
devtool modify -n linux-yocto-hailo <LOCAL_REPO_PATH>
```

To check which recipes are using local repos, run:

```
devtool status
```

Now one can build with bitbake normally, however the source of `<RECIPE>` would be the local cloned repository that was already made using *devtool*.

To build only the newly created recipe:

```
bitbake <RECIPE>
```

Or, to build the entire image:

```
bitbake core-image-hailo-dev
```

To instruct Yocto Project to return using the remote repository, run:

```
devtool reset <RECIPE>
```

5.3. BSP Customization

This section describes how to customize the software package for working with custom boards.

It is assumed that a git fork of the Hailo Linux kernel port and U-Boot repositories will be used.

This section describes the customizations steps for U-Boot, Linux kernel and Yocto Project build system to support the custom board.

Please pay attention to the considerations listed below when following these instructions:

- This section is intended to be followed in order.
- The terms <vendor> and <board-name> will be used and should be replaced by their respective user selected names.
- The Yocto Project environment is initialized by using the command:

```
source poky/oe-init-build-env
```

5.3.1. Yocto BSP Customization

5.3.1.1. Creating a Vendor Yocto Meta Layer

This section will describe how to create a meta-<vendor> Yocto Project layer that will describe board-specific configurations.

The meta-<vendor> Yocto Project layer contains the following content:

- Linux kernel recipe 'bbappend' – for using a custom Linux git repository and for changing the kernel configuration.
- U-Boot recipe 'bbappend' – for using a custom U-Boot git repository.
- Board-specific OS configuration and user applications.

Any other board-or product-specific Yocto Project recipes.

If there is already a customer Yocto Project layer, then this part can be skipped. In the rest of this guide, the term meta-<vendor> will be used as the name of the vendor Yocto Project layer.

1. Copy an existing layer or create a layer using the bit-bake-layers command:

```
mkdir -p <your directory>
```

```
bitbake-layers create-layer --priority 7 <your directory>/meta-<vendor>
```

2. Set the priority, it must be greater than 6. The priority of meta-hailo-bsp (as can be seen in by using the command:

```
 bitbake-layers show-layers)
```

It is highly recommended to use source control on the created meta layer.

Bit-bake will automatically create an example recipe in the "recipes-example" directory inside meta-<vendor> which can be removed.

Reference:
https://docs.yoctoproject.org/4.0.9/dev-manual/common-tasks.html#creating-a-general-layer-using-the-bitbake-layers-script

5.3.1.2. Adding the Vendor Yocto Layer

Add the layer by using the commands:

```
cd build
bitbake-layers add-layer <your directory>/meta-<vendor>
```

and verify it has been added:

```
bitbake-layers show-layers
```

Reference:

https://docs.yoctoproject.org/4.0.9/dev-manual/common-tasks.html#adding-a-layer-using-the-bitbake-layers-script

5.3.1.3. Creating a New Machine

The MACHINE Yocto Project variable controls various board-specific configurations in the Yocto Project build system and is being used by the recipes.

The name of the MACHINE should be the name of the board.

1. Create machine directory 'meta-<vendor>/conf/machine' if there isn't already one.
2. Create a '<board-name>.conf' file in the 'meta-<vendor>/conf/machine' directory with the following content:

```
require conf/machine/hailo15-base.inc
UBOOT_MACHINE = "<board_name>_defconfig"
```

The UBOOT_MACHINE variable controls the 'defconfig' configuration file being used when compiling U-Boot. The 'defconfig' creation is described later in the "U-Boot BSP Customization" section.

3. Change the MACHINE variable, replace the MACHINE line in the 'build/conf/local.conf' file:

From: MACHINE ??= "hailo15-evb-security-camera"

To: MACHINE = "<board-name>"

4. To see that the change has actually been applied, perform a dry-run bitbake, and look for the MACHINE field in the “Build Configuration” section of the output. See Figure 8.

bitbake --dry-run core-image-hailo-dev

```
Build Configuration:
BB_VERSION           = "2.0.0"
BUILD_SYS            = "x86_64-linux"
NATIVELSBSTRING      = "universal"
TARGET_SYS           = "aarch64-poky-linux"
MACHINE              = "vendor-board"
DISTRO               = "poky"
DISTRO_VERSION       = "4.0.2"
TUNE_FEATURES        = "aarch64"
TARGET_FPU           = ""
meta
meta-poky
```

Figure 8. MACHINE variable dry dun result

Reference:

https://docs.yoctoproject.org/4.0.9/ref-manual/variables.html#term-MACHINE

5.3.2. Linux BSP Customization

5.3.2.1. Linux Repository

Adding a new board to the kernel code base requires changes in multiple files. It is advisable to fork the linux-yocto-hailo repository provided by Hailo and make the changes required for the custom board in the forked repo.

5.3.2.2. Create Devicetree

Create a devicetree file in the path:

‘arch/arm64/boot/dts/<vendor> /<board-name>.dts’

The file name should match the Yocto MACHINE variable.

The devicetree should start with the following lines:

```
/dts-v1/;

#include "../hailo/hailo15-base.dtsi"
```

Create ‘arch/arm64/boot/dts/<vendor>/Makefile’ with the following content:

```
dtb-$(CONFIG_ARCH_HAILO15) += <board-name>.dtb
```

Update ‘arch/arm64/boot/dts/Makefile’ and add your vendor directory:

```
subdir-y += <vendor>
```

5.3.2.3. Instruct Yocto to use the Custom Linux Repository

Create ‘meta-<vendor>/ recipes-kernel/linux-yocto-hailo/linux-yocto-hailo.bbappend’ with the following content:

```
LINUX_YOCTO_HAILO_URI = "YOUR_GIT_REPOSITORY"

LINUX_YOCTO_HAILO_BRANCH = "YOUR_BRANCH"

LINUX_YOCTO_HAILO_SRCREV = "YOUR_SRCREV"

LINUX_YOCTO_HAILO_BOARD_VENDOR = "<vendor>"
```

5.3.2.4. Test Compilation

5.3.2.4.1. U-Boot Test Compilation

At this point, Yocto Project should be able to build U-Boot using the new custom repository and the new machine.

Compile using the command:

```
bitbake u-boot
```

5.3.2.4.2. Linux Test Compilation

At this point, Yocto Project should be able to build linux-yocto-hailo using the custom repository and the new machine.

Compile by using the command:

```
bitbake linux-yocto-hailo
```

5.3.2.4.3. DDR Configuration Generation

The DDR configuration is stored as part of the U-Boot device tree under the section ‘*hailo_boot_info*’, with the property ‘*ddr_config*’.

The values are stored in .dtsi files that the board device tree includes.

Example from SBC device tree:

```
#include "hailo15_ddr_configuration.dtsi"

#include "hailo15_ddr_MT53E1G32D2FW-046_regconfig_ca_odtb_pu.dtsi"
```

```
#include "hailo15-base.dtsi"
#include "hailo15_ddr_configuration.dtsi"
//#include "hailo15_ddr_MT53E512M32D1-046_regconfig_2GB.dtsi"
//#include "hailo15_ddr_MT53E2G32D4DE-046_regconfig_8GB.dtsi"
#include "hailo15_ddr_MT53E1G32D2FW-046_regconfig_ca_odtb_pu_4GB.dtsi"
```

The user needs to build the U-Boot image to create a compiled and signed U-Boot device tree file (*u-boot.dtb.signed*).

Also, according to the relevant DDR size (2GB, 4GB, 8GB), the user should select the correct dtsi file to include.

5.3.2.4.4. Enabling ECC

To enable ECC, in the *build/conf/local*.conf add

```
MACHINE_FEATURES:append = " ddr_ecc_en"
```

5.3.3. U-Boot BSP Customization

5.3.3.1. U-Boot Repository

Adding a custom board to the U-Boot code base requires changes in multiple files. It is recommended to fork the U-Boot repository provided by Hailo and make the changes required for your board on said forked repository.

5.3.3.2. Defining Custom Board in U-Boot

U-Boot uses the following terminology:

Architecture – company or line of products. In our case: ARCH_HAILO

Machine – name of the specific VPU. In our case: MACH_HAILO15

Target – name of the board. We will use TARGET_<BOARD_NAME>.

5.3.3.2.1. Creating a Custom Board Directory

Create the board's directory at *'board/<vendor>/<board-name>/'*.

The board name must match the Yocto Project MACHINE name.

Inside the board's directory prepare the following 2 files:

1) Kconfig:

```
if TARGET_<BOARD_NAME>

config SYS_BOARD
        default "<board-name>"

config SYS_VENDOR
        default "<vendor>"

config SYS_CONFIG_NAME
        default "<board-name>"

endif
```

Where:

SYS_BOARD should match the board's directory name

SYS_VENDOR should match the vendor's directory name

SYS_CONFIG_NAME should match the board name

2) Makefile, which should contain the following:

```
obj-y := ../../hailo/common/hailo15_board.o
obj-y += ../../hailo/common/hailo15_mem_map.o
obj-$(CONFIG_SPL_BUILD) += ../../hailo/common/hailo15_spl.o
```

5.3.3.2.2. Create Config Header File

Create the file '*include/configs/<board-name>*.h' (filename should be the same as SYS_CONFIG_NAME in the board's Kconfig) with the following content:

```
/* SPDX-License-Identifier: GPL-2.0+ */

#ifndef __<BOARD_NAME>_H
#define __<BOARD_NAME>_H

#include "hailo15_common.h"

#ifdef CONFIG_HAILO15_DDR_ENABLE_ECC

/* Bank 0 size using ECC */
#define PHYS_SDRAM_1_SIZE (0x70000000)
/* Bank 1 address/size using ECC */
#define PHYS_SDRAM_2 (0x100000000)
#define PHYS_SDRAM_2_SIZE (0x70000000)

#else

/* Bank 0 size not using ECC */
#define PHYS_SDRAM_1_SIZE (0x100000000)

#endif
#endif
```

PHYS_SDRAM_1_SIZE should reflect the size of the board's RAM memory. In this example 4GB is used in non-ECC layout, and 3.5GB is used in ECC layout.

5.3.3.2.3. Adding a Custom Board to the Hailo Architecture Kconfig

Edit 'arch/arm/mach-hailo/Kconfig'

1. Add config to the custom board's target near the existing TARGET_*configs:

```
config TARGET_<BOARD_NAME>
        bool "<board description>"
        select MACH_HAILO15
```

2. Add the following line sourcing the custom board's 'Kconfig' near the existing source "board/..." lines:

```
source "board/<vendor>/<board-name>/Kconfig"
```

5.3.3.2.4. Create the Custom Board's Devicetree

Create the custom board's devicetree in "*arch/arm/dts/<board-name>.dts*".

The first line in the devicetree should include the hailo15-base.dtsi file located in the same directory:

```
#include "hailo15-base.dtsi"
```

`The devicetree should include the DDR configuration dtsi file, refer to Section 5.3.2.4.3 for details.`

Look at "hailo15-base.dtsi" and use *status="okay"* on the devices used by the custom board that are disabled in the Hailo-15 base devicetree (for example, eth, sdio0, sdio1, etc..).

Add a make rule to compile your devicetree in "`arch/arm/dts/Makefile`":

```
dtb-$(CONFIG_TARGET_<BOARD_NAME>) += <board-name>.dtb
```

Should be added before the following line:

```
targets += $(dtb-y)
```

5.3.3.2.5. Create the Custom Board's defconfig

Create 'configs/<board_name>_defconfig' with the following content:

```
CONFIG_ARM=y
CONFIG_ARCH_HAILO=y
CONFIG_TARGET_<BOARD_NAME>=y
CONFIG_DEFAULT_DEVICE_TREE="<board-name>"
```

See Section 5.4.2 for details on adding configuration to this file after customizing u-boot using the menuconfig.

5.3.3.3. Instruct Yocto to Use the Custom U-Boot Repository

Create a directory 'meta-<vendor>/recipes-bsp/u-boot/' and there create a file "u-boot_%.bbappend" with the following content :

```
BRANCH = "BRANCH"
SRCREV = "SRCREV/COMMIT-HASH"
SRC_URI = "YOUR UBOOT GIT REPO"
```

5.3.3.4. Test Compilation

At this point, the Yocto Project should be able to build U-Boot using the new custom repository and the new machine.

Compile using the command:

```
bitbake u-boot
```

5.4. U-Boot Customization

5.4.1. Prerequisites

U-Boot requires the same prerequisites as the Linux kernel. If the installation steps have not yet been performed, please follow the steps described in Section 5.1

5.4.2. U-Boot Menuconfig

To locally edit U-Boot's menuconfig , run:

```
bitbake u-boot -c menuconfig
```

The U-Boot menuconfig screen will be displayed.

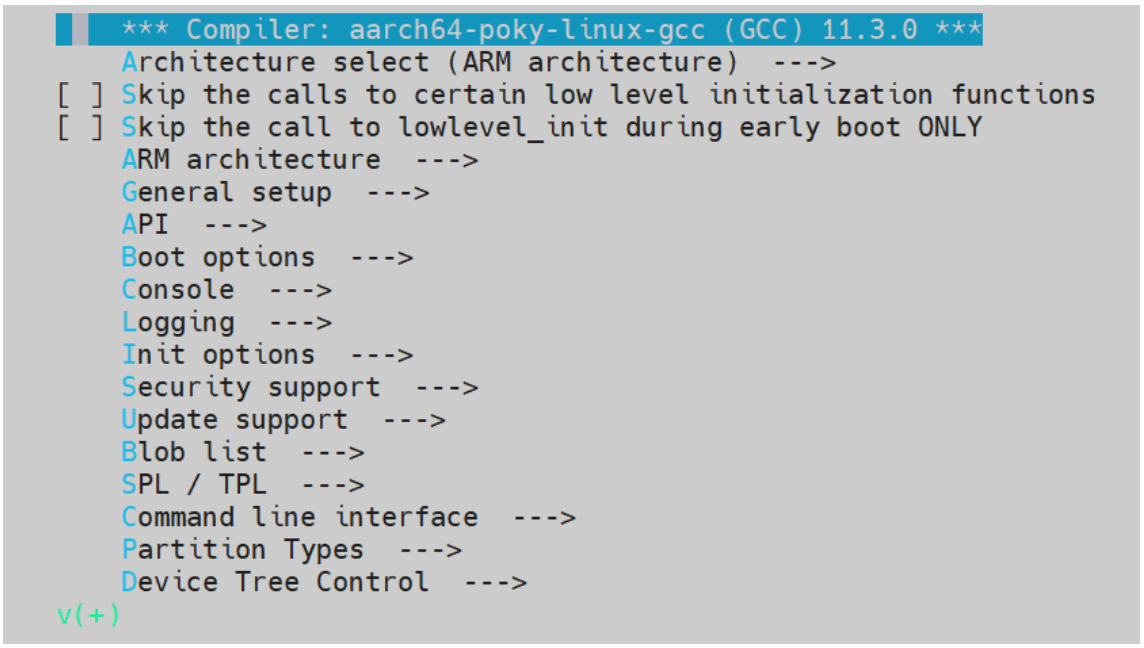


Figure 9. U-Boot Yocto project menuconfig

After saving and exiting the menuconfig, Yocto Project will use the newly saved ".config" file when compiling U-Boot in the current local Yocto environment until local manual changes are cleaned.

To clean the manual local changes in menuconfig, run:

```
bitbake u-boot -c cleansstate
```

To compile U-Boot only, run:

```
bitbake u-boot
```

Yocto Project will indicate it is using an environment with local changes with the following warning:

"WARNING: ...:do_compile is tainted from a forced run".

To compile U-Boot only, run:

```
bitbake u-boot
```

To permanently save the menuconfig changes:

A fragment file must first be generated containing the changed configuration:

```
bitbake u-boot -c diffconfig
```

Yocto Project will generate a ‘fragment.cfg’ file and print its path.

NOTE: Development host compiler information will be added to the beginning of this file. This information needs to be removed by manually deleting the first 2 lines.

Example fragment.cfg file:

```
# Compiler: gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0
CONFIG_GCC_VERSION=90400
CONFIG_CMD_MD5SUM=y
# CONFIG_MD5SUM_VERIFY is not set
CONFIG_CMD_MEMINFO=y
```

To permanently apply the configuration from the fragment,

either:

3. Add them to the board’s defconfig – (see Section 5.3.3.15.3.3.3 or
4. Add the fragment file in a bbappend to the u-boot recipe in your custom Yocto Project layer (meta-<vendor>):
    a. Create a directory ‘meta-<vendor>/recipes-bsp/u-boot/’
    b. Place the ‘fragment.cfg’ file in that directory
    c. Create a file “u-boot_%.bbappend“ in the directory, with the following content:

```
FILESEXTRAPATHS:prepend := "${THISDIR}:"

SRC_URI:append = " file://fragment.cfg"
```

For more information on creating a custom Yocto Project layer, see Section 5.3.1

6. Hailo Tools

6.1. Hailo-15 Profiler

6.1.1. Overview

The Hailo-15 Profiler can be used for system-level performance analysis on Hailo-15 based platforms.

It records and visualizes multiple data sources together on a unified timeline, including DDR bandwidth counter, application-level events from the Hailo Media Library, queue occupancy levels, and Linux scheduler traces. Visualization is achieved through a Linux utility for trace capturing and a web-based viewer for trace and analysis.

The Hailo-15 profiler is built on the Perfetto open-source tracing framework.

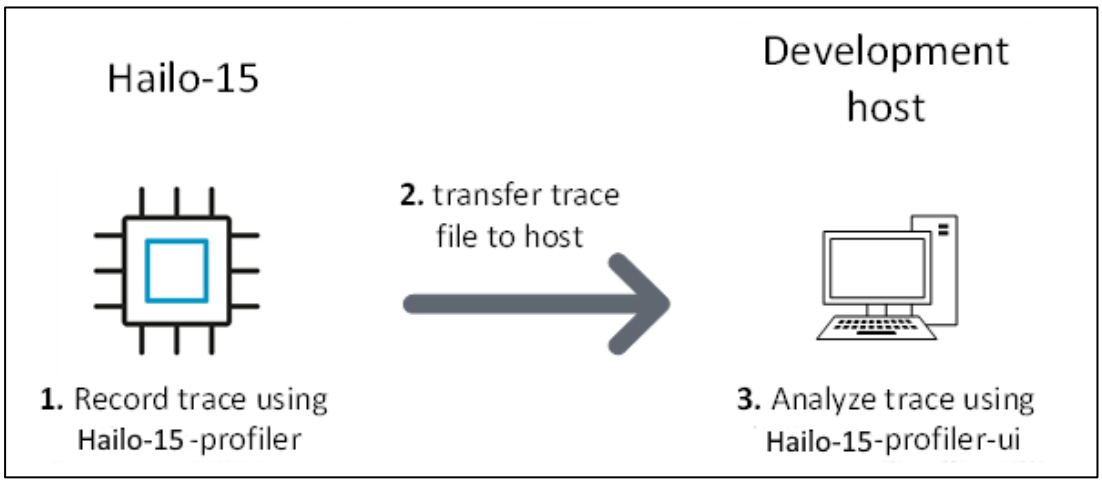


Figure 10. Hailo-15 profiler process flow diagram

6.1.2. Installing and Running the Hailo-15 Profiler

The *'hailo-soc-profiler-ui'* is a Debian (.deb) package that is included as part of the **Hailo-15 vision processor software package**. It is a webserver for the website that is used for viewing and analyzing traces.

To install it, run in the development host terminal:

```
sudo dpkg -i hailo-soc-profiler-ui_<VERSION>_amd64.deb
```

The webserver is set by default to port 15000.

To start the webserver service, run:

```
sudo systemctl start hailo-soc-profiler-ui.service
```

To enable the webserver service during boot, run:

```
sudo systemctl enable hailo-soc-profiler-ui.service
```

The webserver can also be started manually:

For running manually, different port can be assigned using `--port PORT`.

To access the web UI locally, navigate using a web browser to: http://localhost:15000/

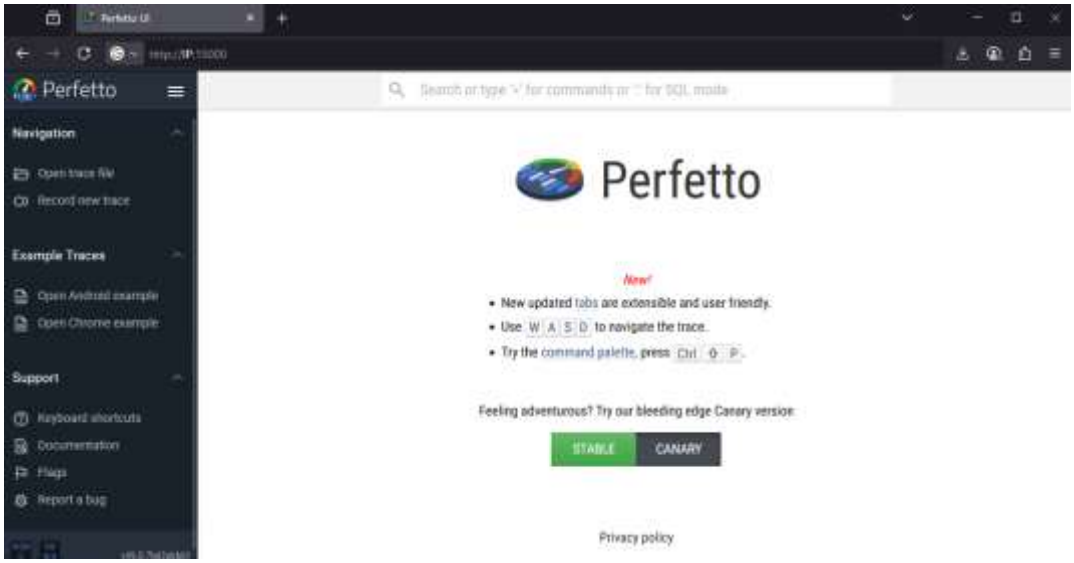


Figure 11. Hailo-15 profiler web UI

6.1.3. Trace Recording

6.1.3.1. Prerequisites

The Hailo-15 Profiler tools that are running on its Application Processor come by default pre-installed on development images core-image-hailo-dev and are absent from core-image-hailo.

The Hailo-15 Profiler has a service that starts the tracing infrastructure during the Application Processor boot.

The service status can be checked using the following command in a Hailo-15 terminal:

```
service hailo-soc-profiler-service status
```

6.1.3.2. Trace Recording

For recording traces, use the `hailo-soc-profiler` tool on the Hailo-15H/L VPU.

The traces are recorded into a file on the Hailo-15 device and the user should transfer this file to the development host for analysis.

Command Usage:

```
hailo-soc-profiler CONFIG [CONFIG...] -t|--time N[s,m,h] [-o out_path.trace]
```

It is possible to supply multiple recording configurations (`CONFIG`), refer to Section 1.4.1 for the description of each configuration.

Example (Hailo-15H):

```
hailo-soc-profiler applications noc-bandwidth-dsp-encoder-isp -t 10s -o soc.trace
```

Example (Hailo-15L):

```
hailo-soc-profiler applications noc-bandwidth-vpu -t 10s -o soc.trace
```

The above example commands record application traces and NoC bandwidth for 10 seconds into a file named *'soc.trace'*.

Example output:

```
root@hailo15:~# hailo-soc-profiler applications noc-bandwidth-dsp-encoder-isp -t 10s -o soc.trace
Starting trace with combined configuration
  - Duration: 10s (10s)
  - Output file: soc.trace
  - Buffer size: 128MB
  - Using 2 config sources
[093.431]    perfetto_cmd.cc:1046 Connected to the Perfetto traced service, TTL: 10s
[104.640]    perfetto_cmd.cc:1209 Wrote 5144797 bytes into soc.trace
Trace completed successfully. Output saved to: soc.trace
```

Figure 12. Record system traces using hailo-soc-profiler command

For detailed description for this command, run it with the --help: parameter.

```
hailo-soc-profiler --help
```

6.1.3.3. Recording Configurations

The `hailo-soc-profiler` supports the following predefined recording configurations for the supported data sources. See Table 13 Multiple configurations can be combined for recording multiple data sources together.

Table 13. Predefined recording configurations

Configuration name	Explanation
applications / applications_detailed	Captures traces from all user applications. The 'application_detailed configuration' is used to capture more detailed events with additional granularity, note that using this configuration

	increases trace file size and may impact system performance.
sched	Captures traces of the Linux kernel scheduler context-switches. Shows which A53 core runs which application at any given time.
noc-bandwidth-*	Captures NoC bandwidth counters. See Section 6.1.3.3.1 for explanations of the various noc-bandwidth counter modes. **Note:** Only 1 NoC-bandwidth configuration can be active at a time
mem_tracker	Captures traces from all existing memory trackers, using the mem_tracker infrastructure. The current implementation includes support for CMA and DMA memories. This will display • Memory allocation / free events • Buffers lifetime data • Total allocated count tracks • Specific CMA heap statistics. • DMA allocation statistics. To view information about buffers that were allocated (and not freed) before starting the hailo-soc-profiler tool, add the --new-mem-tracker-session flag. The module is configurable via */sys/kernel/debug/mem_trackers/.* For more information about the tracker framework, and the debugfs interface, you can check the documentation in the kernel's source. The documentation can be found under "*drivers/media/platform/hailo/mem_tracker/MEMORY_TRACKER_FRAMEWORK.md*"

6.1.3.3.1. NoC Bandwidth Configurations

The different noc-bandwidth-* configurations measure different counters.

As a default:

For Hailo-15H: use noc-bandwidth-dsp-encoder-isp

For Hailo-15L: use noc-bandwidth-vpu.

All the modes measure the following counters in addition to the counters in the table (UI name in parentheses):

- Total bandwidth (Total BW)
- NNC total configuration bandwidth (csm)
- NNC data RX bandwidth (dsm_rx)
- NNC data TX bandwidth (dsm_tx)

Table 14. NOC Bandwidth Configurations

Configuration name	Bandwidth counter measured (name in UI is in parentheses)	
	Hailo-15H	**Hailo-15L**
noc-bandwidth-vpu	N/A	• Application processor (ap_cluster_ace) • Encoder (h265) • ISP main path (isp_mp) • DSP (dsp_idma)
noc-bandwidth-ap-rw	• Application processor (ap_cluster_ace) • Application processor read bandwidth (ap_cluster_ace [read]) • Total read bandwidth (Total BW [read])	
noc-bandwidth-ddma-dsp	• Application processor (ap_cluster_ace) • Sum of Application processor, NNC DMAs, CSI RX0 (AP + DDMA + csi_rx0) • DSP	• Application processor (ap_cluster_ace) • Sum of NNC DMA0 and NNC DMA1 (DDMA0, DDMA1 ddrbus) • NNC DMA2 (DDMA2 ddrbus) • DSP
noc-bandwidth-ddma-encoder	• Application Processor (ap_cluster_ace) • NNC DMA0 • Encoder (h265)	

noc-bandwidth-ddma	• Application Processor (ap_cluster_ace) • Sum of Application Processor, NNC DMAs, CSI RX0 (AP + DDMA + csi_rx0) • NNC DMA descriptors (dram_dma_ddrbus_desc)	• Application Processor (ap_cluster_ace) • Sum of NNC DMA0 and NNC DMA1 (DDMA0, DDMA1 ddrbus) • NNC DMA2 (DDMA2 ddrbus) • NNC DMA descriptors (dram_dma_ddrbus_desc)
noc-bandwidth-dsp-encoder-isp	• DSP • Encoder (h265) • ISP Main Path (isp_mp)	Use noc-bandwidth-vpu instead.
noc-bandwidth-dsp	• Application Processor (ap_cluster_ace) • DSP read bandwidth (dsp read) • DSP write bandwidth (dsp write)	• Application Processor (ap_cluster_ace) • DSP read bandwidth (dsp_idma [read]) • DSP write bandwidth (dsp_idma [write])
noc-bandwidth-encoder	• Application Processor (ap_cluster_ace) • Encoder (h265) • Encoder (h265 [read])	
noc-bandwidth-imaging	• Application Processor (ap_cluster_ace) • Encoder (h265) • ISP Main Path (isp_mp)	Use noc-bandwidth-vpu instead
noc-bandwidth-isp-mp	• Application Processor (ap_cluster_ace) • NNC DMA0 (dram_dma0_ddrbus) • ISP main path (isp_mp)	Use noc-bandwidth-vpu instead
noc-bandwidth-isp	• Application Processor (ap_cluster_ace) • ISP main path (isp_mp) • ISP MCM (isp_mcm)	
noc-bandwidth-sdio	• Application Processor (ap_cluster_ace) • SDIO1 • main2fast	
noc-bandwidth-usb	• Application Processor (ap_cluster_ace) • USB • USB read bandwidth (usb [read])	

6.1.4. Analyzing Traces

On the development host, in the UI website, either drag-and-drop the trace file, or click ‘open trace file’:

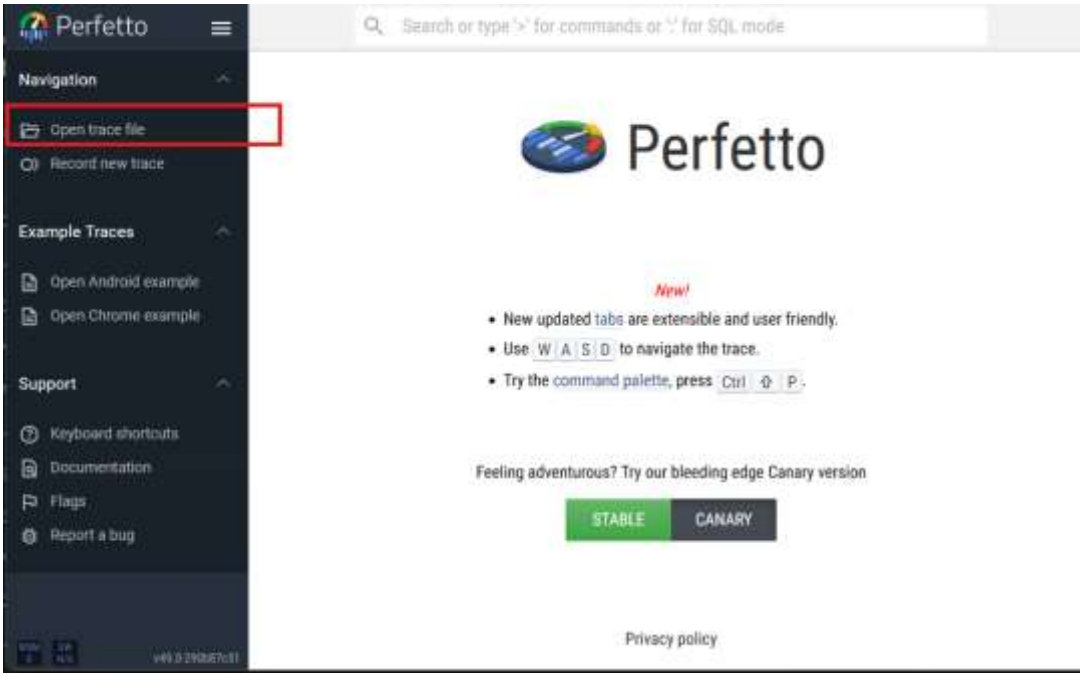


Figure 13. Opening a trace file

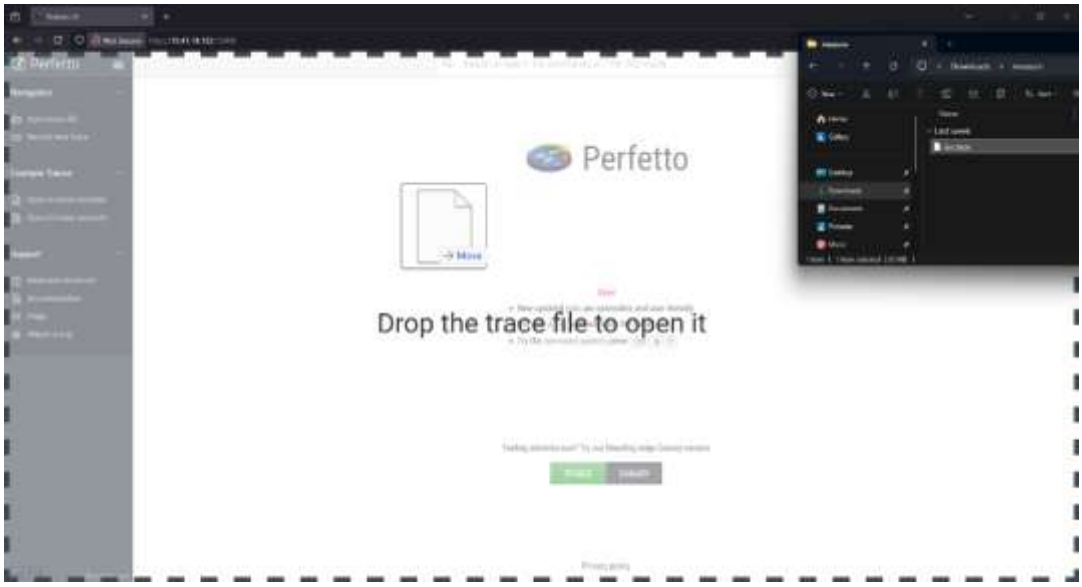


Figure 14. Drag-and-drop trace file

The trace will load.

Initially, most traces are presented collapsed and need to be expanded to view their content.

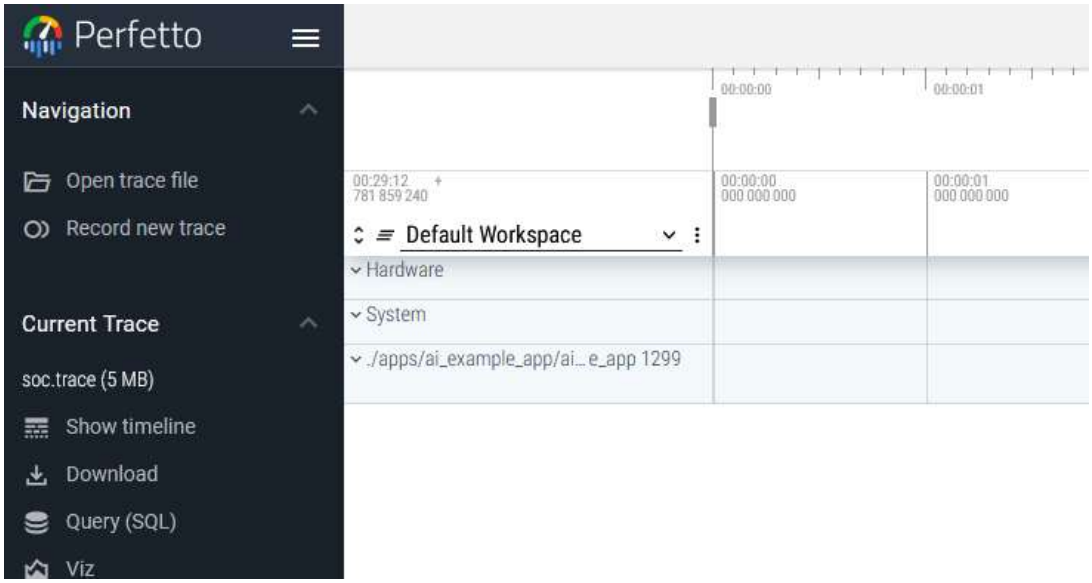


Figure 15. Collapsed traces

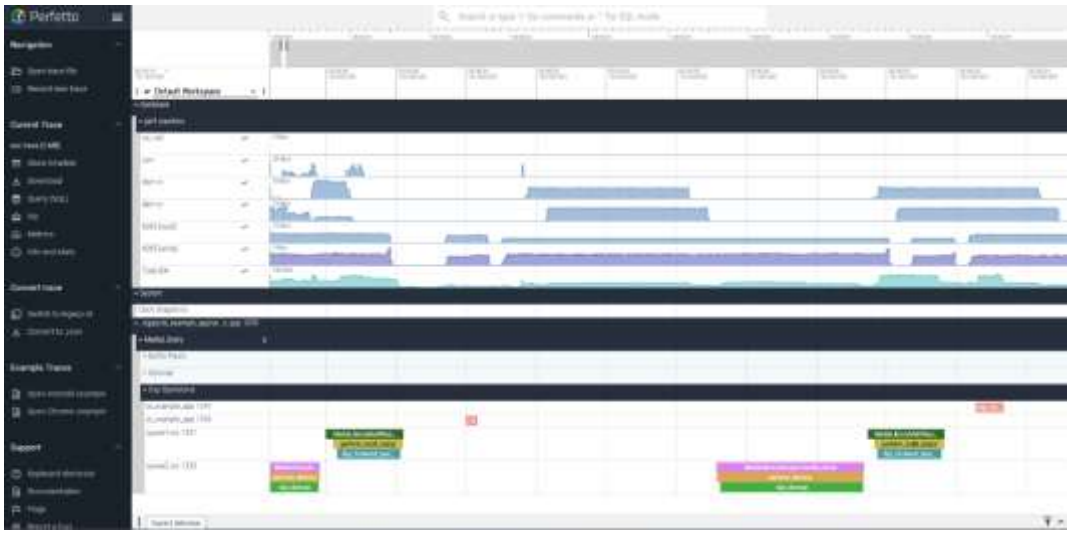


Figure 16. Expanded traces

6.2. NoC Profiler

6.2.1. Overview

The Hailo-15 NoC (Network-on-Chip) has hardware counters used to measure the bandwidth from other masters to the NoC components. This allows for measuring the bandwidth consumption of those masters.

The profiler counts data being transferred from/to the DDR. It supports up to four simultaneous counters. Each counter is associated with a filter which determines which data the counter counts.

In addition, there are three counters measuring the performance of the stream manager: CSM, DSM RX, and DSM TX.

6.2.2. NoC Bandwidth Measurement

All counters are sampled at a specific frequency specified by a given sample-period parameter. Therefore, each sample contains up to four values – one for each counter. Please note that there is a limit of about 1,000,000 samples for each measurement.

A counter value can either be accumulated over the entire measurement or cleared at the start of every sample period. The running-mode parameter controls which mode is set (periodic/accumulated).

A counter can be configured to count only data which passes a given filter that determines if the data is counted based on the component initiated the read/write operation.

Here are the filter-names, corresponding to the hardware component:

```
all
dsp_idma_ro_0
dsp_idma_ro_1
dsp_idma_ro_2
dsp_idma_wo_0
dsp_idma_wo_1
dsp_idma_wo_2
dram_dma0_ddrbus
dram_dma0_ddrbus_lp
dram_dma1_ddrbus
dram_dma1_ddrbus_lp
dram_dma2_ddrbus
dram_dma2_ddrbus_lp
ap_cluster_ace
csi_rx0
```

```
csi_rx1
csi_tx0
debug_etr
dram_dma0_fastbus_ro
dram_dma0_fastbus_wo
dram_dma1_fastbus_ro
dram_dma1_fastbus_wo
dram_dma2_fastbus_ro
dram_dma2_fastbus_wo
dram_dma3_ddrbus_ro
dram_dma3_ddrbus_wo
dram_dma_ddrbus_desc
dsp_ddrbus
ethernet
gic_master
h265
isp_ddrbus0
isp_ddrbus1
isp_fastbus
main2fast
noc_firewall_servic
noc_pcie_firewall_srvice
noc_service
pcie_aux
pcie_desc
pcie_main
sdio1
usb
```

To use the profiler, on the Hailo-15, source its bash script by the terminal command:

```
. /etc/hailo_noc_perf.sh
```

The following functions can now be used:

- `noc_set_counter_total – counter counts every data.`
    - `parameters:`
        - `counter id (0-3)`
- `noc_disable_counter – counter doesn't count any data.`
    - `parameters:`
        - `counter id (0-3)`

- noc_set_counter_filter - counter counts data matching the given parameters.
    - parameters:
        - counter id (0-3)
        - filter name – the name of the component which writes/reads the data
        - opcode (optional) – either write or read. If not specified, both read and write are counted.
- noc_enable_csm – enable measurement of the Configuration Stream Manager
- noc_disable_csm – disable CSM measurement
- noc_enable_dsm – enable measurement of the Data Stream Manager
- noc_disable_csm – disable DSM measurement
- noc_measure_command – perform a measurement for as long as a given command is running.
    - Parameters:
        - command – bash command to run.
        - sample period – in microseconds, must be greater than 30
        - running mode – either 0 (periodic) or 1 (accumulated)
        - output filename
- noc_measure_sleep – perform a measurement for a given amount of time.
    - Parameters:
        - measurement time – in seconds
        - sample period – in microseconds, must be greater than 30
        - running mode – either 0 (periodic) or 1 (accumulated)
        - output filename

For example:

```
# Configure counters
noc_set_counter_filter 0 ap_cluster_ace  # A53 transactions
noc_set_counter_filter 1 all write      # all write transactions
noc_set_counter_total 2                 # all transactions
noc_disable_counter 3

# Measure for 10 seconds
# Set sample interval to 1000us (1ms)
# Use periodic running mode
# Output the results to 'output.txt'
noc_measure_sleep 10 1000 0 output.txt

# Measure for as long as 'mycommand' runs (up to the samples limit)
noc_measure_command  mycommand
```

6.3. CMA Heap info

NAME

hailo-dma-usage.sh

SYNOPSIS

hailo-dma-usage.sh [OPTIONS]

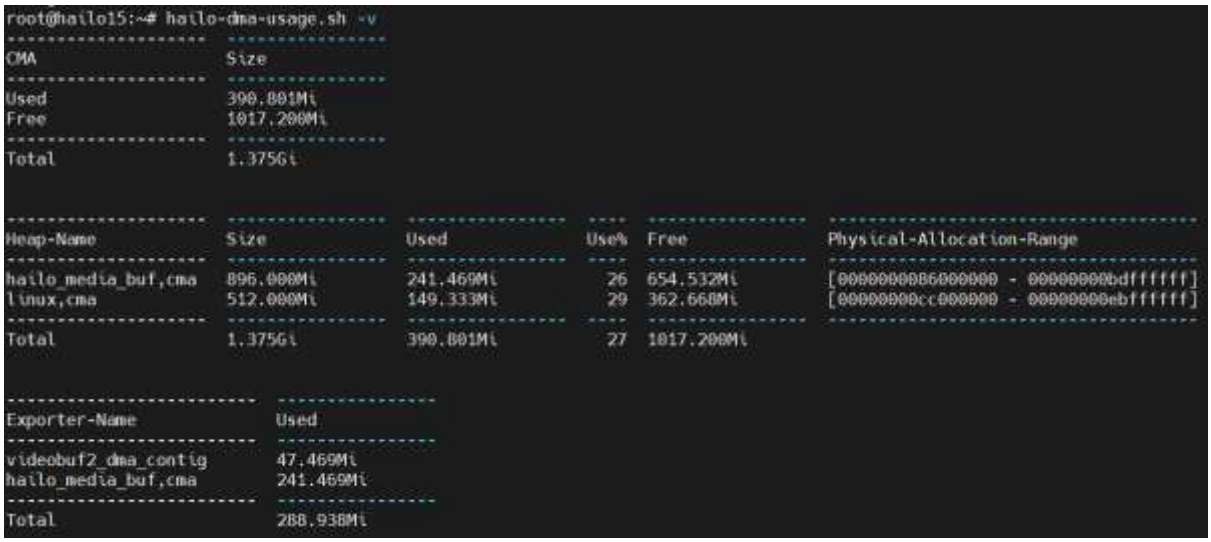
DESCRIPTION

-h|--help: display this help and exit.

-v|--verbose: list also DMA_BUF usage per exporter

-u|--unit B/K/M/G/A: show size format in Bytes/KiB/MiB/GiB units, default to A(auto) format

Usage example:



7. Storage Devices Layout

7.1. SPI Flash Layout (Hailo-15H)

When booting up, the SCU firmware is loaded from SPI flash address 0x0 by the BootROM. The SCU firmware loads U-Boot to memory and releases the AP from reset.

Therefore, there are constraints on the SPI flash memory layout:

- The SCU firmware must reside in SPI flash raw partition at address 0x0
- The first AP execution image (U-Boot SPL) must reside in SPI flash raw partition.

The first 1 MB is reserved for the SCU firmware and U-Boot SPL. The rest of the SPI flash is free to partition and use by the user.

The SPI flash image description and boundaries are described in the following table.

Table 15. SPI flash image

Data	Address Range	Size
SCU-bootloader image	0x0 - 0x5000	20 KB (0x5000)
SCU-bootloader config	0x5000 - 0x6000	4 KB (0x1000)
SCU-bootloader config	0x6000 - 0x7000	4 KB (0x1000)
Device configuration	0x7000 - 0x8000	4 KB (0x1000)
SCU-firmware image	0x8000 - 0x40000	224 KB (0x3_8000)
Boot device tree	0x4_0000 - 0x4_F000	60 KB (0xF000)
Customer key certificate chain	0x4_F000 - 0x5_0000	4 KB (0x1000)
U-boot environmental variables	0x5_0000 - 0x5_4000	16 KB (0x4000)
U-boot SPL image	0x5_4000 - 0x8_0000	176 KB (0x2_C000)

The next images of the Boot chain, U-Boot and the Linux kernel, can be loaded from the SDIO storage. U-Boot and Trusted Firmware specifically can be

loaded from QSPI flash as well.

The required files are to be placed in the first partition of an SDIO storage formatted to FAT filesystem:

- u-boot-tfa.itb –main U-Boot and Trusted Firmware images bundle
- fitImage – monolithic file containing the Linux kernel and it's configuration

7.2. eMMC Layout

Table 16. eMMC image (Hailo-15L)

Data	Address Range	Size
SCU-bootloader image	0x0 - 0x5000	20 KB (0x5000)
SCU-bootloader config	0x5000 - 0x6000	4 KB (0x1000)
SCU-bootloader config	0x6000 - 0x7000	4 KB (0x1000)
Device configuration	0x7000 - 0x8000	4 KB (0x1000)
SCU-firmware image	0x8000 - 0x40000	224 KB (0x3_8000)
Boot device tree	0x4_0000 - 0x4_F000	60 KB (0xF000)
Customer key certificate chain	0x4_F000 - 0x5_0000	4 KB (0x1000)
U-boot environmental variables	0x5_0000 - 0x5_4000	16 KB (0x4000)
U-boot SPL image	0x5_4000 - 0x8_0000	176 KB (0x2_C000)

8. OTA Update

Hailo enables remote OTA (Over-The-Air) software updates by:

- Providing an SCU bootloader
- Integrating a swupdate framework
- Providing optional A/B image fallback

The reference can perform:

- Software image update – pull remote image and update after boot
- Runtime software update – update the inactive image of the A/B images

The swupdate framework includes extensive scripting and update options that can be adjusted according to customer needs. Additional levels may be supported via software. It also includes convenient packaging and tools.

8.1. SCU Bootloader

The SCU Bootloader is the first image running immediately following the BootROM. It oversees the run making sure the system boots properly and will initiate fallbacks if necessary. It has its own configuration file that is loaded during runtime.

It orchestrates:

1. Boot image location and offset
2. Next fallback image in chain
3. Number of boot retries per image

8.2. SCU Bootloader Configuration File

The SCU Bootloader configuration is stored as a binary file in NVM using two identical copies (config 1 and config 2) - to prevent the system from not being able to boot. Each file is protected by a CRC field, to confirm content validity. The content of this configuration is managed by a JSON file, as shown in the example below. During Yocto Project build process of Hailo SW package, this JSON file is automatically converted to a binary file which is ready to be programmed onto the device.

```
{
    "boot_max_retries": "0xFF",
    "image_descriptors": [
        {
            "boot_image_offset": "0x00000000",
            "boot_image_source": “spi_flash”,
            "boot_image_mode": “normal”
        },
        {
            "boot_image_offset": "0x00000000",
            "boot_image_source": “bootstrap”,
            "boot_image_mode": “normal”
        },
        {
            "boot_image_offset": "0x00000000",
            "boot_image_source": “bootstrap”,
            "boot_image_mode": “normal”
        },
        {
            "boot_image_offset": "0x00000000",
            "boot_image_source": “bootstrap”,
            "boot_image_mode": “normal”
        }
    ],
    "last_valid_desc_index": 0,
    "debug_logs_enabled": true
}
```

JSON description:

Field	Description	Values
boot_max_retries	Number of boot retries before switching to next image	Integer 0xFF disables switching images
Image_descriptors.boot_image_offset	Memory address offset of the image	Integer 32-bit address Note – must be 4-bit aligned
Image_descriptors.boot_image_source	Location of image to load	String “bootstrap”, “spi_flash”
Image_descriptors.boot_image_mode	Image loading mode	String “normal”, “remote_update”
Last_valid_desc_index	Highest index implies the no. of actual image currently existing	Integer Values{0..3}
Debug_logs_enabled	Enabling UART-based logs from SCU bootloader	Boolean

Notes about the SCU Bootloader configuration file:

- The binary file size must be 4-byte aligned.
- File is NOT authenticated during secure boot.

8.3. SCU Bootloader Logic

SCU bootloader load sequence:

4. Load SCU bootloader configuration 1.
    a. If corrupted – load configuration 2.
5. If config 2 is corrupted – fallback to UART recovery.
    a. Load the first image, according to configuration.

If the image is corrupted – watchdog timer will expire causing a reboot.

6. Boot the same image again, corresponding to boot_max_retries parameter in configuration.
    a. If reaching max retries, boot the next image, corresponding to last_valid_desc_index parameter in configuration.
7. If there are no more images to try - fallback to UART recovery.

8.4. Software Image Update

The image update process is triggered by commands available in Linux®. The reference implementation requires, among other parameters, image update name and remote IP address to pull the package. A software update application will then load the image update onto the device.

Reference for the software update application:

https://sbabic.github.io/swupdate/index.html

8.5. One-Time Preparation

To enable image updates on the device, it's necessary to prepare relevant memory partitions on the device, and properly setup TFTP service in the host server.
The partition preparation is normally needed only once per device, during device production.

As an example, scripts are provided for partition initialization - for single image and dual image cases.

Limitation

The tables below describe the partitions prepared for each case:

Single image (A only)

Name	Type	Size
boot	FAT32	64MB
rootfs	ext4	N/A (filler to end of memory)

Dual image (A/B)

Name	Type	Size
boot	FAT32	64MB
rootfs	ext4	N/A (filler to end of memory)
boot	FAT32	64MB
rootfs	ext4	4GB
data	ext4	N/A (filler to end of memory)

The partition initialization functionality is available as an example via the following U-Boot commands via u-boot console. Note that these commands, besides partition initialization, are also loading the images into their partitions.

- For updating single image select from the U-Boot boot menu: **SD Card Board Init** See Figure 17
- For updating dual image (A/B) select from the U-Boot boot menu: **SD Card AB Board Init** See Figure 17

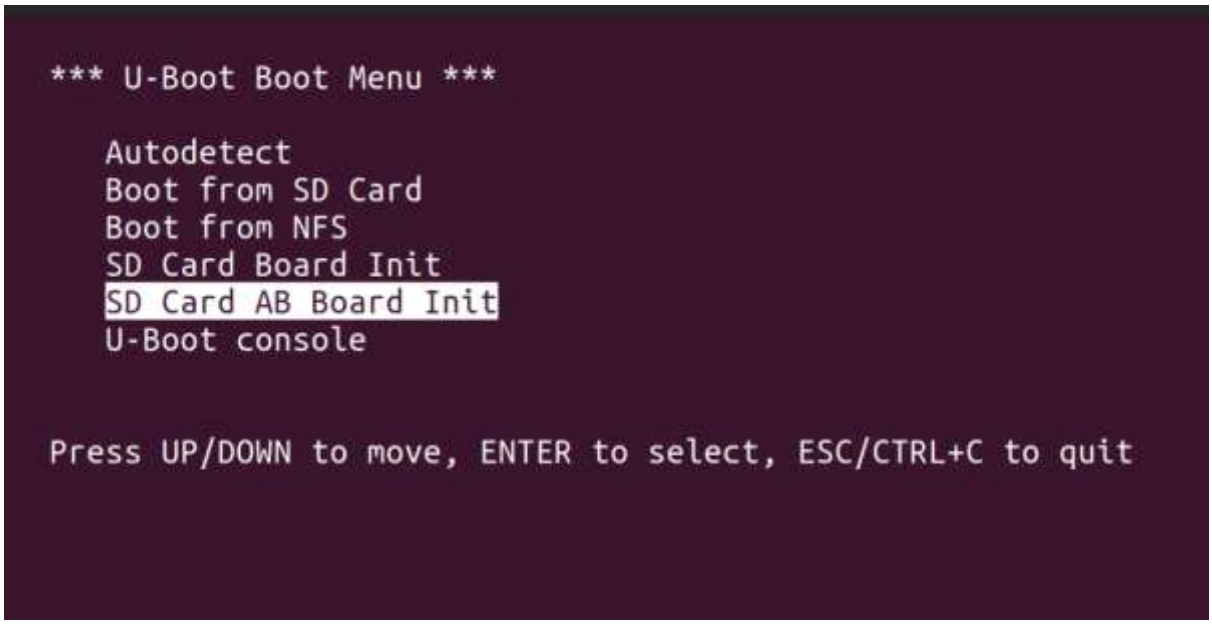


Figure 17. Example of partition initialization from U-Boot menu

Note: data partition can be mounted manually or by editing /etc/fstab.

Setting up TFTP:

Windows Host

TFTPD64 or any other tftpd application

Linux Host

`sudo apt install tftpd-hpa` (install TFTP)
`sudo vi /etc/default/tftpd-hpa` (edit the default TFTP configuration file)

Set the following parameters in the file:

`TFTP_ADDRESS="10.0.0.2:69" (config TFTP address to host IP)`

`TFTP_DIRECTORY=<path to the main kas build images>`

Then save and exit

`sudo systemctl restart tftpd-hpa`

Limitation: When performing this one-time partition preparation, the version on TFTP side should match the version on the device itself – to prevent mismatch. For example, if the device is currently running with release 1.10.1, the fitImage, ext4.gz and *.swu files on TFTP server should also be based on release 1.5.0.

8.6. Image Update Procedure

8.6.1. Triggering

The scripts for triggering image update are in folder /etc of rootfs. In the examples below, a specific software update image name is used, and specific server IP address.

The procedure is triggered by the following commands, either for single or dual image modes:

- Single image mode (A only):

For Hailo-15H `./run_swupdate.sh -r hailo-update-image-hailo15-sbc-rev3-1.swu -s 10.0.0.2`

For Hailo-15L `./run_swupdate.sh -r hailo-update-image-hailo15l-sbc.swu -s 10.0.0.2`

- Dual image mode (A/B):

For Hailo-15H `./run_swupdate.sh -d -r hailo-update-image-hailo15-sbc-rev3-1.swu -s 10.0.0.2`

For Hailo-15L `./run_swupdate.sh -d -r hailo-update-image-hailo15l-sbc.swu -s 10.0.0.2`

Where the *.swu* file is the swupdate package file and *10.0.0.2* is the remote IP to pull it from.

Default protocol for loading the image is TFTP, as given in our example recipes for swupdate.
The protocol and the parameters provided (IP address, package filename) may be modified in the following scripts:

- */meta-hailo-soc/meta-hailo-bsp-examples/hailo-swupdate/recipes-core/initscripts-hailo-swupdate/initscripts-hailo-swupdate/rcS.swupdate*
- */meta-hailo-soc/meta-hailo-bsp-examples/hailo-swupdate/recipes-bsp/run-swupdate-script/files/run_swupdate.sh*

8.6.2. Actions Performed During swupdate

run_swupdate.sh script performs the following steps:

1. *.swu package is downloaded from remote server into the device.
2. swupdate application is triggered – files and images are extracted from *.swu and installed according to specification in sw_description.
3. For dual-image A/B update mode –

- Extraction is performed either to the location of image A or B. This is determined by the image used when booting. For example – if device has

booted with image A, update is done to location of image B. In the next reboot – device will boot with image B.

The filesystem in Hailo linux distribution includes the following scu bootloader configuration files (in folder /etc/scu_bl_cfg):

- scu_bl_cfg_a.bin – indicating a single image, at offset 0x8000
- scu_bl_cfg_a_remote_update.bin – same as the above, plus an indication for U-Boot of remote update mode (see 8.7)
- scu_bl_cfg_b.bin – indicating a single image, at offset 0x80000

Upon A/B update - configuration file usage is switched between A and B. This implies writing new content into the NVM locations (config 1 and config 2) which are used by SCU bootloader upon boot.

As stated above, two identical copies of the configuration should always be present in NVM. Therefore, the following configuration update procedure is defined:

4. Read SCU-BL config 1 and verify its CRC.
5. If valid, then
    a. Compare config 1 to config 2
    b. If these are different
        i. Copy config 1 contents to config 2
        ii. Compare again config 1 to config 2
        iii. Abort flow (couldn't copy the configs)
6. If otherwise, read config 2 and verify its CRC
    a. If config 2 is not valid
        i. Abort (no valid config) - fallback to UART recovery
7. Proceed with updating config 1 with the new config
    a. Check config 1 and compare to the wanted new config
    b. If these are different
        i. **Abort** (new config write failed)

or else

        ii. Copy config 1 to config 2

8.6.3. Verifying Image Update Result

It is possible to confirm the image was updated successfully by reading the following file in Linux rootfs:

/sys/devices/soc0/boot_info/active_boot_image_offset

The file content is the memory offset from which the image has booted, this can be cross-checked with the SCU bootloader configuration file.

8.7. Remote Update Boot Sequence

Remote update mode is a single-image update, in which device has booted from image A and a new image A is to be loaded. This process involves U-Boot functionality which detects remote update mode and runs the swupdate application.

Remote update sequence:

1. Triggering the update via command in Linux (as described in 8.6.1).
2. Update SCU Bootloader configuration to indicate remote update mode in boot_image_mode parameter.
3. Device reset.
4. SCU Bootloader reads its configuration, identifying remote update mode. Remote update mode indication is passed to SCU-FW, which then notifies U-Boot.
5. U-Boot detects remote update and boots update image.
6. Update image boots and runs update scripts, after which the device resets again.
7. SCU Bootloader configuration is updated again to normal boot setting.
8. In case the swupdate image boots but the update process fails, the system will initiate remote update again.

8.8. A/B Software Image Backup

The A/B reference allows to store and boot an active image and another backup image as a fallback. This also allows to load and program an update image into the inactive image during runtime while the system runs the active one.

The A/B images include:

1. SCU Firmware
2. U-Boot and U-Boot environment
3. Linux kernel
4. Device trees
5. Customer certificate

8.9. A/B Boot Sequence

The following block diagram displays the boot sequence in a case of dual image existence.

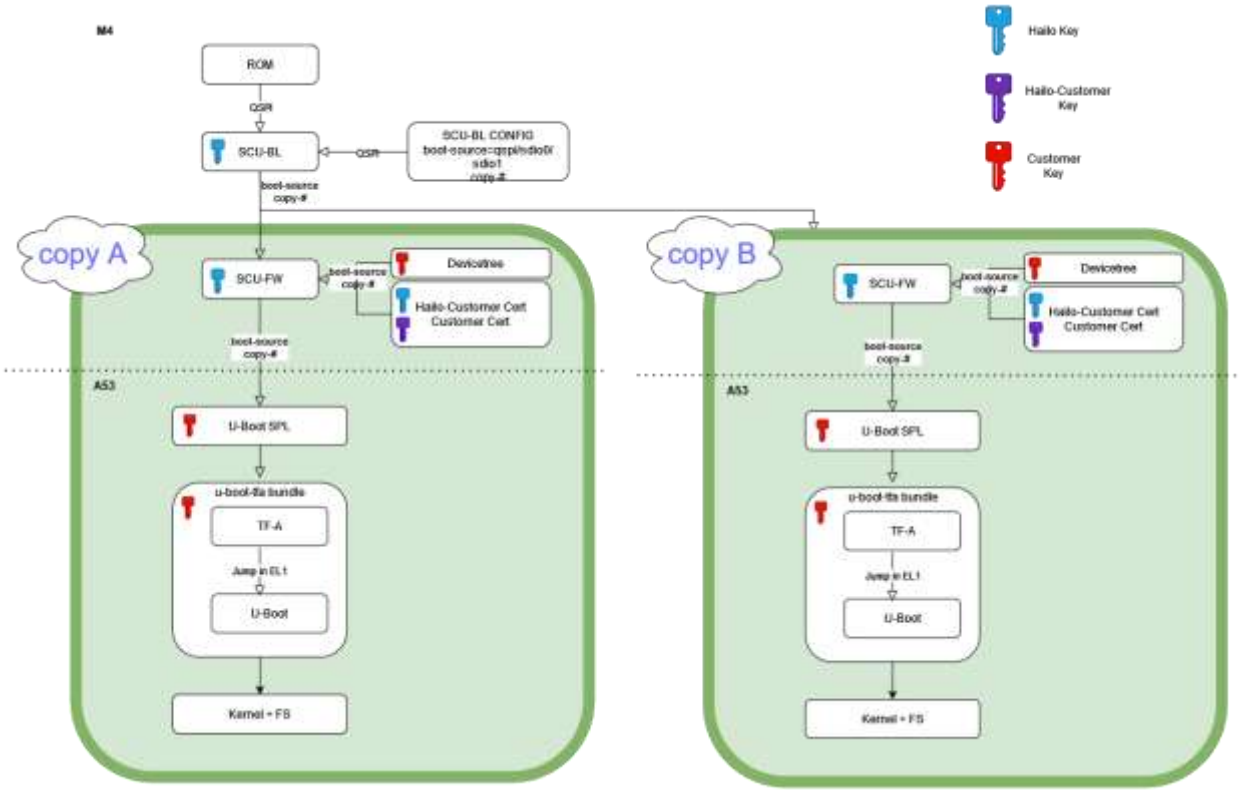


Figure 18. Boot sequence for dual image existence

8.10. A/B Boot SPI Flash Layout

Type	Addresses	Size	Usage
Non A/B	0-0x5_000	20KiB (0x5_000)	SCU-BL image
	0x5_000 - 0x6_000	4KiB (0x1_000)	SCU-BL config **1**
	0x6_000 - 0x7_000	4KiB (0x1_000)	SCU-BL config **2**
	0x7_000 - 0x8_000	4KiB (0x1_000)	Reserved, Device configuration (MAC address)
A	0x8_000-0x40_000	224KiB (0x38_000)	SCU-FW image
	0x40_000-0x4F_000	60KiB (0xF_000)	Boot devicetree
	0x4F_000 - 0x50_000	4KiB (0x1_000)	Customer key certificate chain
	0x50_000-0x54_000	16KiB (0x4_000)	U-boot environment
	0x54_000-0x80_000	176KiB (0x2C_000)	U-boot SPL
B	0x80_000-0xF8_000	480KiB (0x78_000)	Same layout as A

8.11. A/B Boot Partition eMMC Layout

OTA is not yet supported for eMMC.

Note that eMMC boot is only available for Hailo-15L.

9. Supported Functions and Pin Groups

Full list of Supported Functions and Pin Groups and their associated pins.

For Hailo-15H – each pin group may contain multiple pins, therefore attention is needed when assigning group & function to a pin.

For Hailo-15L – each pin group contains a single pin.

9.1. Supported Functions (Hailo-15H)

function 0:	gpio, groups =	gpio, groups = [gpio0_grp gpio1_grp gpio2_grp gpio3_grp gpio4_grp gpio5_grp gpio6_grp gpio7_grp gpio8_grp gpio9_grp gpio10_grp gpio11_grp gpio12_grp gpio13_grp gpio14_grp gpio15_grp gpio16_grp gpio17_grp gpio18_grp gpio19_grp gpio20_grp gpio21_grp gpio22_grp gpio23_grp gpio24_grp gpio25_grp gpio26_grp gpio27_grp gpio28_grp gpio29_grp gpio30_grp gpio31_grp]
function 1:	uart2, groups =	uart2, groups = [uart2_0_grp uart2_1_grp uart2_2_grp uart2_3_grp uart2_4_grp]
function 2:	uart3, groups =	uart3, groups = [uart3_0_grp uart3_1_grp uart3_2_grp uart3_3_grp]
function 3:	uart0_cts_rts, groups =	uart0_cts_rts, groups = [uart0_cts_rts_0_grp uart0_cts_rts_1_grp]
function 4:	uart1_cts_rts, groups =	uart1_cts_rts, groups = [uart1_cts_rts_0_grp uart1_cts_rts_1_grp]
function 5:	uart2_cts_rts, groups =	uart2_cts_rts, groups = [uart2_cts_rts_grp]
function 6:	uart3_cts_rts, groups =	uart3_cts_rts, groups = [uart3_cts_rts_grp]
function 7:	i2c0_current_src_en_out, groups =	i2c0_current_src_en_out, groups = [i2c0_current_src_en_out_0_grp i2c0_current_src_en_out_1_grp i2c0_current_src_en_out_2_grp i2c0_current_src_en_out_3_grp]
function 8:	i2c1_current_src_en_out, groups =	i2c1_current_src_en_out, groups = [i2c1_current_src_en_out_0_grp i2c1_current_src_en_out_1_grp i2c1_current_src_en_out_2_grp i2c1_current_src_en_out_3_grp i2c1_current_src_en_out_4_grp]
function 9:	i2c2_current_src_en_out, groups =	i2c2_current_src_en_out, groups = [i2c2_current_src_en_out_0_grp i2c2_current_src_en_out_1_grp i2c2_current_src_en_out_2_grp i2c2_current_src_en_out_3_grp]
function 10:	i2c3_current_src_en_out, groups =	i2c3_current_src_en_out, groups = [i2c3_current_src_en_out_0_grp i2c3_current_src_en_out_1_grp i2c3_current_src_en_out_2_grp]
function 11:	pwm0, groups =	pwm0, groups = [pwm0_0_grp pwm0_1_grp pwn0_2_grp]

function 12:	pwm1, groups =	pwm1, groups = [pwm1_grp]
function 13:	pwm2, groups =	pwm2, groups = [pwm2_grp]
function 14:	i2c2, groups =	i2c2, groups = [i2c2_0_grp i2c2_1_grp]
function 15:	i2c3, groups =	i2c3, groups = [i2c3_0_grp i2c3_1_grp]
function 16:	parallel_in_16pins, groups =	parallel_in_16pins, groups = [parallel_in_16pins_grp]
function 17:	parallel_in_24pins, groups =	parallel_in_24pins, groups = [parallel_in_24pins_grp]
function 18:	timer_ext_in0, groups =	timer_ext_in0, groups = [timer_ext_in0_grp]
function 19:	timer_ext_in2, groups =	timer_ext_in2, groups = [timer_ext_in2_grp]
function 20:	timer_ext_in3, groups =	timer_ext_in3, groups = [timer_ext_in3_grp]
function 21:	safety_out0, groups =	safety_out0, groups = [safety_out0_0_grp safety_out0_1_grp safety_out0_2_grp safety_out0_3_grp]
function 22:	safety_out1, groups =	safety_out1, groups = [safety_out1_0_grp safety_out1_1_grp safety_out1_2_grp safety_out1_3_grp]
function 23:	sdio0_gp_out, groups =	sdio0_gp_out, groups = [sdio0_gp_out_0_grp sdio0_gp_out_1_grp]
function 24:	sdio0_gp_in, groups =	sdio0_gp_in, groups = [sdio0_gp_in_grp]
function 25:	sdio0_CD_in, groups =	sdio0_CD_in, groups = [sdio0_CD_in_grp]
function 26:	sdio0_wp_in, groups =	sdio0_wp_in, groups = [sdio0_wp_in_grp]
function 27:	sdio1_gp_out, groups =	sdio1_gp_out, groups = [sdio1_gp_out_0_grp sdio1_gp_out_1_grp sdio1_gp_out_2_grp sdio1_gp_out_3_grp]
function 28:	sdio1_gp_in, groups =	sdio1_gp_in, groups = [sdio1_gp_in_grp]
function 29:	sdio1_CD_in, groups =	sdio1_CD_in, groups = [sdio1_CD_in_grp]
function 30:	sdio1_wp_in, groups =	sdio1_wp_in, groups = [sdio1_wp_in_grp]
function 31:	initial_debug_bus_out, groups =	initial_debug_bus_out, groups = [initial_debug_bus_out_grp]
function 32:	flash_cs_out_pad_2, groups =	flash_cs_out_pad_2, groups = [flash_cs_out_pad_2_grp]
function 33:	flash_cs_out_pad_3, groups =	flash_cs_out_pad_3, groups = [flash_cs_out_pad_3_grp]
function 34:	isp_flash_trig_out, groups =	isp_flash_trig_out, groups = [isp_flash_trig_out_grp]
function 35:	isp_pre_flash_out, groups =	isp_pre_flash_out, groups = [isp_pre_flash_out_grp]
function 36:	I2S_SD, groups =	I2S_SD, groups = [I2S_SD_grp]
function 37:	usb_overcurrent_in, groups =	usb_overcurrent_in, groups = [usb_overcurrent_in_grp]
function 38:	usb_drive_vbus_out, groups =	usb_drive_vbus_out, groups = [usb_drive_vbus_out_0_grp usb_drive_vbus_out_1_grp usb_drive_vbus_out_2_grp]

function 39:	pcie_mperst_out, groups =	pcie_mperst_out, groups = [pcie_mperst_out_grp]
function 40:	cpu_trace_clk, groups =	cpu_trace_clk, groups = [cpu_trace_clk_grp]
function 41:	cpu_trace_data, groups =	cpu_trace_data, groups = [cpu_trace_data_grp]

9.2. Supported Pin Groups (Hailo-15H)

Registered pin groups:

group: gpio0_grp	
	pin 0 (GPIO_0)
group: gpio1_grp	
	pin 1 (GPIO_1)
group: gpio2_grp	
	pin 2 (GPIO_2)
group: gpio3_grp	
	pin 3 (GPIO_3)
group: gpio4_grp	
	pin 4 (GPIO_4)
group: gpio5_grp	
	pin 5 (GPIO_5)
group: gpio6_grp	
	pin 6 (GPIO_6)
group: gpio7_grp	
	pin 7 (GPIO_7)
group: gpio8_grp	
	pin 8 (GPIO_8)
group: gpio9_grp	
	pin 9 (GPIO_9)
group: gpio10_grp	
	pin 10 (GPIO_10)
group: gpio11_grp	
	pin 11 (GPIO_11)
group: gpio12_grp	

	pin 12 (GPIO_12)
group: gpio13_grp	
	pin 13 (GPIO_13)
group: gpio14_grp	
	pin 14 (GPIO_14)
group: gpio15_grp	
	pin 15 (GPIO_15)
group: gpio16_grp	
	pin 16 (GPIO_16)
group: gpio17_grp	
	pin 17 (GPIO_17)
group: gpio18_grp	
	pin 18 (GPIO_18)
group: gpio19_grp	
	pin 19 (GPIO_19)
group: gpio20_grp	
	pin 20 (GPIO_20)
group: gpio21_grp	
	pin 21 (GPIO_21)
group: gpio22_grp	
	pin 22 (GPIO_22)
group: gpio23_grp	
	pin 23 (GPIO_23)
group: gpio24_grp	
	pin 24 (GPIO_24)
group: gpio25_grp	
	pin 25 (GPIO_25)
group: gpio26_grp	

	pin 26 (GPIO_26)
group: gpio27_grp	
	pin 27 (GPIO_27)
group: gpio28_grp	
	pin 28 (GPIO_28)
group: gpio29_grp	
	pin 29 (GPIO_29)
group: gpio30_grp	
	pin 30 (GPIO_30)
group: gpio31_grp	
	pin 31 (GPIO_31)
group: I2S_SD_grp	
	pin 5 (GPIO_5)
	pin 24 (GPIO_24)
group: cpu_trace_clk_grp	
	pin 27 (GPIO_27)
group: cpu_trace_data_grp	
	pin 28 (GPIO_28)
	pin 29 (GPIO_29)
	pin 30 (GPIO_30)
	pin 31 (GPIO_31)
group: flash_cs_out_pad_2_grp	
	pin 0 (GPIO_0)
group: flash_cs_out_pad_3_grp	
	pin 1 (GPIO_1)
group: i2c0_current_src_en_out_0_grp	
	pin 0 (GPIO_0)
group: i2c0_current_src_en_out_1_grp	

	pin 2 (GPIO_2)
group: i2c0_current_src_en_out_2_grp	
	pin 24 (GPIO_24)
group: i2c0_current_src_en_out_3_grp	
	pin 25 (GPIO_25)
group: i2c1_current_src_en_out_0_grp	
	pin 1 (GPIO_1)
group: i2c1_current_src_en_out_1_grp	
	pin 3 (GPIO_3)
group: i2c1_current_src_en_out_2_grp	
	pin 13 (GPIO_13)
group: i2c1_current_src_en_out_3_grp	
	pin 25 (GPIO_25)
group: i2c1_current_src_en_out_4_grp	
	pin 26 (GPIO_26)
group: i2c2_0_grp	
	pin 6 (GPIO_6)
	pin 7 (GPIO_7)
group: i2c2_1_grp	
	pin 22 (GPIO_22)
	pin 23 (GPIO_23)
group: i2c2_current_src_en_out_0_grp	
	pin 0 (GPIO_0)
group: i2c2_current_src_en_out_1_grp	
	pin 2 (GPIO_2)
group: i2c2_current_src_en_out_2_grp	
	pin 4 (GPIO_4)
group: i2c2_current_src_en_out_3_grp	

	pin 24 (GPIO_24)
group: i2c3_0_grp	
	pin 6 (GPIO_6)
	pin 7 (GPIO_7)
group: i2c3_1_grp	
	pin 20 (GPIO_20)
	pin 21 (GPIO_21)
group: i2c3_current_src_en_out_0_grp	
	pin 1 (GPIO_1)
group: i2c3_current_src_en_out_1_grp	
	pin 3 (GPIO_3)
group: i2c3_current_src_en_out_2_grp	
	pin 5 (GPIO_5)
group: initial_debug_bus_out_grp	
	pin 0 (GPIO_0)
	pin 1 (GPIO_1)
	pin 2 (GPIO_2)
	pin 3 (GPIO_3)
	pin 4 (GPIO_4)
	pin 5 (GPIO_5)
group: isp_flash_trig_out_grp	
	pin 2 (GPIO_2)
group: isp_pre_flash_out_grp	
	pin 3 (GPIO_3)
group: parallel_in_16pins_grp	
	pin 6 (GPIO_6)
	pin 7 (GPIO_7)
	pin 16 (GPIO_16)

	pin 17 (GPIO_17)
	pin 18 (GPIO_18)
	pin 19 (GPIO_19)
	pin 20 (GPIO_20)
	pin 21 (GPIO_21)
	pin 22 (GPIO_22)
	pin 23 (GPIO_23)
	pin 24 (GPIO_24)
	pin 25 (GPIO_25)
	pin 26 (GPIO_26)
	pin 27 (GPIO_27)
	pin 28 (GPIO_28)
	pin 29 (GPIO_29)
	pin 30 (GPIO_30)
	pin 31 (GPIO_31)
group: parallel_in_24pins_grp	
	pin 6 (GPIO_6)
	pin 7 (GPIO_7)
	pin 8 (GPIO_8)
	pin 9 (GPIO_9)
	pin 10 (GPIO_10)
	pin 11 (GPIO_11)
	pin 12 (GPIO_12)
	pin 13 (GPIO_13)
	pin 14 (GPIO_14)
	pin 15 (GPIO_15)
	pin 16 (GPIO_16)
	pin 17 (GPIO_17)

	pin 18 (GPIO_18)
	pin 19 (GPIO_19)
	pin 20 (GPIO_20)
	pin 21 (GPIO_21)
	pin 22 (GPIO_22)
	pin 23 (GPIO_23)
	pin 24 (GPIO_24)
	pin 25 (GPIO_25)
	pin 26 (GPIO_26)
	pin 27 (GPIO_27)
	pin 28 (GPIO_28)
	pin 29 (GPIO_29)
	pin 30 (GPIO_30)
	pin 31 (GPIO_31)
group: pcie_mperst_out_grp	
	pin 15 (GPIO_15)
group: pwm0_0_grp	
	pin 0 (GPIO_0)
	pin 1 (GPIO_1)
group: pwm0_1_grp	
	pin 20 (GPIO_20)
	pin 21 (GPIO_21)
group: pwm0_2_grp	
	pin 27 (GPIO_27)
	pin 28 (GPIO_28)
group: pwm1_grp	
	pin 2 (GPIO_2)
	pin 3 (GPIO_3)

group: pwm2_grp	
	pin 4 (GPIO_4)
	pin 5 (GPIO_5)
group: safety_out0_0_grp	
	pin 0 (GPIO_0)
group: safety_out0_1_grp	
	pin 2 (GPIO_2)
group: safety_out0_2_grp	
	pin 4 (GPIO_4)
group: safety_out0_2_grp	
	pin 4 (GPIO_4)
group: safety_out1_0_grp	
	pin 1 (GPIO_1)
group: safety_out1_1_grp	
	pin 3 (GPIO_3)
group: safety_out1_2_grp	
	pin 5 (GPIO_5)
group: safety_out1_3_grp	
	pin 12 (GPIO_12)
group: sdio0_CD_in_grp	
	pin 10 (GPIO_10)
group: sdio0_gp_in_grp	
	pin 9 (GPIO_9)
group: sdio0_gp_out_0_grp	
	pin 0 (GPIO_0)
group: sdio0_gp_out_1_grp	
	pin 4 (GPIO_4)
group: sdio0_wp_in_grp	

	pin 6 (GPIO_6)
group: sdio1_CD_in_grp	
	pin 14 (GPIO_14)
group: sdio1_gp_in_grp	
	pin 13 (GPIO_13)
group: sdio1_gp_out_0_grp	
	pin 1 (GPIO_1)
group: sdio1_gp_out_1_grp	
	pin 3 (GPIO_3)
group: sdio1_gp_out_2_grp	
	pin 5 (GPIO_5)
group: sdio1_gp_out_3_grp	
	pin 26 (GPIO_26)
group: sdio1_wp_in_grp	
	pin 7 (GPIO_7)
group: timer_ext_in0_grp	
	pin 18 (GPIO_18)
group: timer_ext_in1_grp	
	pin 19 (GPIO_19)
group: timer_ext_in2_grp	
	pin 6 (GPIO_6)
group: timer_ext_in3_grp	
	pin 7 (GPIO_7)
group: uart0_cts_rts_0_grp	
	pin 10 (GPIO_10)
	pin 11 (GPIO_11)
group: uart0_cts_rts_1_grp	
	pin 30 (GPIO_30)

	pin 31 (GPIO_31)
group: uart1_cts_rts_0_grp	
	pin 6 (GPIO_6)
	pin 7 (GPIO_7)
group: uart1_cts_rts_1_grp	
	pin 26 (GPIO_26)
	pin 27 (GPIO_27)
group: uart2_0_grp	
	pin 6 (GPIO_6)
	pin 0 (GPIO_0)
group: uart2_1_grp	
	pin 6 (GPIO_6)
	pin 2 (GPIO_2)
group: uart2_2_grp	
	pin 6 (GPIO_6)
	pin 4 (GPIO_4)
group: uart2_3_grp	
	pin 8 (GPIO_8)
	pin 9 (GPIO_9)
group: uart2_4_grp	
	pin 16 (GPIO_16)
	pin 17 (GPIO_17)
group: uart2_cts_rts_grp	
	pin 28 (GPIO_28)
	pin 29 (GPIO_29)
group: uart3_0_grp	
	pin 7 (GPIO_7)
	pin 1 (GPIO_1)

group: uart3_1_grp	
	pin 7 (GPIO_7)
	pin 3 (GPIO_3)
group: uart3_2_grp	
	pin 7 (GPIO_7)
	pin 5 (GPIO_5)
group: uart3_3_grp	
	pin 12 (GPIO_12)
	pin 13 (GPIO_13)
group: uart3_cts_rts_grp	
	pin 6 (GPIO_6)
	pin 7 (GPIO_7)
group: usb_drive_vbus_out_0_grp	
	pin 2 (GPIO_2)
group: usb_drive_vbus_out_1_grp	
	pin 5 (GPIO_5)
group: usb_drive_vbus_out_2_grp	
	pin 12 (GPIO_12)
group: usb_overcurrent_in_grp	
	pin 11 (GPIO_11)

9.3. Supported Functions and Pin Groups (Hailo-15L)

Pin Name					Available Functions				
UART0_RXD	uart0_rxd_in_0_grp/uart0_rxd_in	can0_rx_in_0_grp/can0_rx_in	can1_rx_in_0_grp/can1_rx_in						
UART0_TXD	uart0_txd_out_0_grp/uart0_txd_out	can0_tx_out_0_grp/can0_tx_out	can1_tx_out_0_grp/can1_tx_out	can0_stby_out_0_grp/can0_stby_out	can1_stby_out_0_grp/can1_stby_out				
UART1_RXD	uart1_rxd_in_0_grp/uart1_rxd_in	can0_rx_in_1_grp/can0_rx_in	can1_rx_in_1_grp/can1_rx_in						
UART1_TXD	uart1_txd_out_0_grp/uart1_txd_out	can0_tx_out_1_grp/can0_tx_out	can1_tx_out_1_grp/can1_tx_out	can0_stby_out_1_grp/can0_stby_out	can1_stby_out_1_grp/can1_stby_out				
ETH_MDC	eth_mdc_out_grp/eth_mdc_out	gpio16_0_grp/gpio16	gpio0_0_grp/gpio0						
ETH_MDIO	eth_mdio_grp/eth_mdio	gpio17_0_grp/gpio17	gpio1_0_grp/gpio1						
ETH_RX_CLK	eth_rx_clk_in_grp/eth_rx_clk_in	gpio18_0_grp/gpio18	gpio2_0_grp/gpio2						

ETH_RX_CTL_DV	eth_rx_ctl_crs_dv_in_grp/eth_rx_ctl_crs_dv_in	gpio19_0_grp/gpio19	gpio3_0_grp/gpio3						
ETH_RXD0	eth_rxd_0_in_grp/eth_rxd_0_in	gpio20_0_grp/gpio20	gpio4_0_grp/gpio4	uart0_cts_in_0_grp/uart0_cts_in					
ETH_RXD1	eth_rxd_1_in_grp/eth_rxd_1_in	gpio21_0_grp/gpio21	gpio5_0_grp/gpio5	uart1_cts_in_0_grp/uart1_cts_in					
ETH_RXD2	eth_rxd_2_in_grp/eth_rxd_2_in	gpio22_0_grp/gpio22	gpio6_0_grp/gpio6	pwm0_out_0_grp/pwm0_out	uart2_cts_in_0_grp/uart2_cts_in	can0_rx_in_2_grp/can0_rx_in	safety_error_n_out_0_grp/safety_error_n_out	i2s0_sdi1_in_0_grp/i2s0_sdi1_in	can1_rx_in_2_grp/can1_rx_in
ETH_RXD3	eth_rxd_3_er_in_grp/eth_rxd_3_er_in	gpio23_0_grp/gpio23	gpio7_0_grp/gpio7						
ETH_TX_CLK	eth_tx_clk_out_grp/eth_tx_clk_out	gpio24_0_grp/gpio24	gpio8_0_grp/gpio8	i2c2_scl_0_grp/i2c2_scl					
ETH_TX_CTL_EN	eth_tx_ctl_en_out_grp/eth_tx_ctl_en_out	gpio25_0_grp/gpio25	gpio9_0_grp/gpio9						
ETH_TXD0	eth_txd_0_out_grp/eth_txd_0_out	gpio26_0_grp/gpio26	gpio10_0_grp/gpio10	uart0_rts_out_0_grp/uart0_rts_out	cpu_trace_data0_out_0_grp/cpu_trace_data0_out				
ETH_TXD1	eth_txd_1_out_grp/eth_txd_1_out	gpio27_0_grp/gpio27	gpio11_0_grp/gpio11	uart1_rts_out_0_grp/uart1_rts_out	cpu_trace_data1_out_0_grp/cpu_trace_data1_out				

ETH_TXD2	eth_txd_2_out_grp/eth_txd_2_out	gpio28_0_grp/gpio28	gpio12_0_grp/gpio12	pwm1_out_0_grp/pwm1_out	uart2_rts_out_0_grp/uart2_rts_out	can0_stby_out_2_grp/can0_stby_out	i2s0_sdo1_out_0_grp/i2s0_sdo1_out	cpu_trace_data2_out_0_grp/cpu_trace_data2_out	can1_stby_out_2_grp/can1_stby_out
ETH_TXD3	eth_txd_3_out_grp/eth_txd_3_out	gpio29_0_grp/gpio29	gpio13_0_grp/gpio13	safety_error_n_out_1_grp/safety_error_n_out	can0_tx_out_2_grp/can0_tx_out	can1_tx_out_2_grp/can1_tx_out	cpu_trace_data3_out_0_grp/cpu_trace_data3_out		
I2S0_SCK	i2s0_sck_0_grp/i2s0_sck	gpio30_0_grp/gpio30	gpio14_0_grp/gpio14	isp_flash_trig_out_0_grp/isp_flash_trig_out	safety_out_0_0_grp/safety_out_0	timer_ext_0_in_0_grp/timer_ext_0_in			
I2S0_SDI	i2s0_sdi0_in_0_grp/i2s0_sdi0_in	gpio31_0_grp/gpio31	gpio15_0_grp/gpio15	uart2_rxd_in_0_grp/uart2_rxd_in	usb_overcurrent_n_in_0_grp/usb_overcurrent_n_in	isp_pre_flash_out_0_grp/isp_pre_flash_out	safety_out_1_0_grp/safety_out_1	timer_ext_1_in_0_grp/timer_ext_1_in	
I2S0_SDO	i2s0_sdo0_out_0_grp/i2s0_sdo0_out	gpio16_1_grp/gpio16	gpio0_1_grp/gpio0	uart2_txd_out_0_grp/uart2_txd_out	usb_drive_vbus_out_0_grp/usb_drive_vbus_out	cpu_trace_clk_out_0_grp/cpu_trace_clk_out			
I2S0_WS	i2s0_ws_0_grp/i2s0_ws	gpio17_1_grp/gpio17	gpio1_1_grp/gpio1						
FLASH_CS0_N	flash_spi_cs_0_n_out_grp/flash_spi_cs_0_n_out	gpio18_1_grp/gpio18	gpio2_1_grp/gpio2	isp_flash_trig_out_1_grp/isp_flash_trig_out	debug_out22_grp/debug_out22	can0_stby_out_3_grp/can0_stby_out	spi1_cs0_n_out_0_grp/spi1_cs0_n_out	spi0_cs0_n_out_0_grp/spi0_cs0_n_out	
FLASH_CS1_N	flash_spi_cs_1_n_out_grp/flash_spi_cs_1_n_out	gpio19_1_grp/gpio19	gpio3_1_grp/gpio3	isp_pre_flash_out_1_grp/isp_pre_flash_out	debug_out23_grp/debug_out23	uart1_rts_out_1_grp/uart1_rts_out	usb_drive_vbus_out_1_grp/usb_drive_vbus_out	spi1_cs1_n_out_0_grp/spi1_cs1_n_out	spi0_cs1_n_out_0_grp/spi0_cs1_n_out
FLASH_CS2_N	flash_spi_cs2n_clkc_rebar_out_grp/flash_spi_cs2n_clkc_rebar_out	can0_tx_out_3_grp/can0_tx_out	can1_tx_out_3_grp/can1_tx_out	gpio20_1_grp/gpio20	gpio4_1_grp/gpio4	uart3_txd_out_0_grp/uart3_txd_out	cpu_trace_data2_out_1_grp/cpu_trace_data2_out	spi1_mosi_out_0_grp/spi1_mosi_out	spi0_cs2_n_out_0_grp/spi0_cs2_n_out

FLASH_DS	flash_spi_ds_cs3_n_lpbk_grp/ flash_spi_ds_cs3_n_lpbk	can0_rx_in_3_grp/can0_rx_in	can1_rx_in_3_grp/can1_rx_in	gpio21_1_grp/gpio21	gpio5_1_grp/gpio5	uart3_rxd_in_0_grp/uart3_rxd_in	cpu_trace_data3_out_1_grp/cpu_trace_data3_out	spi1_sclk_out_0_grp/ spi1_sclk_out	spi0_cs3_n_out_0_grp/spi0_cs3_n_out
FLASH_DQ0_MOSI	flash_spi_dq_0_grp/flash_spi_dq_0	gpio22_1_grp/gpio22	gpio6_1_grp/gpio6	sdio0_gp_in_0_grp/sdio0_gp_in	sdio0_CD_in_0_grp/sdio0_CD_in	uart0_cts_in_1_grp/uart0_cts_in	spi1_miso_in_0_grp/spi1_miso_in	spi0_mosi_out_0_grp/spi0_mosi_out	
FLASH_DQ1_MISO	flash_spi_dq_1_grp/flash_spi_dq_1	gpio23_1_grp/gpio23	gpio7_1_grp/gpio7	sdio1_gp_in_0_grp/sdio1_gp_in	sdio1_CD_in_0_grp/sdio1_CD_in	uart0_rts_out_1_grp/uart0_rts_out	spi0_miso_in_0_grp/spi0_miso_in		
FLASH_DQ2	flash_spi_dq_2_grp/flash_spi_dq_2	gpio24_1_grp/gpio24	gpio8_1_grp/gpio8	pwm0_out_1_grp/pwm0_out	i2c0_current_src_en_out_0_grp/i2c0_current_src_en_out	uart1_cts_in_1_grp/uart1_cts_in	sdio0_CD_in_1_grp/sdio0_CD_in	spi0_sclk_out_0_grp/ spi0_sclk_out	
FLASH_DQ3	flash_spi_dq_3_grp/flash_spi_dq_3	gpio25_1_grp/gpio25	gpio9_1_grp/gpio9	pwm1_out_1_grp/pwm1_out	i2c1_current_src_en_out_0_grp/i2c1_current_src_en_out	uart1_rts_out_2_grp/uart1_rts_out	sdio1_CD_in_1_grp/sdio1_CD_in		
FLASH_RESET_N	flash_spi_reset_n_out_grp/ flash_spi_reset_n_out	gpio26_1_grp/gpio26	gpio10_1_grp/gpio10	sdio0_gp_out_0_grp/sdio0_gp_out	can0_stby_out_4_grp/can0_stby_out	pwm1_out_2_grp/pwm1_out	can1_stby_out_3_grp /can1_stby_out		
FLASH_SCLK	flash_spi_sclk_out_grp /flash_spi_sclk_out	gpio27_1_grp/gpio27	gpio11_1_grp/gpio11	sdio1_gp_out_0_grp/sdio1_gp_out	pwm0_out_2_grp/pwm0_out				
PCIE_WAKE_N	pcie_wake_n_grp /pcie_wake_n	flash_ospi_dq_4_grp/flash_ospi_dq_4	gpio23_2_grp/gpio23	gpio7_2_grp/gpio7	debug_out24_grp/debug_out24	uart2_rxd_in_1_grp/uart2_rxd_in	sdio0_gp_in_1_grp/sdio0_gp_in		
PCIE_CLKREQ_N	pcie_clkreq_n_grp/pcie_clkreq_n	flash_ospi_dq_5_grp/flash_ospi_dq_5	gpio24_2_grp/gpio24	gpio10_2_grp/gpio10	debug_out25_grp/debug_out25	uart2_txd_out_1_grp/ uart2_txd_out	sdio0_gp_out_1_grp/sdio0_gp_out	cpu_trace_data0_out_1_grp/cpu_trace_data0_out	

PCIE_PERST_N	pcie_perst_n_in_grp/pcie_perst_n_in	flash_ospi_dq_6_grp/flash_ospi_dq_6	gpio25_2_grp/gpio25	gpio11_2_grp/gpio11	debug_out26_grp/debug_out26	uart3_rxd_in_1_grp/uart3_rxd_in	sdio0_CD_in_2_grp/sdio0_CD_in	cpu_trace_data1_out_1_grp/cpu_trace_data1_out	
PCIE_MPERST_N	pcie_mperst_n_out_grp/ pcie_mperst_n_out	flash_ospi_dq_7_grp/flash_ospi_dq_7	gpio26_2_grp/gpio26	gpio9_2_grp/gpio9	debug_out27_grp/debug_out27	uart3_txd_out_1_grp/ uart3_txd_out	can0_stby_out_5_grp /can0_stby_out	cpu_trace_data2_out_2_grp/cpu_trace_data2_out	can1_stby_out_4_grp/can1_stby_out
SDIO0_DATA0_SDIO1_DATA4	SDIO0_DATA0_SDIO1_DATA4_grp / SDIO0_DATA0_SDIO1_DATA4	gpio28_1_grp/gpio28	gpio12_1_grp/gpio12	i2c2_scl_1_grp/i2c2_scl	uart0_rxd_in_1_grp/uart0_rxd_in	uart2_rxd_in_2_grp/uart2_rxd_in	i2s1_sck_0_grp/i2s1_sck		
SDIO0_DATA1_SDIO1_DATA5	SDIO0_DATA1_SDIO1_DATA5_grp / SDIO0_DATA1_SDIO1_DATA5	gpio29_1_grp/gpio29	gpio13_1_grp/gpio13	i2c2_sda_0_grp/i2c2_sda	uart0_txd_out_1_grp/uart0_txd_out	uart2_txd_out_2_grp/ uart2_txd_out	i2s1_sdi0_in_0_grp/i2s1_sdi0_in		
SDIO0_DATA2_SDIO1_DATA6	SDIO0_DATA2_SDIO1_DATA6_grp / SDIO0_DATA2_SDIO1_DATA6	gpio30_1_grp/gpio30	gpio14_1_grp/gpio14	i2c3_scl_0_grp/i2c3_scl	uart1_rxd_in_1_grp/uart1_rxd_in	uart3_rxd_in_2_grp/uart3_rxd_in	i2s1_sdo0_out_0_grp /i2s1_sdo0_out		
SDIO0_DATA3_SDIO1_DATA7	SDIO0_DATA3_SDIO1_DATA7_grp / SDIO0_DATA3_SDIO1_DATA7	gpio31_1_grp/gpio31	gpio15_1_grp/gpio15	i2c3_sda_0_grp/i2c3_sda	uart1_txd_out_1_grp/uart1_txd_out	uart3_txd_out_2_grp/ uart3_txd_out	i2s1_ws_0_grp/i2s1_ws	i2s0_mclk_0_grp/i2s0_mclk	
SDIO0_SDCLK_SDIO1_DS	SDIO0_SDCLK_SDIO1_DS_grp/ SDIO0_SDCLK_SDIO1_DS	gpio4_2_grp/gpio4	i2s0_sdi1_in_1_grp/i2s0_sdi1_in	uart0_cts_in_2_grp/uart0_cts_in	uart1_cts_in_2_grp/uart1_cts_in	uart3_cts_in_0_grp/uart3_cts_in	i2s1_mclk_0_grp/i2s1_mclk		
SDIO0_CMD_SDIO1_RSTN	SDIO0_CMD_SDIO1_RSTN_grp/ SDIO0_CMD_SDIO1_RSTN	gpio5_2_grp/gpio5	i2s0_sdo1_out_1_grp/i2s0_sdo1_out	uart0_rts_out_2_grp/uart0_rts_out	uart1_rts_out_3_grp/uart1_rts_out	uart3_rts_out_0_grp/uart3_rts_out			
SDIO1_DATA0	SDIO1_DATA0_grp/SDIO1_DATA0	gpio8_2_grp/gpio8	uart2_rxd_in_3_grp/uart2_rxd_in	i2c2_scl_2_grp/i2c2_scl	i2s1_sck_1_grp/i2s1_sck	uart0_rxd_in_2_grp/uart0_rxd_in			

SDIO1_DATA1	SDIO1_DATA1_grp/SDIO1_DATA1	gpio9_3_grp/gpio9	uart2_txd_out_3_grp/uart2_txd_out	i2c2_sda_1_grp/i2c2_sda	i2s1_sdi0_in_1_grp/i2s1_sdi0_in	uart0_txd_out_2_grp/uart0_txd_out			
SDIO1_DATA2	SDIO1_DATA2_grp/SDIO1_DATA2	gpio20_2_grp/gpio20	uart3_rxd_in_3_grp/uart3_rxd_in	i2c3_scl_1_grp/i2c3_scl	i2s1_sdo0_out_1_grp/i2s1_sdo0_out	uart1_rxd_in_2_grp/uart1_rxd_in			
SDIO1_DATA3	SDIO1_DATA3_grp/SDIO1_DATA3	gpio21_2_grp/gpio21	uart3_txd_out_3_grp/uart3_txd_out	i2c3_sda_1_grp/i2c3_sda	i2s1_ws_1_grp/i2s1_ws	uart1_txd_out_2_grp/uart1_txd_out			
SDIO1_SDCLK	SDIO1_SDCLK_grp/SDIO1_SDCLK	gpio25_3_grp/gpio25	gpio9_4_grp/gpio9						
PARALLEL_PCLK	parallel_pclk_in_grp/parallel_pclk_in	uart1_cts_in_3_grp/uart1_cts_in	uart2_cts_in_1_grp/uart2_cts_in	i2c2_current_src_en_out_0_grp/i2c2_current_src_en_out	debug_out28_grp/debug_out28	isp_flash_trig_out_2_grp/isp_flash_trig_out	gpio0_2_grp/gpio0	i2s1_mclk_1_grp/i2s1_mclk	i2s0_mclk_1_grp/i2s0_mclk
USB_DRIVE_VBUS	usb_drive_vbus_out_2_grp/usb_drive_vbus_out	uart1_rts_out_4_grp/uart1_rts_out	uart2_rts_out_1_grp/uart2_rts_out	i2c3_current_src_en_out_0_grp/i2c3_current_src_en_out	debug_out29_grp/debug_out29	isp_pre_flash_out_2_grp/isp_pre_flash_out	gpio10_3_grp/gpio10		
JTAG_TCK	jtag_tck_in_grp/jtag_tck_in	gpio16_2_grp/gpio16	gpio0_3_grp/gpio0	timer_ext_0_in_1_grp/timer_ext_0_in	uart3_cts_in_1_grp/uart3_cts_in	uart2_rxd_in_4_grp/uart2_rxd_in	i2c2_scl_3_grp/i2c2_scl	i2s1_sck_2_grp/i2s1_sck	
JTAG_TDI	jtag_tdi_in_grp/jtag_tdi_in	gpio17_2_grp/gpio17	gpio1_2_grp/gpio1	timer_ext_1_in_1_grp/timer_ext_1_in	uart3_rts_out_1_grp/uart3_rts_out	uart2_txd_out_4_grp/uart2_txd_out	i2c2_sda_2_grp/i2c2_sda	i2s1_sdi0_in_2_grp/i2s1_sdi0_in	
JTAG_TDO	jtag_tdo_out_grp/jtag_tdo_out	gpio18_2_grp/gpio18	gpio2_2_grp/gpio2	pwm2_out_0_grp/pwm2_out	safety_out_0_1_grp/safety_out_0	uart3_txd_out_4_grp/uart3_txd_out	i2c3_scl_2_grp/i2c3_scl	i2s1_sdo0_out_2_grp/i2s1_sdo0_out	

JTAG_TMS	jtag_tms_grp/jtag_tms	gpio19_2_grp/gpio19	gpio3_2_grp/gpio3	timer_ext_3_in_0_grp/timer_ext_3_in	safety_out_1_1_grp/safety_out_1	uart3_rxd_in_4_grp/uart3_rxd_in	i2c3_sda_2_grp/i2c3_sda	i2s1_ws_2_grp/i2s1_ws	
JTAG_TRSTN	jtag_trstn_in_grp/jtag_trstn_in	gpio22_2_grp/gpio22	gpio6_2_grp/gpio6	safety_error_n_out_2_grp/safety_error_n_out	timer_ext_2_in_0_grp/timer_ext_2_in	i2s1_mclk_2_grp/i2s1_mclk			
GPIO_0	gpio0_4_grp/gpio0	eth_txd_4_out_grp/eth_txd_4_out	pwm0_out_3_grp/pwm0_out	debug_out0_grp/debug_out0	i2c0_current_src_en_out_1_grp/i2c0_current_src_en_out	can0_stby_out_6_grp/can0_stby_out	can1_stby_out_5_grp/can1_stby_out	sdio0_uhs_sel_out_0_grp/sdio0_uhs_sel_out	sdio1_uhs_sel_out_0_grp/sdio1_uhs_sel_out
GPIO_1	gpio1_3_grp/gpio1	eth_txd_5_out_grp/eth_txd_5_out	pwm1_out_3_grp/pwm1_out	debug_out1_grp/debug_out1	i2c1_current_src_en_out_1_grp/i2c1_current_src_en_out	usb_overcurrent_n_in_1_grp/usb_overcurrent_n_in	uart0_rts_out_3_grp/uart0_rts_out	can1_rx_in_4_grp/can1_rx_in	can1_stby_out_6_grp/can1_stby_out
GPIO_2	gpio2_3_grp/gpio2	eth_txd_6_out_grp/eth_txd_6_out	pwm2_out_1_grp/pwm2_out	debug_out2_grp/debug_out2	uart2_rxd_in_5_grp/uart2_rxd_in	usb_overcurrent_n_in_2_grp/usb_overcurrent_n_in	safety_error_n_out_3_grp/safety_error_n_out	uart0_cts_in_3_grp/uart0_cts_in	spi2_cs0_n_out_grp/spi2_cs0_n_out
GPIO_3	gpio3_3_grp/gpio3	eth_txd_7_out_grp/eth_txd_7_out	pwm3_out_0_grp/pwm3_out	debug_out3_grp/debug_out3	uart2_txd_out_5_grp/uart2_txd_out	can0_rx_in_4_grp/can0_rx_in	sdio1_vdd1_on_out_0_grp/sdio1_vdd1_on_out	uart0_rts_out_4_grp/uart0_rts_out	spi2_cs1_n_out_grp/spi2_cs1_n_out
GPIO_4	gpio4_3_grp/gpio4	eth_gmii_tx_er_out_grp/eth_gmii_tx_er_out	pwm4_out_0_grp/pwm4_out	debug_out4_grp/debug_out4	usb_drive_vbus_out_3_grp/usb_drive_vbus_out	uart1_rts_out_5_grp/uart1_rts_out	safety_out_0_2_grp/safety_out_0	can0_stby_out_7_grp/can0_stby_out	spi2_sclk_out_grp/spi2_sclk_out
GPIO_5	boot_rom_failure_out_grp/boot_rom_failure_out	gpio5_3_grp/gpio5	pwm5_out_0_grp/pwm5_out	debug_out5_grp/debug_out5	can0_tx_out_4_grp/can0_tx_out	uart2_rts_out_2_grp/uart2_rts_out	safety_out_1_2_grp/safety_out_1	can1_tx_out_4_grp/can1_tx_out	spi2_mosi_out_grp/spi2_mosi_out
GPIO_6	gpio6_3_grp/gpio6	eth_rxd_4_in_grp/eth_rxd_4_in	sdio0_gp_in_2_grp/sdio0_gp_in	debug_out6_grp/debug_out6	i2c3_scl_3_grp/i2c3_scl	uart0_cts_in_4_grp/uart0_cts_in	sdio0_vdd1_on_out_0_grp/sdio0_vdd1_on_out	parallel0_in_grp/parallel0_in	spi2_miso_in_grp/spi2_miso_in

GPIO_7	sdio0_CD_in_3_grp/sdio0_CD_in	eth_rxd_5_in_grp/eth_rxd_5_in	gpio7_3_grp/gpio7	debug_out7_grp/debug_out7	i2c3_sda_3_grp/i2c3_sda	uart1_cts_in_4_grp/uart1_cts_in	timer_ext_0_in_2_grp/timer_ext_0_in	parallel1_in_grp/parallel1_in	sdio0_vdd1_on_out_1_grp/sdio0_vdd1_on_out
GPIO_8	gpio8_3_grp/gpio8	eth_rxd_6_in_grp/eth_rxd_6_in	sdio1_gp_in_1_grp/sdio1_gp_in	debug_out8_grp/debug_out8	isp_flash_trig_out_3_grp/isp_flash_trig_out	uart2_cts_in_2_grp/uart2_cts_in	timer_ext_1_in_2_grp/timer_ext_1_in	parallel2_in_grp/parallel2_in	sdio0_host_vdd1_stable_in_grp/sdio0_host_vdd1_stable_in
GPIO_9	sdio1_CD_in_2_grp/sdio1_CD_in	eth_rxd_7_in_grp/eth_rxd_7_in	gpio9_5_grp/gpio9	debug_out9_grp/debug_out9	safety_error_n_out_4_grp/safety_error_n_out	uart2_rts_out_3_grp/uart2_rts_out	timer_ext_2_in_1_grp/timer_ext_2_in	parallel3_in_grp/parallel3_in	sdio1_vdd1_on_out_1_grp/sdio1_vdd1_on_out
GPIO_10	gpio10_4_grp/gpio10	gmii_rx_er_in_grp/gmii_rx_er_in	debug_out10_grp/debug_out10	i2s0_sdi1_in_2_grp/i2s0_sdi1_in	sdio0_wp_in_grp/sdio0_wp_in	timer_ext_3_in_1_grp/timer_ext_3_in	can0_rx_in_5_grp/can0_rx_in	parallel4_in_grp/parallel4_in	spi1_cs0_n_out_1_grp/spi1_cs0_n_out
GPIO_11	gpio11_3_grp/gpio11	gmii_rx_cs_in_grp/gmii_rx_cs_in	debug_out11_grp/debug_out11	i2s0_sdo1_out_2_grp/i2s0_sdo1_out	sdio1_wp_in_0_grp/sdio1_wp_in	sdio1_CD_in_3_grp/sdio1_CD_in	can0_tx_out_5_grp/can0_tx_out	parallel5_in_grp/parallel5_in	spi1_cs1_n_out_1_grp/spi1_cs1_n_out
GPIO_12	gpio12_2_grp/gpio12	mii_tx_clk_in_grp/mii_tx_clk_in	debug_out12_grp/debug_out12	i2s0_sck_1_grp/i2s0_sck	sdio0_gp_out_2_grp/sdio0_gp_out	uart3_cts_in_2_grp/uart3_cts_in	i2s1_sck_3_grp/i2s1_sck	parallel6_in_grp/parallel6_in	spi1_mosi_out_1_grp/spi1_mosi_out
GPIO_13	gpio13_2_grp/gpio13	gmii_rx_col_in_grp/gmii_rx_col_in	debug_out13_grp/debug_out13	i2s0_sdi0_in_1_grp/i2s0_sdi0_in	safety_error_n_out_5_grp/safety_error_n_out	uart3_rts_out_2_grp/uart3_rts_out	i2s1_sdi0_in_3_grp/i2s1_sdi0_in	parallel7_in_grp/parallel7_in	spi1_miso_in_1_grp/spi1_miso_in
GPIO_14	gpio14_2_grp/gpio14	cpu_trace_clk_out_1_grp/cpu_trace_clk_out	debug_out14_grp/debug_out14	i2s0_sdo0_out_1_grp/i2s0_sdo0_out	can0_stby_out_8_grp/can0_stby_out	uart2_cts_in_3_grp/uart2_cts_in	i2s1_sdo0_out_3_grp/i2s1_sdo0_out	parallel8_in_grp/parallel8_in	spi1_sclk_out_1_grp/spi1_sclk_out
GPIO_15	gpio15_2_grp/gpio15	isp_pre_flash_out_3_grp/isp_pre_flash_out	debug_out15_grp/debug_out15	i2s0_ws_1_grp/i2s0_ws	safety_out_1_3_grp/safety_out_1	uart2_rts_out_4_grp/uart2_rts_out	i2s1_ws_3_grp/i2s1_ws	parallel9_in_grp/parallel9_in	sdio0_vdd1_on_out_2_grp/sdio0_vdd1_on_out

GPIO_16	gpio16_3_grp/gpio16	i2c2_scl_4_grp/i2c2_scl	uart3_cts_in_3_grp/uart3_cts_in	safety_out_0_3_grp/safety_out_0	sdio0_CD_in_4_grp/sdio0_CD_in	uart1_cts_in_5_grp/uart1_cts_in	i2s1_mclk_3_grp/i2s1_mclk	parallel10_in_grp/parallel10_in	sdio1_vdd1_on_out_2_grp/sdio1_vdd1_on_out
GPIO_17	gpio17_3_grp/gpio17	i2c2_sda_3_grp/i2c2_sda	uart3_rts_out_3_grp/uart3_rts_out	safety_out_1_4_grp/safety_out_1	sdio1_CD_in_4_grp/sdio1_CD_in	uart1_rts_out_6_grp/uart1_rts_out	i2s0_mclk_2_grp/i2s0_mclk	parallel11_in_grp/parallel11_in	sdio1_host_vdd1_stable_in_grp/sdio1_host_vdd1_stable_in
GPIO_18	gpio18_3_grp/gpio18	i2c3_scl_4_grp/i2c3_scl	uart2_cts_in_4_grp/uart2_cts_in	cpu_trace_clk_out_2_grp/cpu_trace_clk_out	i2c0_current_src_en_out_2_grp/i2c0_current_src_en_out	i2s0_sdi1_in_3_grp/i2s0_sdi1_in	can0_rx_in_6_grp/can0_rx_in	parallel12_in_grp/parallel12_in	uart2_rxd_in_6_grp/uart2_rxd_in
GPIO_19	gpio19_3_grp/gpio19	i2c3_sda_4_grp/i2c3_sda	uart2_rts_out_5_grp/uart2_rts_out	isp_pre_flash_out_4_grp/isp_pre_flash_out	i2c1_current_src_en_out_2_grp/i2c1_current_src_en_out	i2s0_sdo1_out_3_grp/i2s0_sdo1_out	can0_tx_out_6_grp/can0_tx_out	parallel13_in_grp/parallel13_in	uart2_txd_out_6_grp/uart2_txd_out
GPIO_20	gpio20_3_grp/gpio20	uart2_rxd_in_7_grp/uart2_rxd_in	pwm0_out_4_grp/pwm0_out	i2c0_current_src_en_out_3_grp/i2c0_current_src_en_out	debug_out16_grp/debug_out16	sdio0_gp_in_3_grp/sdio0_gp_in	i2s1_sck_4_grp/i2s1_sck	parallel14_in_grp/parallel14_in	spi0_cs0_n_out_1_grp/spi0_cs0_n_out
GPIO_21	gpio21_3_grp/gpio21	uart2_txd_out_7_grp/uart2_txd_out	pwm1_out_4_grp/pwm1_out	i2c1_current_src_en_out_3_grp/i2c1_current_src_en_out	debug_out17_grp/debug_out17	sdio0_CD_in_5_grp/sdio0_CD_in	i2s1_sdi0_in_4_grp/i2s1_sdi0_in	parallel15_in_grp/parallel15_in	spi0_cs1_n_out_1_grp/spi0_cs1_n_out
GPIO_22	gpio22_3_grp/gpio22	uart3_rxd_in_5_grp/uart3_rxd_in	pwm2_out_2_grp/pwm2_out	i2c2_current_src_en_out_1_grp/i2c2_current_src_en_out	debug_out18_grp/debug_out18	sdio1_gp_in_2_grp/sdio1_gp_in	i2s1_sdo0_out_4_grp/i2s1_sdo0_out	parallel16_in_grp/parallel16_in	spi0_cs2_n_out_1_grp/spi0_cs2_n_out
GPIO_23	gpio23_3_grp/gpio23	uart3_txd_out_5_grp/uart3_txd_out	pwm3_out_1_grp/pwm3_out	i2c3_current_src_en_out_1_grp/i2c3_current_src_en_out	debug_out19_grp/debug_out19	sdio1_CD_in_5_grp/sdio1_CD_in	i2s1_ws_4_grp/i2s1_ws	parallel17_in_grp/parallel17_in	spi0_cs3_n_out_1_grp/spi0_cs3_n_out
GPIO_24	gpio24_3_grp/gpio24	uart0_cts_in_5_grp/uart0_cts_in	pwm4_out_1_grp/pwm4_out	cpu_trace_data0_out_2_grp/cpu_trace_data0_out	debug_out20_grp/debug_out20	sdio0_vdd1_on_out_3_grp/sdio0_vdd1_on_out	can0_rx_in_7_grp/can0_rx_in	parallel18_in_grp/parallel18_in	spi0_mosi_out_1_grp/spi0_mosi_out

GPIO_25	gpio25_4_grp/gpio25	uart0_rts_out_5_grp/uart0_rts_out	pwm5_out_1_grp/pwm5_out	cpu_trace_data1_out_2_grp/cpu_trace_data1_out	debug_out21_grp/debug_out21	sdio1_wp_in_1_grp/sdio1_wp_in	can0_tx_out_7_grp/can0_tx_out	parallel19_in_grp/parallel19_in	spi0_miso_in_1_grp/spi0_miso_in
GPIO_26	gpio26_3_grp/gpio26	pwm0_out_5_grp/pwm0_out	sdio0_gp_out_3_grp/sdio0_gp_out	i2s0_sdi1_in_4_grp/i2s0_sdi1_in	cpu_trace_data2_out_3_grp/cpu_trace_data2_out	debug_out30_grp/debug_out30	uart1_cts_in_6_grp/uart1_cts_in	parallel20_in_grp/parallel20_in	spi0_sclk_out_1_grp/spi0_sclk_out
GPIO_27	gpio27_2_grp/gpio27	pwm1_out_5_grp/pwm1_out	sdio1_gp_out_1_grp/sdio1_gp_out	i2s0_sdo1_out_4_grp/i2s0_sdo1_out	cpu_trace_data3_out_2_grp/cpu_trace_data3_out	debug_out31_grp/debug_out31	uart1_rts_out_7_grp/uart1_rts_out	parallel21_in_grp/parallel21_in	can1_stby_out_7_grp/can1_stby_out
GPIO_28	gpio28_2_grp/gpio28	can0_rx_in_8_grp/can0_rx_in	i2s1_sck_5_grp/i2s1_sck	can0_stby_out_9_grp/can0_stby_out	usb_overcurrent_n_in_3_grp/usb_overcurrent_n_in	spi3_cs0_n_in_grp/spi3_cs0_n_in	uart2_rxd_in_8_grp/uart2_rxd_in	parallel22_in_grp/parallel22_in	can1_rx_in_5_grp/can1_rx_in
GPIO_29	gpio29_2_grp/gpio29	can0_tx_out_8_grp/can0_tx_out	i2s1_sdi0_in_5_grp/i2s1_sdi0_in	usb_overcurrent_n_in_4_grp/usb_overcurrent_n_in	usb_drive_vbus_out_4_grp/usb_drive_vbus_out	spi3_mosi_in_grp/spi3_mosi_in	uart2_txd_out_8_grp/uart2_txd_out	parallel23_in_grp/parallel23_in	can1_tx_out_5_grp/can1_tx_out
GPIO_30	gpio30_2_grp/gpio30	can0_rx_in_9_grp/can0_rx_in	i2s1_sdo0_out_5_grp/i2s1_sdo0_out	can0_stby_out_10_grp/can0_stby_out	pwm0_out_6_grp/pwm0_out	spi3_miso_out_grp/spi3_miso_out	sdio0_uhs_sel_out_1_grp/sdio0_uhs_sel_out	parallel_hsync_in_grp/parallel_hsync_in	sdio1_uhs_sel_out_1_grp/sdio1_uhs_sel_out
GPIO_31	gpio31_2_grp/gpio31	can0_tx_out_9_grp/can0_tx_out	i2s1_ws_5_grp/i2s1_ws	usb_overcurrent_n_in_5_grp/usb_overcurrent_n_in	pwm1_out_6_grp/pwm1_out	spi3_sclk_in_grp/spi3_sclk_in	sdio1_uhs_sel_out_2_grp/sdio1_uhs_sel_out	parallel_vsync_in_grp/parallel_vsync_in	sdio0_uhs_sel_out_2_grp/sdio0_uhs_sel_out